



进阶篇之-

基于 RK 平台的 AI 模型训练到部署

Embedded Learning Fans Board

Instruction Manual

Rev. 1.1

2025/11/25



修 订

日期	手册版本	更新内容
2024.11.09	V1.0	初版
2025.11.25	V1.1	RK 平台通用化调整



适用范围

本文档适用于本公司的部分产品，这些平台使用的 **NPU** 驱动以及相关工具版本有所差异，但操作方法相同，以下内容将以现以 **ELF 2_Lite (RK3576)** 为例展开讲解。

下表列出了各平台对应的 **NPU** 驱动 版本及经测试适配的 **RKNN** 工具版本，建议优先使用表中指定的 **RKNN** 工具版本进行操作。

平台	NPU 版本	RKNN-Toolkit2 版本	RKNN-Toolkit-Lite2 版本
ELF 2 (RK3588)	0.9.6	2.1.0	2.1.0
ELF 2_Lite (RK3576)	0.9.8	2.3.2	2.3.2
ELF-RV1126B	0.9.8	2.3.2	2.3.2



目 录

修 订	- 1 -
适用范围	- 2 -
目 录	- 3 -
第一章 概述	- 5 -
第二章 深度学习框架与经典模型	- 6 -
2.1. 深度学习框架	- 6 -
2.1.1. PyTorch 框架	- 6 -
2.1.2. Tensor Flow 框架	- 7 -
2.2. 经典模型的介绍	- 7 -
2.2.1. ResNet	- 7 -
2.2.2. Mobilenet	- 8 -
2.2.3. YOLOv5	- 9 -
2.2.4. YOLOv8	- 9 -
2.2.5. LPRNet	- 10 -
2.2.6. PPOCR	- 11 -
2.2.7. DeepLab	- 11 -
2.2.8. RetinaFace	- 12 -
2.3. RK 平台适用的深度学习框架及模型	- 13 -
第三章 数据收集与处理	- 14 -
3.1. 数据收集	- 14 -
3.2. 数据标注	- 14 -
3.3. 图像数据增强	- 14 -
3.3.1. 常见方法	- 15 -
3.3.2. 注意事项	- 15 -
3.4. 数据集划分	- 15 -
3.4.1. 划分方法	- 15 -
3.4.2. 注意事项	- 15 -
第四章 模型训练与导出	- 16 -
4.1. 项目文件介绍	- 16 -
4.2. 主要文件介绍	- 17 -
4.3. 模型评估的指标	- 18 -
第五章 模型转换与部署	- 20 -
5.1. RKNN 工具介绍	- 20 -
5.1.1. RKNN-Toolkit2	- 20 -
5.1.2. RKNN-Toolkit-Lite2	- 21 -
5.1.3. RKNN Model Zoo	- 21 -
5.2. 转换过程	- 22 -
5.3. 模型推理	- 23 -
第六章 基于 YOLOv8 的目标识别实战项目	- 25 -
6.1. 模型训练与导出	- 25 -
6.1.1. 环境搭建	- 25 -
6.1.1.1. Anaconda 安装	- 25 -
6.1.1.2. Labellmg 安装	- 31 -
6.1.1.3. PyCharm 安装	- 31 -
6.1.2. 数据收集与处理	- 34 -
6.1.2.1. 数据集获取和标注方法	- 34 -
6.1.2.2. 数据集划分	- 39 -
6.1.3. 模型训练	- 40 -
6.1.3.1. 环境配置	- 40 -



6.1.3.2. 训练模型	- 42 -
6.1.4. 模型导出	- 46 -
6.2. 模型转换	- 47 -
6.2.1. 环境搭建	- 48 -
6.2.1.1. 安装 Python 环境	- 48 -
6.2.1.2. 安装 Miniforge 工具	- 48 -
6.2.1.3. 创建 RKNN-Toolkit2 Conda 环境	- 48 -
6.2.1.4. 安装 RKNN-Toolkit2	- 48 -
6.2.1.5. 安装 RKNN Model Zoo	- 49 -
6.2.2. 模型转换	- 49 -
6.3. 模型部署	- 50 -
6.3.1. RKNN-Toolkit-Lite2 安装	- 50 -
6.3.2. 编写推理代码	- 51 -
6.3.3. 板端运行效果	- 54 -
第七章 FAQ	- 56 -
声明	- 59 -
更多帮助	- 60 -



第一章 概述

随着人工智能技术的不断演进，深度学习作为其核心驱动力，已深度融入生活与产业的方方面面。从智能手机上的面部解锁、自动驾驶汽车的安全行驶，到医疗影像的精准分析、安防监控的智能预警，深度学习技术的广泛应用不仅极大地提升了生活的便利性，也深刻地改变了传统行业的运作模式。这一切的落地，离不开高效的数据处理、成熟的深度学习框架以及不断优化的模型算法。

一、深度学习核心概念与框架基础

深度学习是机器学习的重要分支，通过构建深层次神经网络模型，自动从数据中学习并提取层级特征，进而解决复杂的分类、回归和生成任务。其核心优势在于无需人工设计特征，能通过逐层抽象的表示方法，精准捕捉数据本质规律。

深度学习框架作为构建、训练和部署神经网络模型的基石，提供了丰富的工具和函数库，极大地简化了开发流程。本书将对比 **PyTorch** 和 **TensorFlow** 两大主流框架的优缺点，并深入探讨它们的核心特性，如 **TensorFlow** 的动态图与静态图、张量表示、自动微分机制、高级 API（如 **Keras**）以及分布式训练等。这些框架不仅支持高效的模型训练，还提供了便捷的模型部署工具，使得深度学习模型能够轻松地应用到实际场景中。

二、从数据到嵌入式部署的完整链路

将深度学习模型部署到嵌入式设备，是一个贯穿数据处理、模型训练、硬件适配的系统工程。其核心流程围绕“数据准备 — 模型构建 — 硬件落地”三大核心环节展开，整个过程大致可以分为以下几个步骤：

1. 数据收集与处理

该阶段是模型训练的基础，核心目标是为训练提供高质量、多样化的数据源。首先进行数据收集，根据具体任务场景采集足量相关数据样本；随后开展数据预处理工作，通过清洗、去重、数据增强等操作，剔除无效或干扰数据，提升数据质量，同时丰富数据多样性，确保数据能够满足后续模型训练的需求，为模型性能打下良好基础。

2. 模型训练与导出

此阶段围绕模型构建、训练优化及格式导出展开。根据具体任务需求，选取适当的神经网络架构（如 **CNN**、**RNN** 或 **Transformer** 变体），并设计相应的损失函数与评估指标。采用迭代训练范式，通过反向传播算法优化模型参数，同时实施超参数调优与正则化策略以增强模型泛化能力。训练完成后，需通过严格的交叉验证与消融实验评估模型性能，最终将优化后的模型导出为标准化中间表示格式（如 **ONNX**），确保模型的可移植性与框架兼容性。

3. 模型转换与部署

该阶段实现模型从训练环境到嵌入式设备的落地应用，分为转换量化和部署验证两大环节。在模型转换与量化方面，首先利用 **RKNN-Toolkit2** 等专用工具，将导出的标准格式模型转换为嵌入式设备支持的专用格式（如瑞芯微的 **.rknn** 格式）；其次根据设备性能需求选择 **8 位整数量化**、**16 位浮点量化** 等方式进行模型量化处理，大幅减小模型体积和推理时间；同时可结合模型剪枝、知识蒸馏等技术，进一步降低模型计算复杂度，提升在嵌入式设备上的运行效率。

三、本书核心亮点

本书立足“理论 + 实战”双导向，以计算机视觉技术为核心，系统梳理深度学习落地的关键环节。全书先夯实深度学习框架与经典模型的理论基础，再详解数据收集处理、模型训练导出、转换部署的完整流程，最终通过 **YOLOv8** 商品识别实战项目实现知识闭环。同时，书中将分享模型训练与部署过程中的实操经验，帮助读者规避常见问题、少走弯路，快速掌握从“理论”到“落地”的核心技能，高效解决嵌入式部署中的各类工程问题。



第二章 深度学习框架与经典模型

在本章节中，我们将介绍常见的深度学习框架与经典模型。深度学习框架作为构建、训练和部署神经网络模型的基石，提供了丰富的工具和函数库，极大地简化了开发流程。同时，我们也将回顾并解析那些在历史长河中留下深刻印记的经典模型，它们不仅是深度学习发展的里程碑，也为后续研究与应用奠定了坚实基础。

2.1. 深度学习框架

深度学习框架是用于构建和训练深度学习模型的工具或库。它们提供了一系列用于定义、训练和评估神经网络的功能和接口。以下是一些常见的深度学习框架：

1. **TensorFlow**: 由 Google 开发的开源框架，广泛用于机器学习和深度学习任务。它支持多种平台，并且拥有强大的社区和丰富的资源。
2. **PyTorch**: 由 Facebook 的 AI 研究实验室开发的开源框架，以其动态计算图的特性而著名，使得模型开发和调试变得更加灵活和直观。
3. **Keras**: 一个高层次的 API，最初独立开发，后来被整合进 TensorFlow 中。它提供了简洁的接口，便于快速构建和训练神经网络。
4. **MXNet**: 由 Apache 基金会开发的开源深度学习框架，具有高效的计算性能和灵活的编程接口，支持多语言绑定。
5. **Caffe**: 由伯克利视觉与学习中心开发，专注于图像分类和卷积神经网络（CNN），以其高效的计算和模块化设计受到欢迎。
6. **Chainer**: 一个灵活的深度学习框架，以动态计算图为特点，允许用户以自然的方式定义和修改计算图。
7. **PaddlePaddle**: 由百度开发的开源深度学习平台，具有多样化的功能和较好的性能，适用于大规模机器学习任务。

这些框架各有特点，可以根据需求选择适合的工具来进行深度学习任务。

2.1.1. PyTorch 框架

PyTorch 是一个广泛使用的开源机器学习库，特别受到深度学习研究者和从业者的青睐。它由 Facebook 的人工智能研究团队（FAIR）开发和维护。PyTorch 提供了一个强大的、灵活的张量（Tensor）库，这些张量类似于 NumPy 的 ndarray，但可以在 GPU 上运行以加速计算。此外，PyTorch 还提供了自动微分系统，使得构建和训练神经网络变得非常简单和直观。

主要特点：

1. **动态图（Dynamic Graphs）**: PyTorch 使用动态图来构建计算图，这意味着图是在运行时构建的，与 TensorFlow 的静态图（Static Graphs）相比，这可以使得代码更加直观，调试更加容易。
2. **易用性和灵活性**: PyTorch 的设计让初学者和专家都能轻松上手。其灵活的编程模型允许快速原型设计和实验。
3. **丰富的 API**: PyTorch 提供了丰富的 API，包括用于构建神经网络、优化器、数据加载等的库。
4. **社区支持**: PyTorch 拥有一个活跃的社区，提供大量的教程、文档和预训练模型，帮助用户快速入门和深入研究。
5. **可扩展性**: PyTorch 的设计使其易于扩展，允许研究人员开发新的神经网络层和自定义的自动微分操作。

基本组件：

1. **张量（Tensors）**: PyTorch 的核心数据结构，类似于 NumPy 的 ndarray，但可以在 GPU 上运行。
2. **自动微分（Autograd）**: PyTorch 的自动微分系统可以自动计算所有梯度，这对于神经网络训练至关重要。
3. **神经网络（nn）**: PyTorch 提供了构建神经网络的模块和函数，如全连接层、卷积层、循环层等。
4. **优化器（Optim）**: PyTorch 提供了多种优化算法，如 SGD、Adam 等，用于更新神经网络的权重。
5. **数据加载（DataLoader）**: PyTorch 提供了高效的数据加载和预处理工具，支持多进程数据加载。

应用领域：

Pytorch 被广泛应用于多个领域，以下是其主要应用方向：

1. **计算机视觉**: 图像分类、目标检测、语义分割、图像生成等任务。
2. **自然语言处理**: 文本分类、机器翻译、大语言模型训练与微调等任务。



3. 语音识别与处理：语音转文本、文本转语音、语音合成、声纹识别等任务。
4. 强化学习：训练智能体在游戏、机器人控制等场景中决策。
5. 工业与边缘部署：结合 ONNX 或 TorchScript 格式，将训练好的模型部署到工业设备、嵌入式系统或移动端，实现实时推理，如智能监控、工业质检。

2.1.2. Tensor Flow 框架

概述：

TensorFlow 是由 Google Brain 团队开发并维护的一个开源机器学习库，广泛应用于深度学习领域。它提供了强大的工具和库来构建和训练各种神经网络模型。

核心特性：

1. 动态图与静态图：TensorFlow 1.x 版本以静态图为核心，计算图在运行时之前构建。TensorFlow 2.x 引入了 Eager Execution（动态图），使得模型开发更加直观和灵活。
2. 张量（Tensors）：TensorFlow 中的基本数据单位是张量，它是一个多维数组或列表，用于存储和表示数据。
3. 自动微分：TensorFlow 提供了自动微分机制，能够自动计算模型参数关于损失函数的梯度，极大地简化了模型训练过程。
4. 高级 API（如 Keras）：TensorFlow 集成了 Keras 作为其高级 API，使得模型的构建、训练和评估更加简洁和高效。
5. 分布式训练：支持在多个计算节点上分布式地训练模型，提高训练速度和扩展性。
6. 可视化工具（TensorBoard）：TensorBoard 是 TensorFlow 的可视化工具，用于展示模型结构、训练过程中的各项指标和可视化数据等。
7. 生态系统：TensorFlow 拥有庞大的社区和丰富的生态系统，包括大量的教程、文档、预训练模型和第三方库。

应用领域：

TensorFlow 在多个领域都有广泛的应用，包括但不限于：

1. 图像识别：用于图像分类、目标检测、图像分割等任务。
2. 自然语言处理：支持文本分类、情感分析、机器翻译等任务。
3. 语音识别：实现语音到文本的转换。
4. 推荐系统：构建个性化推荐模型。
5. 强化学习：与 TensorFlow Agents 等库结合，用于解决复杂的决策问题。

2.2. 经典模型的介绍

2.2.1. ResNet

核心理念：

ResNet 的核心理念是引入残差块（Residual Block），这种块通过直接将输入加到输出上来创建一个“捷径”或“快捷连接”（skip connection）。这种设计使得深层网络能够更容易地训练，并改善了梯度传播问题，减少了梯度消失或梯度爆炸的现象。

残差块：

基本结构：一个残差块包含两个或更多卷积层和一个直接连接（shortcut connection）。快捷连接绕过卷积层，直接将输入加到块的输出上。

具体来说，假设输入为 x ，通过卷积层得到 $F(x)$ （这里 $F(x)$ 是卷积操作后的特征图），那么残差块的输出就是 $F(x) + x$ 。这种设计使得网络能够学习到与输入相关的残差，而不是直接学习目标函数。

公式：残差块的输出 y 可以表示为： $Y = F(x) + x$ 。

网络架构：

ResNet 的网络架构是由多个残差块堆叠而成的。常见的 ResNet 架构有：

1. ResNet-18 和 ResNet-34：较浅的网络，适合资源有限的环境。
2. ResNet-50：引入了 Bottleneck 结构，以减少计算量。
3. ResNet-101 和 ResNet-152：更深的网络，适用于复杂任务和大规模数据集。

Bottleneck 结构：

对于更深的网络，ResNet 引入了 Bottleneck 结构来提高计算效率。Bottleneck 结构通常由三层卷积组成：



1x1 卷积层（降维）：这一层的主要目的是减少输入特征图的通道数。通过应用 1x1 的卷积核，可以将高维的特征图映射到一个较低维度的空间，从而减少后续 3x3 卷积层的计算量。这一步骤通常被称为“降维”或“压缩”。

3x3 卷积层（提取特征）：在降维之后，使用一个 3x3 的卷积核来提取特征。由于此时特征图的通道数已经减少，因此这一层的计算量相对较小。3x3 卷积层是特征提取的核心，它能够在减少计算负担的同时，有效地捕捉输入数据的空间特征。

1x1 卷积层（升维）：最后，通过另一个 1x1 的卷积层将特征图的通道数恢复到与输入相同的维度。这一步骤被称为“升维”或“扩展”，它确保了 Bottleneck 结构的输出可以与网络中的其他层无缝连接。

这种结构有效地减少了计算复杂度，同时保留了网络的表达能力。

优势：

- 1.深度训练：**ResNet** 可以训练非常深的网络，因为残差块使得梯度可以更好地传播。
- 2.特征再利用：残差块允许网络重用特征，通过加法操作提高了信息的流动性。
- 3.性能优越：在许多计算机视觉任务中，**ResNet** 的表现超越了之前的网络架构，如 **VGG** 和 **AlexNet**。

应用：

ResNet 在各种计算机视觉任务中取得了显著成功，包括：

- 1.图像分类：在大型数据集（如 **ImageNet**）上取得了优异的成绩。
- 2.目标检测：在如 **Faster R-CNN**、**YOLO** 等目标检测算法中作为骨干网络使用。
- 3.语义分割：如 **DeepLab** 系列中使用 **ResNet** 作为主干网络。

总结：

ResNet 通过引入残差学习机制，成功地训练了非常深的神经网络，并在许多计算机视觉任务中成为了标准架构。其成功的关键在于残差块的设计，使得网络能够更有效地学习和优化。

2.2.2. Mobilenet

核心思想：

MobileNet 的设计目标是提高计算效率，特别是在移动设备和嵌入式系统中。为了实现这一目标，**MobileNet** 采用了深度可分离卷积（**Depthwise Separable Convolution**）作为其核心组件，这种方法显著减少了计算量和模型的参数量。

深度可分离卷积（Depthwise Separable Convolution）：

深度可分离卷积是 **MobileNet** 的核心技术，它将传统的卷积操作分解为两个步骤：

1.深度卷积（**Depthwise Convolution**）：对每个输入通道分别进行卷积操作。这种卷积操作只涉及每个通道内的空间维度，计算量大大减少。

2.逐点卷积（**Pointwise Convolution**）：使用 1x1 卷积对深度卷积的输出进行线性组合，整合不同通道的特征。

这种分解的好处在于它显著降低了计算复杂度，同时仍然能够提取丰富的特征。

网络架构：

MobileNet 的主要网络架构包括以下几个部分：

- 1.输入层：通常接受 RGB 图像作为输入。
- 2.深度可分离卷积层：由多个深度卷积和逐点卷积层组成。这些层交替堆叠，形成 **MobileNet** 的特征提取部分。
- 3.全局平均池化层：在卷积层之后使用全局平均池化，减少特征图的维度，得到每个通道的平均值。
- 4.全连接层（分类层）：通常是一个带有 **softmax** 激活的全连接层，用于图像分类任务。

MobileNet 变种：

- 1.**MobileNetV1**：最初的 **MobileNet** 版本，介绍了深度可分离卷积，并提供了一个高效的网络结构。
- 2.**MobileNetV2**：引入了线性瓶颈（**Linear Bottleneck**）和倒残差块（**Inverted Residual Block**），进一步提高了网络的性能和效率。
- 3.**MobileNetV3**：结合了自动化机器学习（**AutoML**）和神经架构搜索（**NAS**）技术，优化了模型结构，并引入了新的激活函数（如硬 **swish**）。

优势：

1.高效计算：**MobileNet** 通过减少卷积操作的计算量和参数量，提高了网络的计算效率，非常适合在移动设备上运行。

2.较小的模型大小：由于深度可分离卷积的设计，**MobileNet** 模型的参数数量较少，存储和传输成本



低。

3.良好的性能：尽管 MobileNet 主要关注效率，但它在图像分类任务中的表现仍然相当不错。

应用：

MobileNet 适用于各种计算机视觉任务，特别是在资源受限的环境中。常见应用包括：

- 1.图像分类：在移动设备上执行图像分类任务。
- 2.对象检测：作为轻量级的骨干网络，MobileNet 可以用作目标检测算法中的特征提取器。
- 3.语义分割：在移动设备上执行图像分割任务。

总结：

MobileNet 通过深度可分离卷积等创新设计，在保持较高分类性能的同时，大幅度降低了计算复杂度和模型大小。这使得 MobileNet 成为在移动设备和嵌入式系统上进行深度学习推理的理想选择。

2.2.3. YOLOv5

YOLOv5 概述：

YOLOv5 继承了 YOLO 系列模型的一些核心优势，并在性能、速度和易用性方面进行了多方面的改进。YOLO 系列模型的主要特点是通过一次前向传播就能同时检测图像中的多个目标，从而实现实时检测。

YOLOv5 的主要特性：

- 1.高效的推理速度：YOLOv5 的设计旨在实现高速的目标检测，适合实时应用。
- 2.较高的检测精度：在不同的数据集上，YOLOv5 提供了优越的检测精度和召回率。
- 3.可扩展的模型结构：YOLOv5 提供了不同的模型变种（如 YOLOv5s、YOLOv5m、YOLOv5l 和 YOLOv5x），适应不同的应用场景和计算资源。
- 4.支持多种数据集和训练方式：YOLOv5 支持多种常见数据集（如 COCO、VOC 等），并且提供了方便的训练和验证工具。

网络架构：

YOLOv5 的网络架构主要由以下几个部分组成：

- 1.Backbone：特征提取网络，通常使用 CSPNet（Cross-Stage Partial Network）或其他高效的网络结构。
- 2.Neck：用于特征融合的部分，通常使用 PANet（Path Aggregation Network）或其他结构来增强不同尺度特征的融合。
- 3.Head：目标检测的输出部分，负责生成边界框、类别预测和置信度评分。

YOLOv5 模型变种：

- 1.YOLOv5 提供了几种不同大小的模型，以平衡速度和精度：
- 2.YOLOv5s：小型模型，计算量和内存占用较低，适合实时应用。
- 3.YOLOv5m：中型模型，提供了较好的平衡点，适用于大多数任务。YOLOv5l：大型模型，适合需要更高检测精度的任务。
- 4.YOLOv5x：超大型模型，提供了最优的精度，但计算量较大。
- 5.YOLOv5n：这是 YOLOv5 系列中的一个变体，专为 Nano 设备（如 NVIDIA Jetson Nano）进行优化。YOLOv5n 在保持较快速度的同时，提供适用于边缘设备的准确度。

应用场景：

YOLOv5 可以应用于各种目标检测任务，包括但不限于：

- 1.实时视频监控：在监控视频中检测和跟踪目标。
- 2.自动驾驶：检测道路上的车辆、行人和交通标志。
- 3.无人机图像分析：从无人机拍摄的图像中检测目标。
- 4.工业检测：检测生产线上的缺陷或异常。

总结：

YOLOv5 结合了高效的模型设计和优化技术，提供了出色的实时目标检测性能。由于其高效性和易用性，YOLOv5 被广泛应用于各种实际场景中。

2.2.4. YOLOv8

YOLOv8 概述：

YOLOv8 继承了前代模型的优点，并在此基础上进行了多项改进，旨在提供更高的准确性和速度，支持图像分类、物体检测和实例分割等任务。

主要特性：



- 1.高性能: YOLOv8 在多个标准数据集上达到了前所未有的检测性能,通过更深更复杂的网络结构和改进的训练技巧,大幅提高了检测的准确度,同时保持了 YOLO 系列的实时检测特性。
- 2.灵活性: YOLOv8 支持从 CPU 到 GPU 的各种硬件平台,可以在不同场景下灵活部署。
- 3.可扩展性: Ultralytics 将 YOLOv8 定位为算法框架,而非特定算法,支持非 YOLO 模型以及分类、分割、姿态估计等各类任务。
- 4.多尺度预测: YOLOv8 引入了改进的多尺度预测技术,可以更好地检测不同大小的对象。
- 5.自适应锚框: 在自适应调整锚框方面做了优化,可以更准确地预测对象的位置和大小。
- 6.高效训练: YOLOv8 提供了优化的训练流程和更强大的特征提取能力,使得训练过程更加高效。

网络架构:

YOLOv8 的网络架构主要包括三个部分: **Backbone**、**Neck** 和 **Head**。

1.**Backbone**: 采用了一系列卷积和反卷积层来提取特征,使用了残差连接和瓶颈结构来减小网络的大小并提高性能。YOLOv8 的 **Backbone** 部分使用了 **C2f** 模块来替代了 YOLOv5 中的 **C3** 模块,实现了进一步的轻量化。**C2f** 模块具有更少的参数量和更优秀的特征提取能力。

2.**Neck**: 采用了多尺度特征融合技术,将来自 **Backbone** 的不同阶段的特征图进行融合,以增强特征表示能力。YOLOv8 的 **Neck** 部分包括一个 **SPPF** 模块、一个 **PAA** 模块和两个 **PAN** 模块。

3.**Head**: 负责最终的目标检测和分类任务,包括一个检测头和一个分类头。YOLOv8 采用了解耦头结构,将分类和检测任务分离开来,提高了分类的准确性和检测的精确性。

模型变种:

1.yolov8s: yolov8s 是 yolov8 系列中最小的模型,它的骨干网络采用 **Darknet-53**,预训练好的模型经过稀疏化、剪枝和 finetune 等步骤后,可以达到一定的 mAP 值和模型大小减少的效果。

2.yolov8n: yolov8n 是 yolov8 系列中稍微大一点的模型,它在 yolov8s 的基础上增加了一些网络层和参数,以提高模型的性能和精度。

3.yolov8m: yolov8m 是 yolov8 系列中更大的模型,它相比于 yolov8n 进一步增加了网络层和参数,以达到更高的检测精度。

4.yolov8l: yolov8l 是 yolov8 系列中较大的模型,它在 yolov8m 的基础上进一步增加了网络层和参数,以提高模型的性能和精度。

5.yolov8x: yolov8x 是 yolov8 系列中最大的模型,它相比于 yolov8l 进一步增加了网络层和参数,以获得最高的检测精度和性能。

应用场景:

YOLOv8 在多个领域具有广泛的应用前景,包括但不限于:

1.工业视觉识别: 在工业场景中, YOLOv8 可以用于生产线上的物体检测和定位,如自动化装配线上的 零部件识别、产品表面缺陷检测等。

2.自动驾驶: 在自动驾驶领域, YOLOv8 可以用于车辆、行人等目标的实时检测和跟踪,为自动驾驶系统提供重要的环境感知信息。

3.安防监控: 在安防监控领域, YOLOv8 可以用于实时监测视频画面中的异常行为或目标,如人脸识别、行为分析等。

4.农业自动化: 在农业领域, YOLOv8 可以用于农作物的病虫害检测、作物生长监测等,提高农业生产的智能化水平。

2.2.5. LPRNet

LPRNet 概述:

LPRNet 的设计目标是在各种情况下都能准确地识别车牌上的字符,这些情况可能包括不同的光照条件、天气条件、车牌的设计和背景的复杂性。LPRNet 通常结合了多个神经网络结构来完成车牌识别的所有阶段,包括预处理、候选区域检测 (region proposal)、字符分割、特征提取和分类。

LPRNet 的主要特性:

1.专门构建: LPRNet 是专门为车牌识别设计的,它能够处理车牌图像的特点,如字符的维度不确定性和多种字符集。

2.高效性: 虽然车牌识别的任务可能需要精细的特征,但 LPRNet 设计时考虑到了计算效率,特别是在实际应用中需要快速响应的场景。

3.鲁棒性: LPRNet 能够处理各种干扰,如亮光照射、模糊、不同角度拍摄等。

网络架构:



1. LPRNet 通常由多部分组成，这些部分协同工作，以达到最佳的车牌识别效果：
2. 预处理：对输入的图像进行处理，以提高字符识别的质量。
3. 候选区域检测：使用卷积神经网络（CNN）或类似技术来定位可能的车牌区域。
4. 字符分割：在找到的车牌区域中，进一步分割出独立的字符。
5. 特征提取和分类：对于分割出的字符，提取特征并进行分类，以确定具体的字符。

训练和使用：

LPRNet 的训练通常需要大量的标记良好的车牌图像数据。使用这些数据，可以训练和微调网络的各个部分，以达到所需的车牌识别性能。

在实际使用中，LPRNet 通常从输入图像开始，经过预处理和候选区域检测后，进入到字符分割和分类阶段，最终输出识别到的车牌号码。

应用场景：

LPRNet 常被应用于交通管理、智能监控和安全系统等领域，用于自动执行车牌识别任务。

1. 交通监控：自动记录通过摄像头的车辆信息，用于电子收费、违章检测等。
2. 智能停车管理：可以自动识别进出车辆的车牌，进行车位管理和收费。
3. 智能安防系统：用于识别和跟踪进出特定区域的车辆。

总结：

LPRNet 是一类特殊设计的深度学习模型，专门用于车牌识别。它解决了与车牌相关的一系列复杂问题，如字符的定位、分割、识别等。在实际应用中，LPRNet 能够提高自动化车牌识别的速度和准确性，适用于各种交通系统和智能监控系统。

2.2.6. PPOCR

PPOCR（PaddlePaddle Optical Character Recognition）是由百度的 PaddlePaddle 团队开发的一套光学字符识别（OCR）系统。它基于 PaddlePaddle 深度学习框架，旨在提供高效且准确的文本检测和识别解决方案。以下是 PPOCR 的详细介绍：

PPOCR 概述：

PPOCR 是一个开源的 OCR 系统，致力于解决各种文本识别问题。它支持从图像中检测文本区域并进行字符识别，适用于多种应用场景，包括文档扫描、车牌识别、名片识别等。

PPOCR 的主要特性：

1. 高准确度：PPOCR 提供高精度的文本检测和识别，支持多种语言和字符集。高效率：系统经过优化，能够在较短时间内处理大规模的文本数据。
2. 多场景支持：支持不同类型的文本图像，包括但不限于印刷文本、手写文本、标志、广告等。开放源代码：PPOCR 是一个开源项目，可以根据需要进行修改和扩展。

网络架构：

PPOCR 通常包括以下几个主要模块：

1. 文本检测网络：例如 DBNet（Differentiable Binarization Network）或 EAST（Efficient and Accurate Scene Text detector），用于检测文本区域。
2. 文本识别网络：例如 CRNN（Convolutional Recurrent Neural Network）或 Transformer-based 网络，用于识别文本中的字符。
3. 模型融合：结合多个检测和识别模型以提高系统的整体性能。

应用场景：

1. 文档扫描和数字化：将纸质文档转换为可编辑的文本格式。
2. 车牌识别：自动识别车辆的车牌信息。
3. 名片识别：从名片中提取联系信息。
4. 广告和标志识别：识别和分析广告或标志上的文本信息。

总结：

PPOCR 是一个功能强大的光学字符识别系统，利用 PaddlePaddle 框架实现了高效且准确的文本检测和识别。其开源特性和高性能使得它适用于各种实际应用场景中的文本识别任务。

2.2.7. DeepLab

DeepLab 概述：

DeepLab 是一个语义分割模型，旨在实现高精度的像素级别分类。它利用卷积神经网络（CNN）和先进的特征提取技术来处理各种复杂的图像分割任务。

**DeepLab 的主要版本:**

DeepLab 模型有多个版本, 随着时间的推移不断改进和优化。主要版本包括:

1. DeepLab v1: 最初的版本, 使用了一个全卷积网络 (FCN) 来进行语义分割。

2. DeepLab v2: 引入了空洞卷积 (Dilated Convolution), 使得模型能够更好地捕捉上下文信息和细节。

3. DeepLab v3: 进一步改进了空洞卷积, 结合了空洞空间金字塔池化 (ASPP), 提高了多尺度特征的处理能力。

4. DeepLab v3+: 在 DeepLab v3 的基础上增加了编码器-解码器结构, 进一步提高了分割的精度, 特别是在边缘区域。

DeepLab 的主要特性:

1. 空洞卷积 (Dilated Convolution): DeepLab v2 及之后的版本使用空洞卷积来扩展卷积核的感受野, 捕捉更大范围的上下文信息, 同时保持特征图的分辨率。

2. 空洞空间金字塔池化 (ASPP): DeepLab v3 引入了 ASPP, 用于提取多尺度特征, 通过多个空洞卷积层提取不同尺度的信息。

3. 编码器-解码器结构: DeepLab v3+ 增加了一个解码器网络, 帮助更好地恢复细节和边界信息, 提高了模型在精细边界区域的表现。

网络架构:

1. DeepLab 的网络架构通常包括以下几个主要部分:

2. 主干网络 (Backbone Network): 用于提取图像特征的卷积网络, 常用的主干网络有 ResNet、Xception 等。

3. 空洞卷积层: 用于扩展感受野而不降低特征图分辨率的卷积层。

4. ASPP 层: 用于捕捉不同尺度的上下文信息。

5. 解码器 (仅在 DeepLab v3+ 中): 用于恢复精细的边界和细节信息。

应用场景:

1. 自动驾驶: 分割道路、车辆、行人等, 帮助自动驾驶系统进行环境感知。

2. 医学图像分析: 分割医学图像中的不同组织或病变区域, 用于诊断和分析。

3. 城市规划: 分析卫星图像, 分割城市中的建筑物、道路和绿地。

4. 机器人视觉: 用于机器人进行环境感知和物体识别。

总结:

DeepLab 是一个强大的语义分割模型, 通过引入空洞卷积、ASPP 层和编码器-解码器结构等技术, 显著提升了语义分割的精度和细节捕捉能力。它在多个领域中具有广泛的应用, 能够处理各种复杂的图像分割任务。

2.2.8. RetinaFace

RetinaFace 概述:

RetinaFace 是一个用于面部检测的深度学习模型, 它可以在图像中检测出人脸的位置和关键点。与传统的人脸检测方法相比, RetinaFace 在准确性和鲁棒性方面都有显著的提高, 特别是在高密度人脸场景和各种复杂背景中表现出色。

主要特性:

1. 高精度: RetinaFace 能够在各种条件下准确检测到人脸, 包括遮挡、不同的光照条件和复杂的背景。

2. 多尺度检测: 它能够检测不同尺寸的人脸, 包括非常小的人脸。

3. 关键点检测: 除了人脸的边界框外, RetinaFace 还可以检测人脸的关键点 (如眼睛、鼻子、嘴巴等)。

4. 实时性: 经过优化的 RetinaFace 可以实现实时人脸检测, 适用于各种应用场景。

网络架构:

RetinaFace 的网络架构主要包括以下几个部分:

1. 主干网络 (Backbone Network): 用于提取图像特征, 常用的主干网络包括 ResNet 和 MobileNet 等。

2. 特征金字塔网络 (Feature Pyramid Network, FPN): 用于多尺度特征提取, 以便检测不同尺寸的人脸。



3.人脸检测头（Face Detection Head）：在特征图上进行人脸检测，输出边界框和人脸类别。

4.关键点检测头（Face Landmark Head）：检测人脸关键点的位置。

算法细节：

1.角点回归（Corner Regression）：RetinaFace 使用角点回归技术来预测人脸的边界框位置，从而提高检测精度。

2.高效的损失函数：使用了高效的损失函数来优化人脸检测的准确性，包括边界框回归损失和关键点回归损失。

3.细粒度的特征处理：通过细粒度的特征处理来改进小尺寸人脸的检测。

应用场景：

1.安全监控：在监控视频中进行人脸检测和识别。

2.社交网络：自动标记和识别社交媒体上的人脸。

3.人脸识别：作为人脸识别系统的一部分，用于验证和识别用户身份。

4. 增强现实：用于 AR 应用中的面部跟踪和特效应用。

总结：

RetinaFace 是一种先进的面部检测算法，通过引入特征金字塔网络、角点回归等技术，实现了高精度和高鲁棒性的人脸检测。它在各种复杂场景中表现出色，广泛应用于安全监控、社交媒体、人脸识别等领域。

2.3. RK 平台适用的深度学习框架及模型

目前 RK 平台支持多个主流深度学习框架的模型转换，包括：

Caffe（推荐版本为 1.0）

TensorFlow（推荐版本为 1.12.0~2.8.0）

TensorFlow Lite（推荐版本为 Schema version = 3）

ONNX（推荐版本为 1.7.0~1.10.0）

Pytorch（推荐版本为 1.6.0~1.13.1）

Darknet（推荐版本为 Commit ID = 810d7f7）

支持的经典模型有：

1.图像分类模型：Mobilenet（Mobilenetv2-12）；ResNet（ResNet50-v2-7）。

2.目标检测模型：YOLOv5（YOLOv5s_relu、YOLOv5n、YOLOv5s、YOLOv5m）；YOLOv6（YOLOv6n、YOLOv6s、YOLOv6m）；YOLOv7（YOLOv7-tiny、YOLOv7）；YOLOv8（YOLOv8n、YOLOv8s、YOLOv8m）；YOLOv8_obb（YOLOv8n-obb）；YOLOv10（YOLOv10n、YOLOv10s）；YOLO11（YOLO11n、YOLO11s、YOLO11m）；YOLOx（YOLOx_s、YOLOx_m）；Ppyoloe（Ppyoloe_s、Ppyoloe_m）；YOLO_world（YOLO_world_v2s、Clip_text）。

3.人体关键点模型：YOLOv8_pose（YOLOv8n-pose）。

4.图像分割模型：Deeplabv3（Deeplab-v3-plus-mobilenet-v2）；YOLOv5_seg（YOLOv5n-seg、YOLOv5s-seg、YOLOv5m-seg）；YOLOv8_seg（YOLOv8n-seg、YOLOv8s-seg、YOLOv8m-seg）；Ppseg（Ppseg_lite_1024x512）；Mobilesam（Mobilesam_encoder_tiny、Mobilesam_decoder）。

5.人脸关键点模型：RetinaFace（RetinaFace_mobile320、RetinaFace_resnet50_320）。

6.车牌识别模型：LPRNet。

7.文字检测模型：PPOCR-Det（PPOCRv4_Det）。

8.文字识别模型：PPOCR-Rec（PPOCRv4_Rec）。

9.自然语言翻译模型：Lite_transformer（Lite-transformer-encoder-16、Lite-transformer-decoder-16）。

10.图文匹配模型：Clip（Clip_images、Clip_text）。

11.语音识别模型：Wav2vec2（Wav2vec2_base_960h_20s）；Whisper（Whisper_base_20s）；

Zipformer（Zipformer-bilingual-zh-en-t）。

12.语音分类模型：Yamnet（Yamnet_3s）。

13.文字转语音模型：Mms_tts（Mms_tts_eng_200）。



第三章 数据收集与处理

数据处理是对收集到的数据进行清洗、转换、分析等一系列操作，使其变成有价值的信息,在本章节中，我们将介绍如何进行数据收集以及利用图像标注工具对收集的图片进行图像的标注工作，并探讨一系列常用的数据增强技术以及如何合理划分数据集以促进模型训练与评估的有效性。

本章将分为如下几个小节：数据收集、数据标注、数据增强、数据集划分。

3.1. 数据收集

这里对数据收集方法进行介绍：

1.使用开源数据集

开源数据集可以通过数据集搜索网站或者 Kaggle、天池等竞赛平台进行检索下载，常用的数据集有 COCO 数据集、VOC 数据集等。这种方法省时省力且基准可靠，无需从零开始收集和标注，常作为学术研究和算法比较的基准。常用的数据集搜索网站：

2.Roboflow Universe 计算机视觉数据集：<https://universe.roboflow.com/>;

3.HyperAI 超神经：<https://hyper.ai/cn/datasets>。

4.网络爬取

通过爬虫脚本，从图片网站、搜索引擎等自动化地批量下载图片，这种方法可以快速获取大量、特定主题的图片，但是需要确保使用方式符合网站规定和法律法规。

5.真实场景拍摄

使用手机、相机、摄像头等设备在实际环境中拍摄收集，这种方法贴近应用场景并且可控性强，数据分布与真实场景完全一致，能极大提升模型的实战能力，可以针对性地补全模型易出错的场景（如反光、遮挡、夜间）。

3.2. 数据标注

图像数据标注是指通过人工或自动化的方式，对图像中的目标对象、场景、属性等进行标记和注释的过程。其目的是为了生成有标签的图像数据集，这些数据集可以作为机器学习模型的训练数据，帮助模型学习如何识别和理解图像中的不同元素。

图像数据标注有多种类型和方法，根据不同的应用场景和任务需求，可以选择合适的标注方式。以下是一些常见的图像数据标注类型和方法：

1.目标检测标注：在图像中标注出目标对象的位置和边界框，并给出对象的类别标签。这种方法常用于自动驾驶、安防监控等领域。

2.语义分割标注：将图像中的每个像素都标注上对应的类别标签，以实现像素级别的分类。这种方法常用于医学图像分析、自动驾驶等领域。

3.关键点标注：在图像中标注出目标对象的关键点位置，如人脸的眼角、嘴角等。这种方法常用于人脸识别、姿态估计等领域。

4.多边形标注：使用多边形框标注出图像中的不规则目标物体。这种方法比矩形框标注更精确，适用于复杂场景下的目标检测。

5.3D 标注：对图像中的物体进行三维空间中的标注，包括物体的形状、大小、位置等。这种方法常用于自动驾驶、虚拟现实等领域。

此外，还有 OCR 转写（对图像中的文字内容进行标记与转写）、属性判别（识别并标注图像中目标物体的属性）等标注方法。

3.3. 图像数据增强

图像数据增强是指通过一系列图像处理技术，对原始图像进行变换或修改，以生成与原始图像相似但又不完全相同的新图像样本。这些新样本被用于训练机器学习或深度学习模型，以提高模型在不同场景下的识别能力和准确性。图像数据增强的主要作用包括：

1.增加数据多样性：通过变换图像的亮度、对比度、颜色、角度等属性，生成多样化的图像样本，使模型能够学习到更加全面的特征表示。

2.提高模型泛化能力：增加训练数据集的规模和多样性，有助于模型摆脱对特定样本的过度依赖，提高在未见过的数据上的表现能力。

3.减少过拟合：在训练数据有限的情况下，模型容易陷入过拟合状态，即过度拟合训练数据而无法泛化到新的数据上。图像数据增强可以通过增加训练样本的多样性来降低过拟合的风险。



3.3.1. 常见方法

1. 几何变换：

- (1) 旋转：将图像围绕某一点旋转一定的角度。
- (2) 翻转：包括水平翻转和垂直翻转，即将图像沿水平或垂直轴进行翻转。
- (3) 缩放：改变图像的大小，包括放大和缩小。
- (4) 平移：将图像沿水平或垂直方向移动一定的距离。

2. 颜色变换：

- (1) 亮度调整：增加或减少图像的亮度。
- (2) 对比度调整：改变图像的对比度，使图像更加清晰或模糊。
- (3) 饱和度调整：通过调整图像的饱和度，可以生成与原始图像相似但色彩鲜艳程度不同的新图像样本。

3. 添加噪音：

- (1) 高斯噪声：向输入数据添加服从高斯分布（即正态分布）的随机噪声。
- (2) 椒盐噪声：通过在原始图像上随机添加白色（盐噪声）或黑色（椒噪声）的像素点。

3.3.2. 注意事项

1. 保持数据特性：增强后的图像应尽可能保持原始图像的重要信息和特征，避免引入过多与任务无关的变化。
2. 合理控制增强程度：过强的增强可能会导致图像失真或引入噪声，影响模型的训练效果。因此，需要合理控制增强的程度和范围。
3. 结合任务需求：不同的任务对图像数据的需求不同，因此需要根据具体任务选择合适的增强方法和参数。
4. 验证增强效果：通过实验验证不同增强方法的效果，选择最适合当前任务和数据集的增强策略。

3.4. 数据集划分

在深度学习中，数据集划分是一个至关重要的步骤，它直接影响到模型的训练效果和泛化能力。数据集通常被划分为训练集、验证集和测试集三个部分，每个部分都有其特定的用途。

1. 训练集（Training Set）：用于训练模型，使模型能够学习到数据的内在规律和特征。

2. 验证集（Validation Set）：在训练过程中使用，用于评估模型的性能，并根据评估结果调整模型的参数，如学习率、网络结构等，以改善模型的表现。验证集不参与模型的最终评估。

3. 测试集（Test Set）：在模型训练完成后使用，用于评估模型的最终性能，确保模型在未见过的数据上也能表现良好。测试集的结果反映了模型的泛化能力。

3.4.1. 划分方法

1. 按比例划分：常见的划分比例是 70% 的训练集、20% 的验证集和 10% 的测试集或 80% 的训练集、10% 的验证集和 10% 的测试集，需要根据具体情况调整比例。通过均匀随机抽样的方式将数据无交集地划分为三个集合，确保训练集、验证集和测试集同分布。

2. 分层抽样划分：根据样本的类别或某个关键特征进行分层。在每一层中，按照一定比例随机抽取样本分配给训练集、验证集和测试集。这种方法可以确保训练集、验证集和测试集中的类别分布与整个数据集相似，有助于提高模型的泛化能力。

3.4.2. 注意事项

1. 同分布性：训练集、验证集和测试集必须同分布，以确保模型能够在未见过的但分布相同的数据上表现良好。
2. 无交集性：三个集合之间不能有交集，以避免信息泄露导致评估结果不准确。
3. 随机性：划分过程应该是随机的，以确保结果的公平性和可重复性。
4. 代表性：划分后的各个集合应该具有代表性，能够反映整个数据集的特点。



第四章 模型训练与导出

4.1. 项目文件介绍

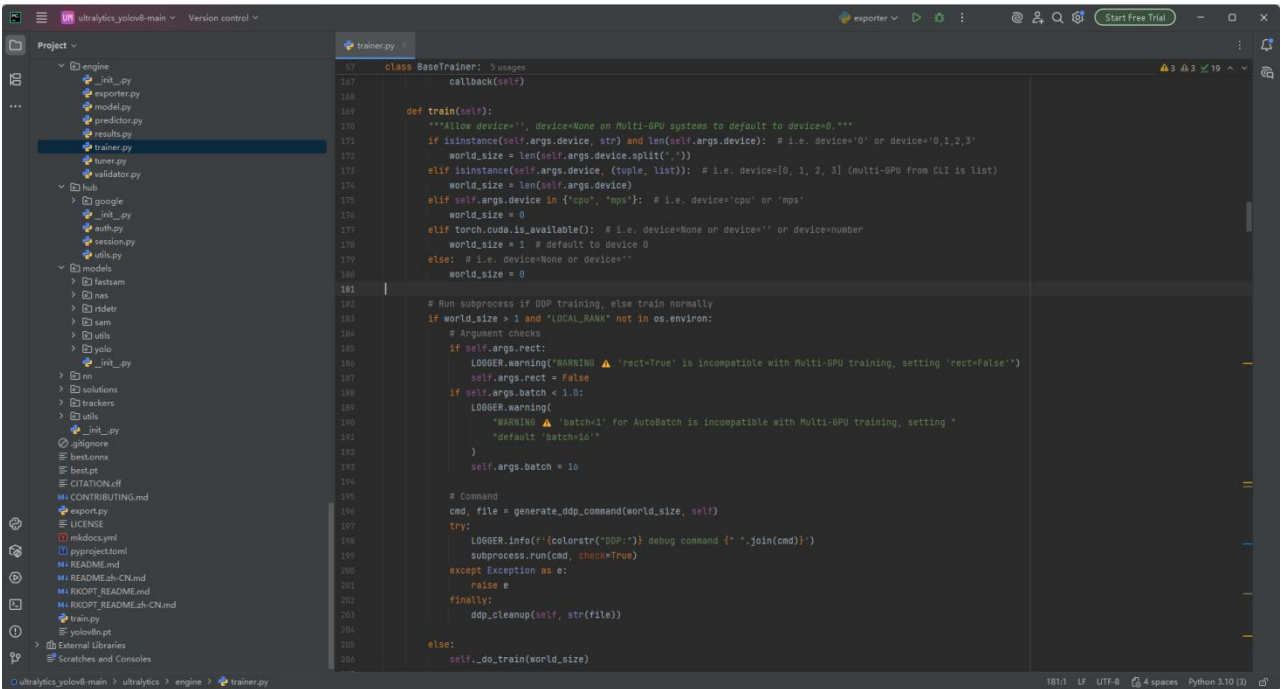


ultralytics_yolov8 的项目文件结构通常包含多个关键部分，以下是一些主要文件夹和文件的介绍：

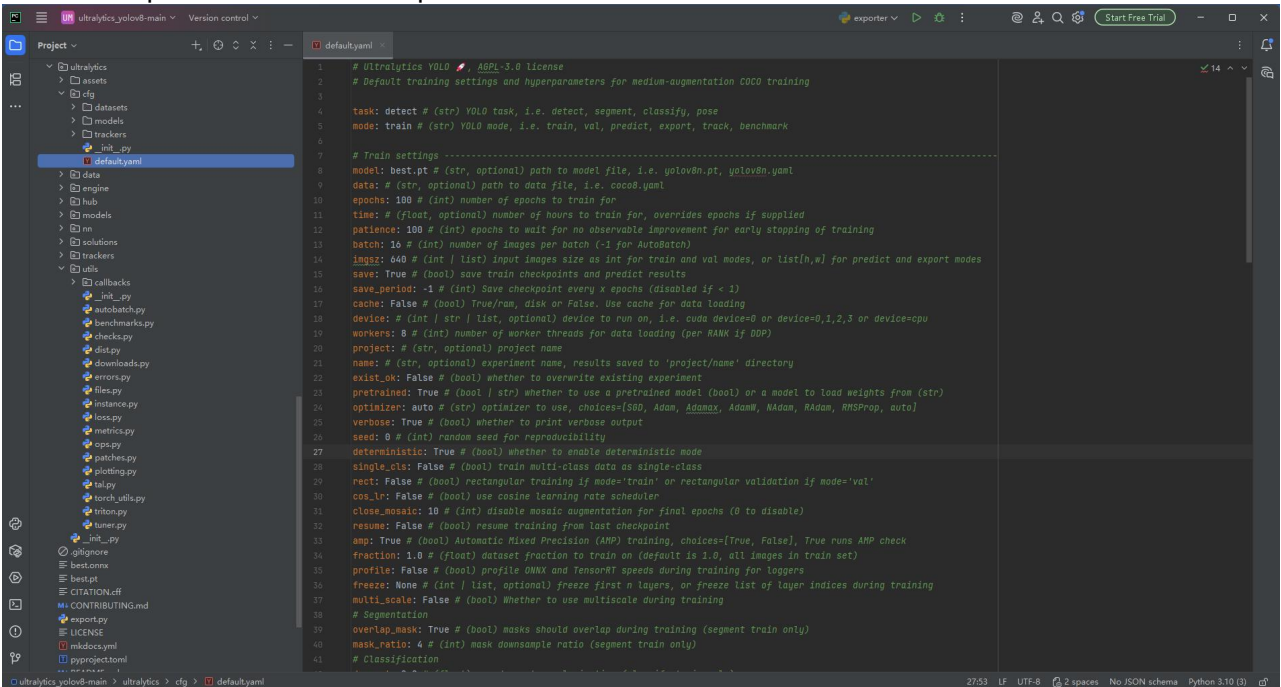
- 1..**github**: GitHub 配置文件夹，存储 **issue** 模板、CI 持续集成工作流（如代码检查、构建测试）等，仅在 GitHub 仓库中存在。
- 2..**docker**: Docker 环境配置文件夹，包含不同环境（CPU/ ARM）的 **Dockerfile**，用于快速生成对应环境的容器，方便在不同硬件、架构或开发场景下部署应用。
- 3..**docs**: 文档资源文件夹，存放项目说明文档、API 文档等。
- 4..**examples**: 示例代码集合，包括 C++ 环境下的 YOLOv8 推理示例，基于 ONNX Runtime 引擎的跨平台推理示例，结合 OpenCV 的推理示例等。
- 5..**runs**: 运行结果文件夹，YOLOv8 训练/推理/验证后自动生成的目录，包含训练日志、权重文件、可视化结果。
- 6..**tests**: 单元测试文件夹，包含对 YOLOv8 功能（如模型加载、指标计算、CUDA 兼容性）的测试用例，用于验证代码正确性。
- 7..**ultralytics**: 核心代码文件夹，YOLOv8 的功能实现核心，包括 **assets** 资源文件目录，存放示例图片，用于演示、测试等场景；**cfg** 配置文件目录，管理模型结构、数据集、训练超参数等配置；**data** 数据处理模块目录，实现数据集加载、数据增强、标签解析等功能；**engine** 核心引擎目录，封装模型的训练（**trainer.py**）、推理（**predictor.py**）、验证（**validator.py**）、模型导出（**exporter.py**）等核心流程，是用户调用 API 的核心层；**PyTorch Hub** 集成目录，实现通过 **PyTorch Hub** 快速加载 YOLOv8 预训练模型的功能；**models** 模型架构目录，定义 YOLOv8 的网络结构（**Backbone/Neck/Head**）及各任务（检测/分割/姿态）的模型类，是算法实现的核心层；**nn** 神经网络组件目录，提供自定义层、损失函数、激活函数等基础组件，支撑模型架构的构建；**solutions** 解决方案目录，存放针对特定业务场景的拓展功能（如目标计数、动作识别等），是算法落地的场景化封装；**utils** 工具函数目录，提供资源适配、性能测试、异常处理、指标计算等辅助功能，支撑各核心模块的运行。



4.2. 主要文件介绍



上图是 `ultralytics_yolov8` 中的核心训练类实现文件 `trainer.py`，主要定义了用于模型训练的基础类 `BaseTrainer`，封装了 YOLO 系列模型（如目标检测、分类、分割等）训练的通用流程和核心逻辑。首先，解析用户配置与默认参数，确定训练设备（CPU/GPU），创建结果保存目录（含权重与日志子目录），校验并加载训练/验证数据集，同时初始化随机种子以保证实验可复现性。训练前会完成模型加载、网络层冻结、优化器与学习率调度器构建，并根据设备数量配置单卡或多卡分布式训练环境。然后，按 `epochs` 迭代执行训练流程，按 `batch` 迭代处理数据，通过前向传播计算损失，反向传播累积梯度后更新参数。最后，每轮结束后在验证集上评估模型性能，记录损失、精度等指标至 `CSV` 文件，根据性能优劣保存最佳模型（`best.pt`）与最新模型（`last.pt`）。



`trainer.py` 作为模型训练的核心执行文件，通过加载 `default.yaml` 中定义的默认配置（包含参数和超参数）初始化训练环境，这些配置涵盖了从任务类型、数据路径、设备选择等基础执行参数，到训练轮次、批次大小、学习率调度等关键超参数，为训练流程提供了标准化的初始设定。以下是对 `default.yaml` 中一



些关键训练参数的解析。

1.任务类型 (Task)

参数: **task**

作用: 定义模型训练的任务类型, 默认任务类型为目标检测, 可选实例分割、图像分类和姿态估计。

示例: **task: detect**

2.运行模式 (Mode)

参数: **mode**

作用: 指定运行模式, 让框架加载对应模式的专属逻辑。

示例: **mode: train**

3.数据集 (Data)

参数: **data**

作用: 指定数据集配置文件的路径。该配置文件包含了数据集的路径、类别等信息。

示例: **data: coco8.yaml**

4.训练轮次 (Epochs)

参数: **epochs**

作用: 指定训练总轮次, 默认为 100, 每轮会遍历一次完整训练集。

示例: **epochs: 100**

5.批量大小 (Batch Size)

参数: **batch**

作用: 指定每个训练批次的大小, 这个值会影响模型的训练速度和内存使用。

示例: **batch: 16**

6.图片尺寸 (Image Size)

参数: **imgsz**

作用: 指定模型训练和测试时输入图片的尺寸大小。这通常是一个整数, 表示图片将被缩放到的宽度和高度。

示例: **imgsz: 640**

7.线程数量 (Workers)

参数: **workers**

作用: 指定数据加载线程数, 默认为 8, 多线程并行加载数据, 加速预处理。

示例: **workers: 8**

8.优化器 (Optimizer)

参数: **optimizer**

作用: 指定模型训练时使用的优化器, 默认为 **auto**, 表示自动选择优化器, 可选 **SGD, Adam, Adamax, AdamW, Nadam, Radam, RMSProp**。

示例: **optimizer: auto**

9.训练设备 (Device)

参数: **device**

作用: 指定训练使用的设备, 默认为空, 框架会自动检测。

示例: **device: cpu**

10.总结

default.yaml 中的这些参数允许用户灵活地配置训练过程, 以满足不同的需求。通过调整这些参数, 用户可以优化模型的训练效果、速度和资源使用。在实际应用中, 建议根据数据集的大小、模型的复杂度和计算资源的情况来选择合适的参数设置。

请注意, **ultralytics_yolov8** 的具体参数可能会随着版本的更新而有所变化, 因此建议查阅最新的官方文档或源代码以获取最准确的信息。

4.3. 模型评估的指标

模型评估是机器学习和深度学习中至关重要的环节, 它帮助我们判断模型的性能, 并据此进行模型的优化和选择。模型评估的指标多种多样, 针对不同的任务和模型类型, 评估指标也会有所不同。以下是一些常见的模型评估指标:

1.准确率 (Accuracy)



定义：准确率是指模型正确预测的样本数量占总样本数量的比例。

公式： $\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$

适用范围：适用于分类问题，尤其是类别分布均衡的情况。

2. 精确率 (Precision)

定义：精确率是指模型预测为正类的样本中，真正为正类的样本数量占所有预测为正类的样本数量的比例。

公式： $\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$

适用范围：当误报成本较高时，精确率尤为重要。

3. 召回率 (Recall)

定义：召回率是指模型正确预测为正类的样本数量占所有真正为正类的样本数量的比例。

公式： $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$

适用范围：当漏检成本较高时，召回率尤为重要。

4. F1 分数 (F1 Score)

定义：F1 分数是精确率和召回率的调和平均值，它综合了精确率和召回率的性能。

公式： $\text{F1 Score} = 2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})$

适用范围：F1 分数适用于需要同时考虑精确率和召回率的情况。

5. 平均精确率 (Average Precision, AP)

定义：AP 是指在不同的召回率水平下，模型精确率的平均值。通常用于目标检测任务中。

计算方法：首先绘制 Precision-Recall 曲线，然后计算曲线下的面积 (Area Under the Curve, AUC)。

适用范围：目标检测、语义分割等任务。

6. 平均精确率均值 (Mean Average Precision, mAP)

定义：mAP 是指在所有类别上计算的 AP 的平均值。它提供了模型在所有类别上的综合性能评估。

计算方法：对每个类别分别计算 AP，然后取所有类别的 AP 的平均值。

适用范围：多类别目标检测、语义分割等任务。

7. 混淆矩阵 (Confusion Matrix)

定义：混淆矩阵是用于评估分类模型性能的一种表格形式，它将模型的预测结果与真实标签进行比较，并将它们分类为四种不同的情况：真正例 (TP)、真负例 (TN)、假正例 (FP) 和假负例 (FN)。

用途：混淆矩阵是评估分类模型性能的基础，可以从中计算出精确率、召回率、F1 分数等指标。

8. FPS (Frames Per Second)

定义：FPS 衡量的是模型每秒处理的帧数，反映了模型的运行速度。

用途：FPS 通常用于评估模型的实时性，特别是在对速度要求较高的应用中，如实时视频目标检测等。

9. 置信度 (Confidence)

定义：置信度是模型对预测结果的信心程度，通常以 0 到 1 之间的数值表示。

用途：置信度通常用于过滤低置信度的预测结果，提高模型预测的准确性。

以上指标在模型评估中各有侧重，选择合适的评估指标对于模型的优化和选择至关重要。在实际应用中，可以根据具体任务和模型特点选择合适的指标进行评估。

第五章 模型转换与部署

随着深度学习应用的发展，训练框架与推理（部署）框架逐渐分化，没有统一框架能同时适配多样的训练需求、不同厂商的硬件设备及部署场景。各大硬件厂商的架构存在差异，对应的处理器或加速设备对模型的输入数据类型、格式要求各不相同，需将模型的参数与操作转化为目标平台支持的形式，因此模型转换成为必要环节，转换后的模型将可以在硬件设备上部署推理。

本章节将主要对模型转换与部署过程用到的 RKNN 相关工具、转换过程、模型部署推理做整体介绍。

5.1. RKNN 工具介绍

瑞芯微提供了强大的工具链，能够帮助我们高效地将主流 AI 框架训练的模型（如 PyTorch, TensorFlow）转换、优化，并部署到多样化的边缘计算硬件平台上，显著提升部署效率，降低门槛。

5.1.1. RKNN-Toolkit2

RKNN-Toolkit2 是由瑞芯微电子开发的一套深度学习模型优化和推理工具。它支持多种深度学习框架（如 Caffe、TensorFlow、PyTorch 等）的模型转换为 RKNN 格式，并提供了模型转换、量化、推理等功能。RKNN-Toolkit2 主要面向在瑞芯微 SoC 上进行 AI 应用开发，但也可以用于 PC 平台进行模型的转换、量化、推理等操作。

1. 主要功能

（1）模型转换：RKNN-Toolkit2 支持将多种主流框架（如 TensorFlow、PyTorch、Caffe 等）训练得到的模型转换为 RKNN 模型，以便在 Rockchip 的硬件平台上运行。

（2）模型优化：工具包提供了模型优化功能，可以对转换后的 RKNN 模型进行进一步的优化，以提高模型在 Rockchip NPU 上的运行效率。

（3）性能和内存评估：将 RKNN 模型分发到指定 NPU 设备上运行，以评估模型在实际设备上运行时的性能和内存占用情况。

（4）模型部署：通过 RKNN-Toolkit2，用户可以将优化后的 RKNN 模型部署到 Rockchip 的开发板上，进行实际的推理测试和应用开发。

2. 支持情况

支持的模型格式：RKNN-Toolkit2 支持多种模型格式的转换，包括但不限于 ONNX、TensorFlow 的 .pb 格式、PyTorch 的 .pth 或 .pt 格式等。

支持的硬件平台：该工具包主要针对 Rockchip 的硬件平台进行优化和部署，如 RK3588、RK3576、RV1126 等。

系统兼容性：RKNN-Toolkit2 可以在多种操作系统上运行，包括但不限于 Debian10、Debian11 以及基于 ARMv8-A 架构的系统。

3. 使用流程

（1）准备阶段：用户需要准备训练好的模型文件，并确保其格式与 RKNN-Toolkit2 支持的格式相匹配。

（2）模型转换阶段：使用 RKNN-Toolkit2 将原始模型转换为 RKNN 模型。这一过程中，用户需要指定目标硬件平台和 NPU 型号。

（3）模型优化与测试阶段：对转换后的 RKNN 模型进行优化，并在 Rockchip 的开发板上进行推理测试，以确保模型的准确性和性能。

（4）部署阶段：将优化后的模型部署到实际的应用场景中，进行实际的应用开发。

4. 注意事项

在使用 RKNN-Toolkit2 进行模型转换和优化时，需要确保开发环境的配置正确，包括 Python 版本、相关依赖库的安装等。

5.1.2. RKNN-Toolkit-Lite2

RKNN-Toolkit-Lite2 是 Rockchip 为其轻量级神经网络处理单元（NPU）提供的一款工具包。它专注于简化和加速深度学习模型在 Rockchip 硬件平台上的部署和推理过程。与 RKNN-Toolkit2 相比，RKNN-Toolkit-Lite2 主要针对轻量级应用和较小的计算需求进行优化。

1. 主要功能

（1）模型转换：支持将多种主流深度学习框架（如 TensorFlow Lite、ONNX 等）训练得到的模型转换为 RKNN Lite 模型，便于在 Rockchip 的轻量级 NPU 上进行推理。

（2）模型优化：提供了针对轻量级模型的优化功能，以提高模型在 Rockchip 轻量级 NPU 上的运行效率和速度。

（3）推理部署：使得优化后的 RKNN Lite 模型可以在 Rockchip 的开发板或嵌入式设备上推理，适合实时应用场景。

2. 支持情况

（1）支持的模型格式：支持 TensorFlow Lite 的 .tflite 格式和 ONNX 格式等。

（2）支持的硬件平台：针对 Rockchip 的轻量级 NPU 进行优化，适用于相应的开发板和嵌入式设备。

（3）系统兼容性：兼容多种基于 ARM 架构的操作系统，如 Android 和 Linux 系统。

3. 使用流程

（1）准备模型：准备已经训练好的深度学习模型，并确保模型格式与 RKNN-Toolkit-Lite2 支持的格式一致。

（2）模型转换：使用 RKNN-Toolkit-Lite2 工具将模型文件转换为 RKNN Lite 格式。这一过程需要指定目标硬件平台和 NPU 类型。

（3）模型优化：对转换后的 RKNN Lite 模型进行优化，以提升其在硬件上的推理性能和效率。

（4）推理测试：在 Rockchip 的开发板或目标设备上推理测试，确保模型的性能和准确性符合要求。

（5）部署：将优化后的模型部署到实际的应用场景中，进行应用开发和实际运行测试。

4. 注意事项

（1）环境配置：确保开发环境的配置符合要求，包括 Python 版本和依赖库的安装。

（2）系统兼容性：确保开发板或设备的系统版本和编译环境与 RKNN-Toolkit-Lite2 工具包兼容。

（3）官方文档和支持：在使用过程中，如遇到问题，可以参考 Rockchip 提供的官方文档和技术支持，或在相关开发者社区寻求帮助。

5.1.3. RKNN Model Zoo

RKNN Model Zoo 是瑞芯微（Airockchip）基于 RKNPU SDK 工具链开发的优化模型集合，为主流算法提供了完整部署示例，涵盖 RKNN 模型的使用与推理全流程。集合内置 YOLO、ResNet 等大量预训练模型，搭配配套模型转换脚本，能助力开发者快速将模型转为 RKNN 格式，高效适配瑞芯微芯片部署。因此，我们需借助该集合提供的模型转换脚本，完成目标模型的格式转化。

以下以 RKNN Model Zoo-2.3.2 版本为例，对整个工程的目录结构做简单介绍：

```
rknn_model_zoo-2.3.2
├── 3rdparty # 第三方库
├── asset # 资源文件
├── datasets # 数据集
├── docs # 文档目录
├── examples # 示例代码
├── py_utils # Python 工具库
└── utils # 通用工具目录
```



```
— build-android.sh # 用于目标为Android系统开发板的编译脚本
— build-linux.sh # 用于目标为Linux系统开发板的编译脚本
—
```

其中，examples 目录包括了一些常用模型的示例，例如 MobileNet 和 YOLO 等。每个模型示例提供了 Python 和 C/C++ 两个版本的示例代码（为了方便描述，后续用 RKNN Python Demo 和 RKNN C Demo 来表示）。以 resnet 模型为例，其目录结构如下：

```
rknn_model_zoo-2.3.2
├── examples
│   └── resnet
│       ├── cpp # C/C++ 版本的示例代码
│       ├── model # 模型、测试图片等文件
│       ├── python # 模型转换脚本和 Python 版本的示例代码
│       └── README.md
└── ...
```

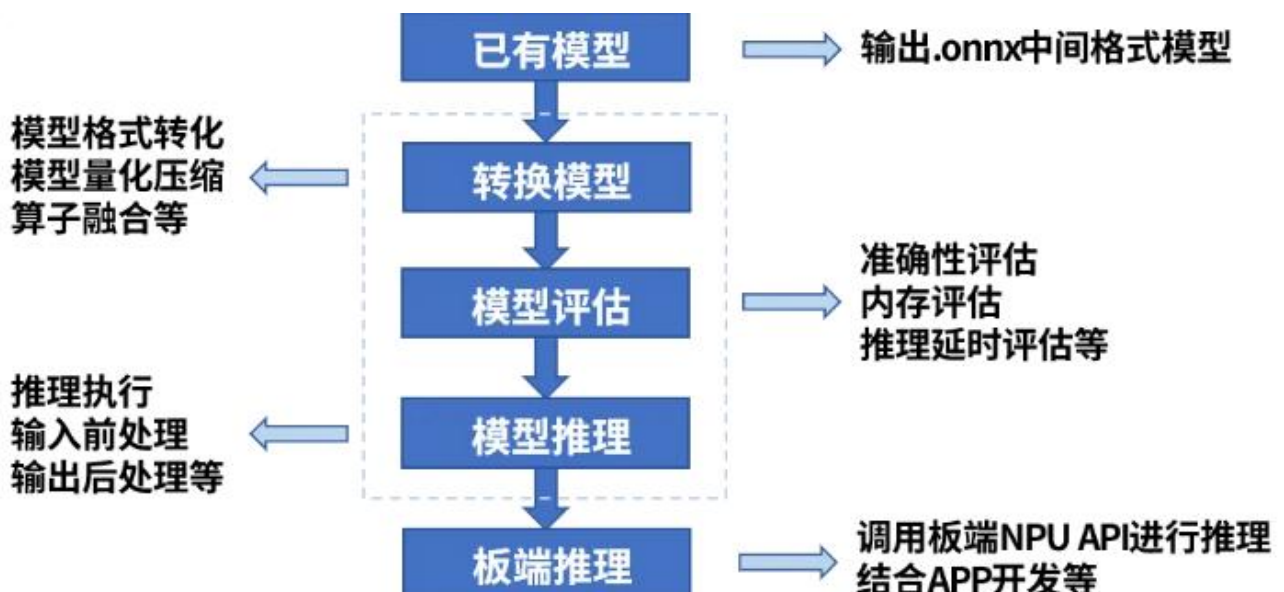
在虚拟机上将 RKNN C Demo 编译完成后目录结果如下：

```
install
├── rk3576_linux_aarch64 # rk3576 平台
└── rknn_resnet_demo
    ├── lib # 依赖库
    ├── model # 存放模型、测试图片等文件
    └── rknn_resnet_demo # 可执行文件
```

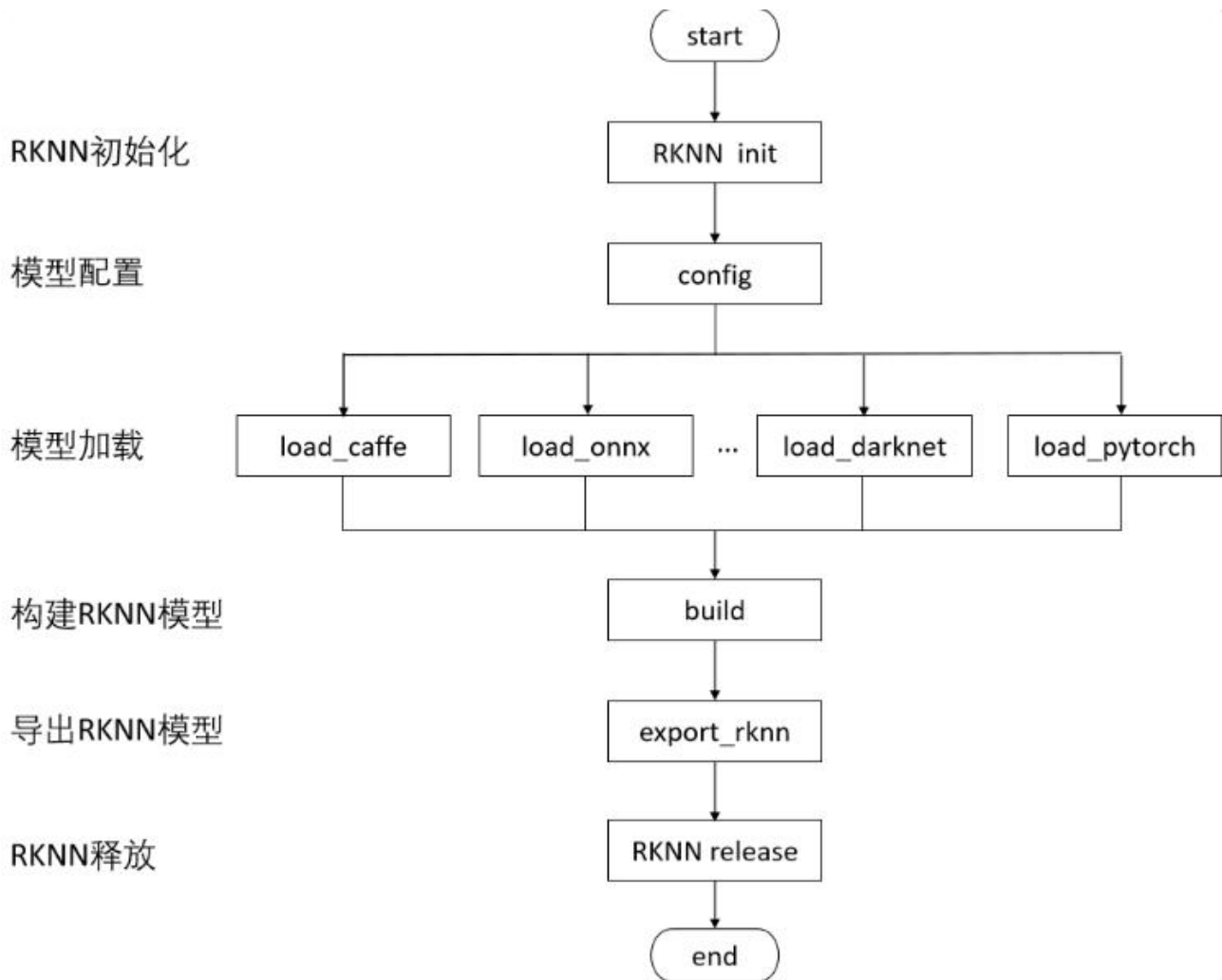
将 rknn_resnet_demo 文件夹拷贝到开发板上，并执行其中的 rknn_resnet_demo 文件，即使用 ResNet 模型对图片进行分类。

5.2. 转换过程

模型转换是 RKNN-Toolkit2 的核心功能之一，它允许用户将各种不同框架的深度学习模型转换为 RKNN 格式以在 RKNPU 上运行。此外嵌入式等部署设备往往受限于内存、磁盘等资源，通常无法直接运行深度学习框架导出的原始模型，需要通过剪枝、量化等压缩操作优化模型。因此模型转换和部署推理的整体流程大致如下：



模型转换是 RKNN-Toolkit2 的核心功能之一，它允许用户将各种不同框架的深度学习模型转换为 RKNN 格式以在 RKNPU 上运行，如下为模型转换流程图以理解如何进行模型转换。

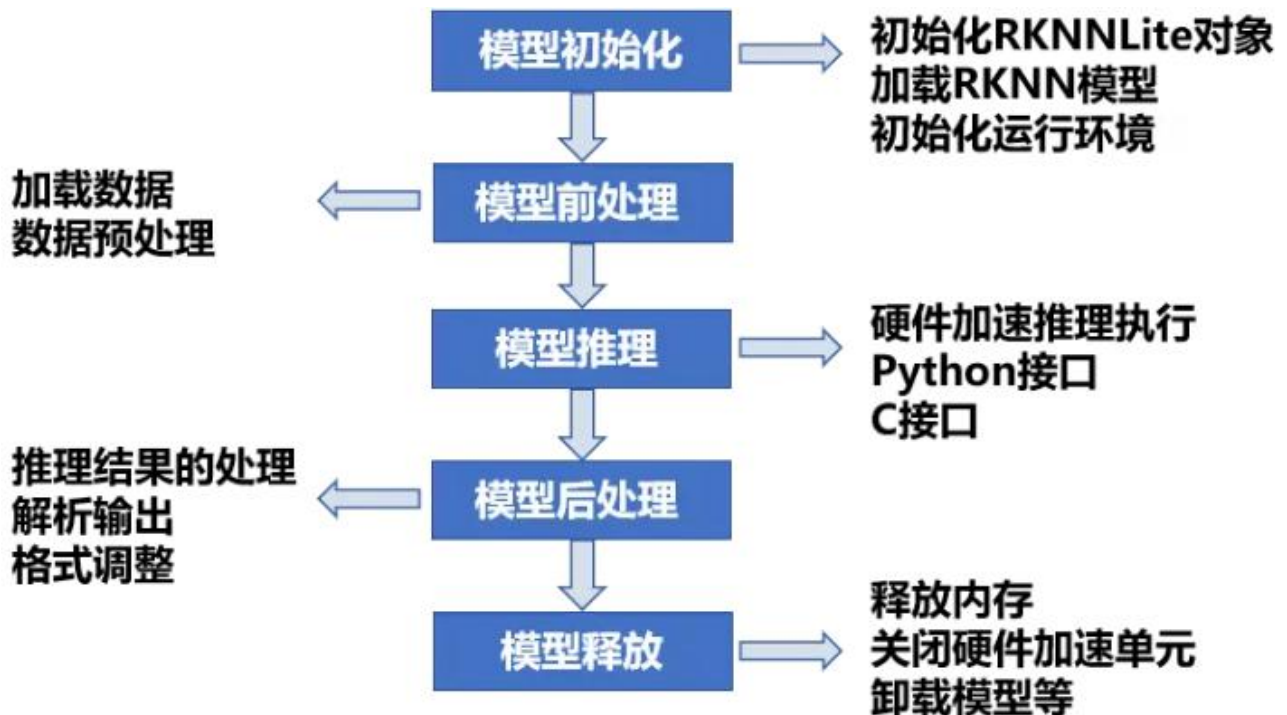


这一阶段包括以下几个步骤：

1. 获取原始模型：获取或训练深度学习模型，建议使用主流框架，如 ONNX、PyTorch 或 TensorFlow。
2. 模型配置：在 RKNN-Toolkit2 中进行必要的配置，如归一化参数、量化参数和目标平台等。
3. 模型加载：使用适当的加载接口将模型导入 RKNN-Toolkit2，根据模型框架选择正确的加载接口。
4. 模型构建：通过 `rknn.build()` 接口构建 RKNN 模型，可选择是否进行量化，提高模型在硬件上推理的性能。
5. 模型导出：通过 `rknn.export_rknn()` 接口将 RKNN 模型导出成一个文件（.rknn 格式），用于后续部署。

5.3. 模型推理

RKNN 模型在 RKNPU 平台的推理流程主要包含模型初始化、数据前处理、模型推理、结果后处理及资源释放五个核心环节，各环节环环相扣，确保推理高效、稳定执行。模型推理流程如下图：



这一阶段包括以下几个步骤：

- 1.模型初始化：加载 RKNN 模型到 RKNPU 平台，准备进行前处理。
- 2.模型前处理：加载待推理数据到 RKNPU 平台，准备进行推理。
- 3.模型推理：执行推理操作，将输入数据传递给模型，调用板端 NPU API 进行推理，并获取推理结果。
- 4.模型后处理：获取推理结果进行后处理，后处理结果传给应用端，结合 APP 开发。
- 5 模型释放：完成推理流程后，释放模型资源，以便其他任务使用 RKNN 模型，避免内存泄漏。

第六章 基于 YOLOv8 的目标识别实战项目

在智能监控、工业质检等场景中，对各类目标的快速识别与定位是提升效率的核心需求。传统人工识别成本高、效率低，而基于深度学习的目标检测技术可实现对复杂场景中多类目标的实时识别与定位。本项目基于当前主流的目标检测算法 YOLOv8，构建一个完整的目标识别实战项目，涵盖从数据准备到部署落地的全流程。本章将围绕模型训练与导出、模型转换和模型部署三个方面详细讲解实际操作流程，在模型部署部分，介绍了多线程推理优化，通过模型池与线程池的协同设计，充分利用 NPU 硬件资源，确保边缘端实时推理性能。本章节内容旨在为读者提供可参考、可落地的实战方案，助力深度学习目标检测技术在嵌入式板卡上的落地部署与应用。

6.1. 模型训练与导出

6.1.1. 环境搭建

6.1.1.1. Anaconda 安装

Anaconda 是一个流行的开源软件包管理和环境管理系统，用于科学计算、数据分析和大数据处理，特别是在 Python 环境中。它包含了许多常用的科学计算和数据分析的软件包，并提供了一个方便的方式来管理这些软件包的安装、更新和环境配置。

以下是 Anaconda 的一些主要特点和用途：

1. 软件包管理：Anaconda 提供了一个名为 conda 的包管理工具，可以轻松安装、更新和删除软件包。它支持数千个科学计算和数据分析相关的软件包，如 NumPy、Pandas、Matplotlib 等。

2. 环境管理：Anaconda 允许用户创建和管理多个独立的 Python 环境。每个环境可以有不同版本的 Python 和不同的软件包集合，这样可以避免版本冲突和环境污染。

3. 跨平台：Anaconda 可以在 Windows、macOS 和 Linux 上运行，使其成为开发人员和研究人员之间共享和复制数据分析项目的理想工具。

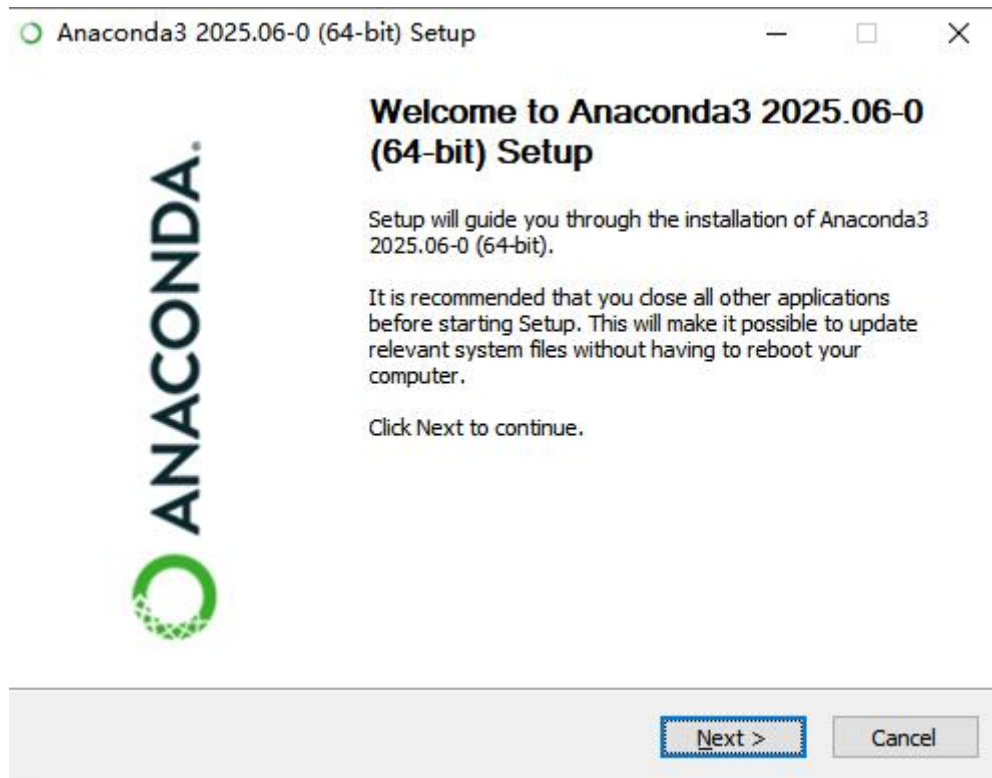
4. 集成开发环境（IDE）：Anaconda 还包含了 Jupyter Notebook 等强大的交互式开发工具，使用户能够在浏览器中创建和分享文档，结合代码、公式、可视化和解释性文本。

5. 大数据支持：Anaconda 不仅适用于单机环境，还支持分布式计算平台，如 Apache Spark，从而能够处理大规模数据和复杂的计算任务。

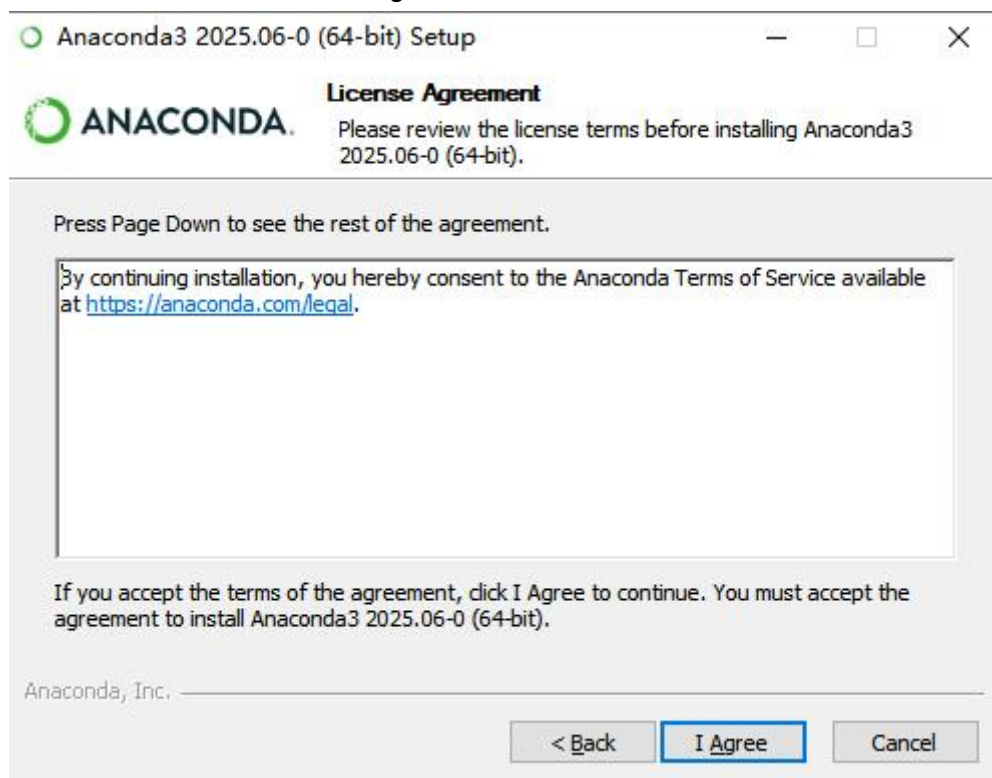
总之，Anaconda 提供了一个全面的解决方案，使得科学计算和数据分析和部署变得更加简单和高效。

首先需要下载 Anaconda 工具，Anaconda 的下载网址 [Download Now | Anaconda](#)，这里下载的是 Anaconda3-2025.06-0-Windows-x86_64.exe 版本。

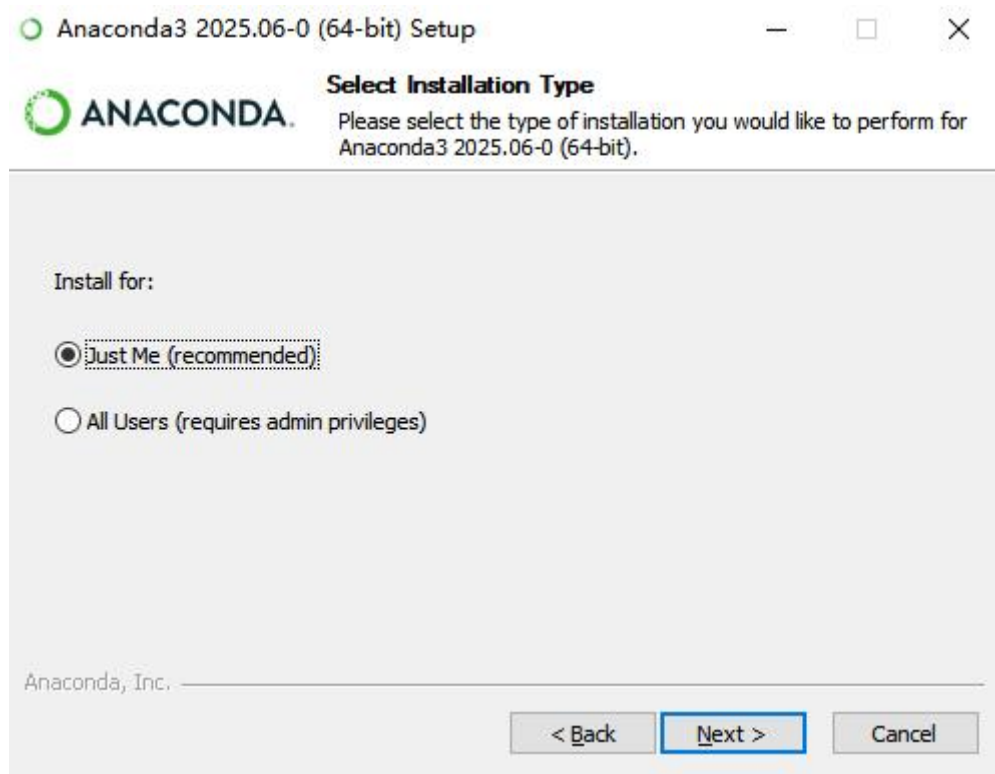
下载完成后双击 Anaconda3-2025.06-0-Windows-x86_64.exe 进行 Anaconda 的安装。



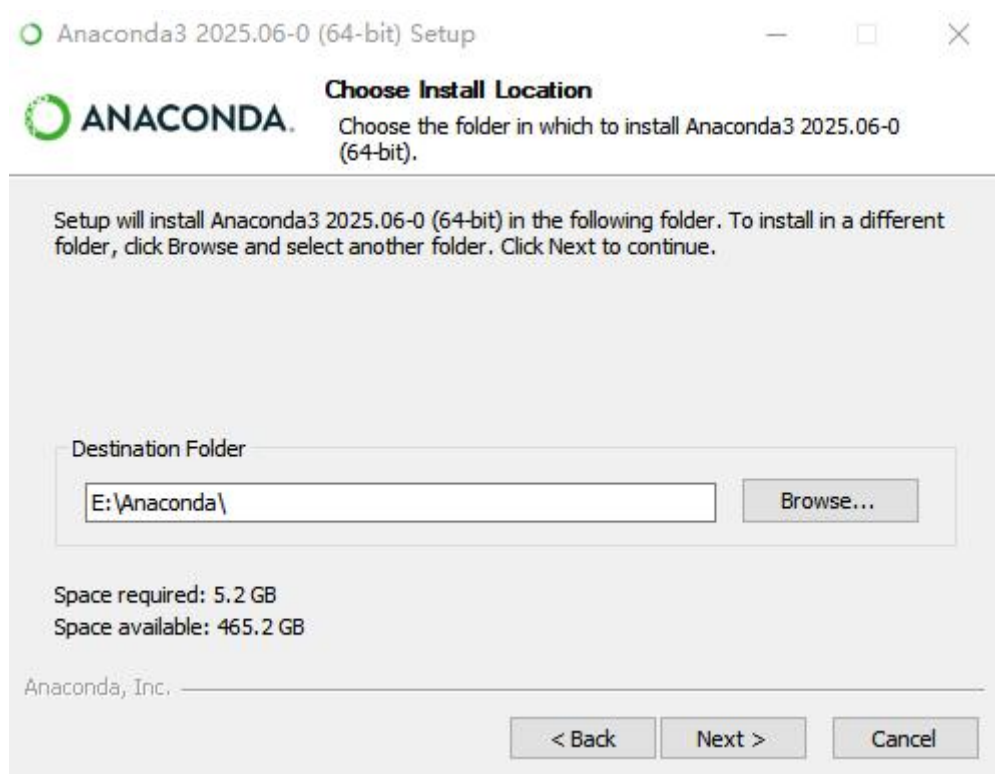
单击 **Next**，进入安装协议页，单击 **I Agree**。



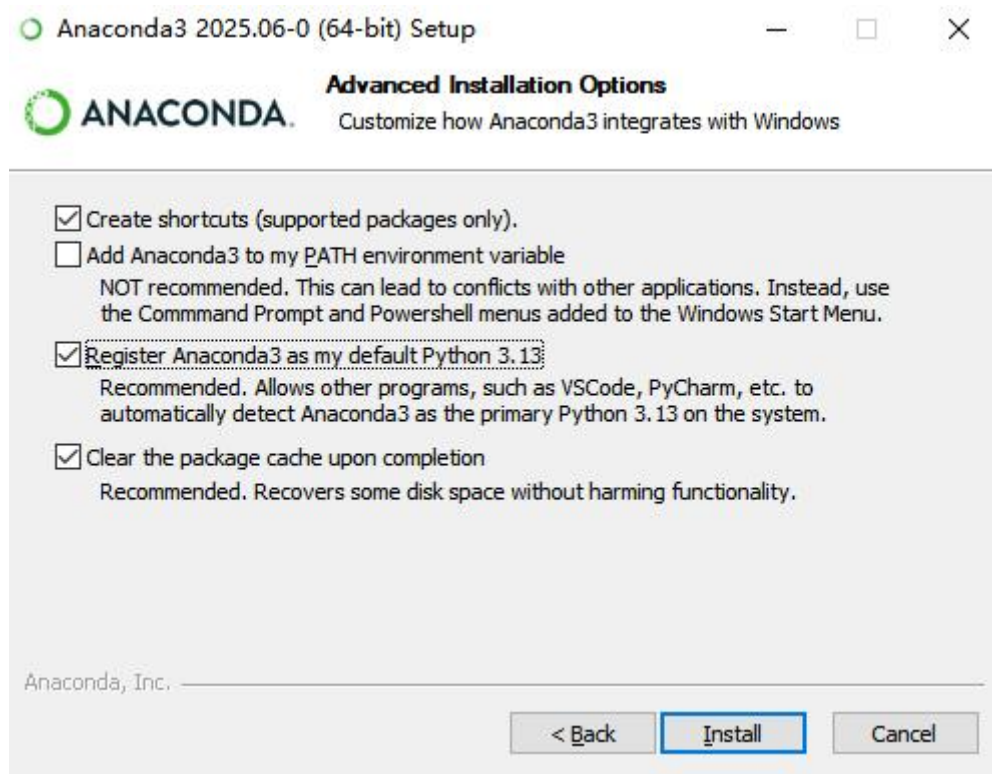
选择 **Just Me** 代表只为当前登录用户安装，**All Users** 代表为所有用户安装，可以根据自己需要选择，一般推荐选择 **Just Me**，单击 **Next**。



单击 **Browse...** 选择要安装 Anaconda 的目录，如果有多个磁盘的话（不建议安装在 C 盘）这里选择安装在 E 盘的 Anaconda 目录下，单击 **Next**。



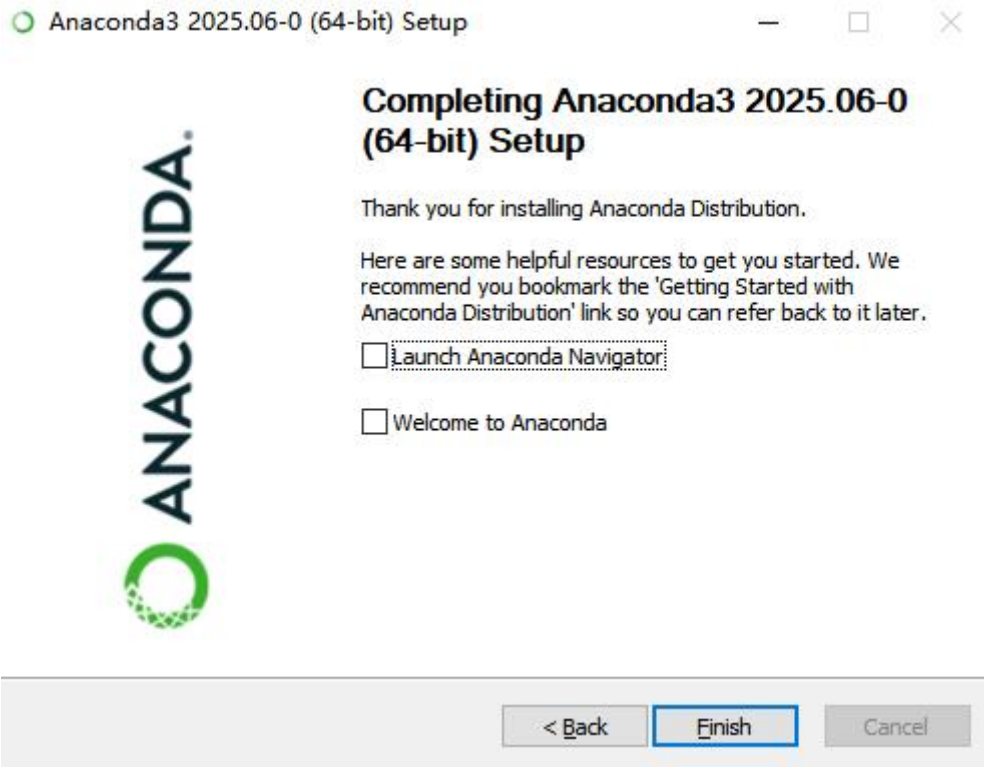
下图复选框分别是创建开始菜单快捷方式、添加 Anaconda 为我的环境变量、注册 Anaconda3 的 python3.13 为默认 Python 环境、完成安装后清理安装包缓存，这里选中第一个、第三个、第四个复选框，点击 **Install**，等待安装完毕。



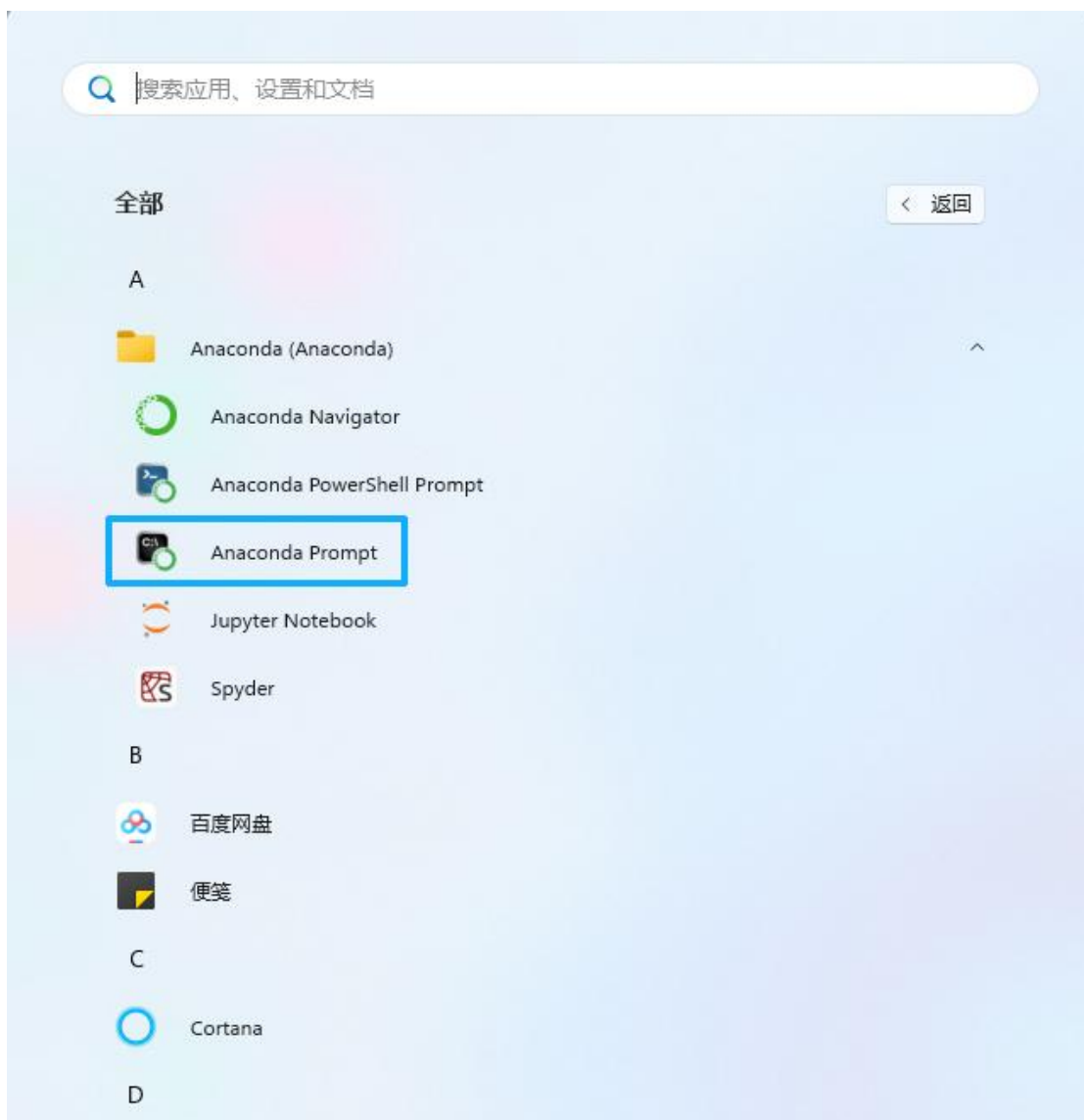
安装完成后，单击 Next。



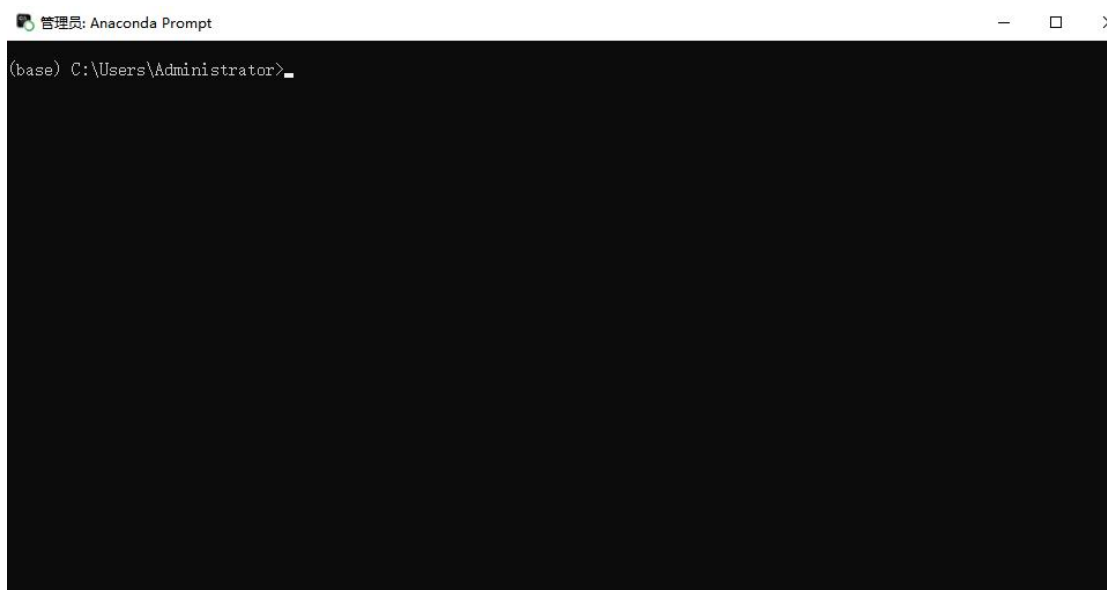
按下图取消勾选两个复选框，点击 Finish。



在开始菜单找到 Anaconda Prompt，单击进入 conda 虚拟环境。



左键单击即可进入虚拟环境界面。





6.1.1.2. Labellmg 安装

Labellmg 是一款用 Python 编写的开源图像标注工具，其图形界面基于 Qt，简洁易用（尽管是英文版）。该工具主要用于分类和目标检测的数据标注，其注释可以保存为 PASCAL VOC 格式的 XML 文件，这也是 ImageNet 所使用的格式。此外，Labellmg 还支持 COCO 数据集格式。接下来，我们将介绍如何使用 Labellmg 进行目标检测数据标注，以及如何使用 Labelme 进行图像分割数据标注。

建议在 Anaconda 虚拟环境里安装 Labellmg，所以这里先介绍 Labellmg 的安装方法，搭建安装 Labellmg 所需要的虚拟环境。

首先，点击 Anaconda Prompt 进入 conda 虚拟环境终端，并使用以下指令来创建一个新的虚拟环境，将新环境命名为 labeling。（注：不建议在 conda 自带的 base 环境里安装）

```
(base) C:\Users\Administrator>conda create --name labeling python=3.9
```

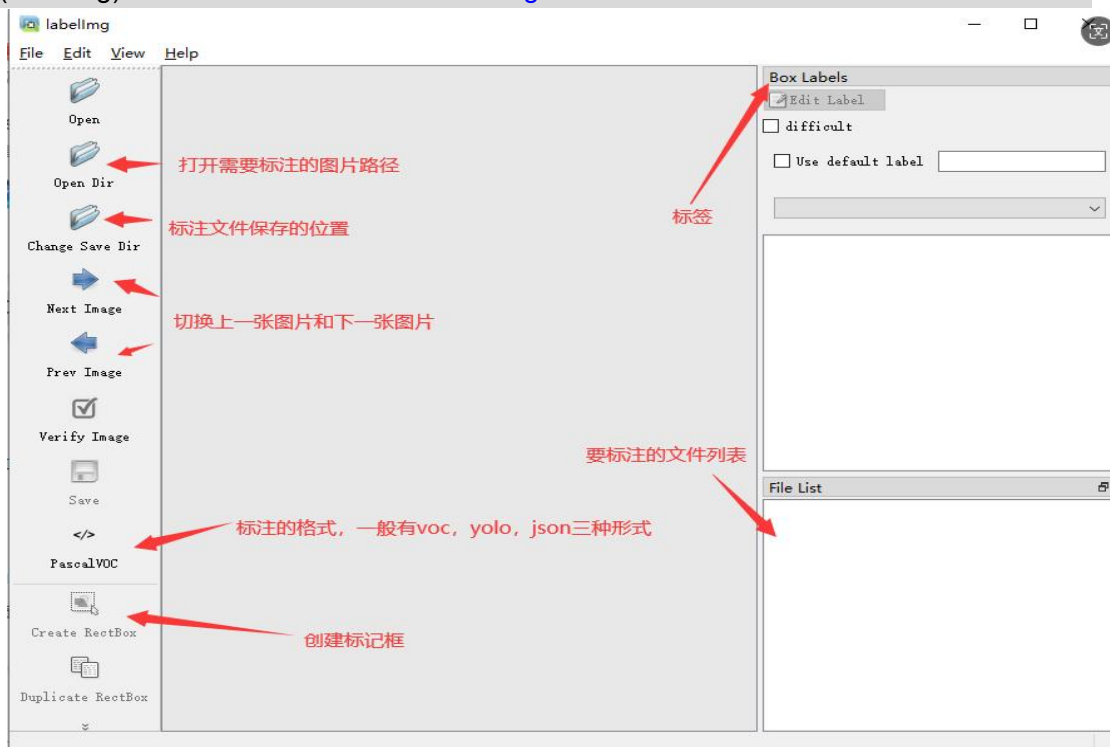
接着，请激活 labeling 虚拟环境，并在该环境中使用以下指令来安装 Labellmg 工具，等待其自动帮您安装 labeling 以及 lxml 和 pyqt5。

```
(base) C:\Users\Administrator>conda activate labeling
```

```
(labeling) C:\Users\Administrator>pip install labellmg -i https://pypi.tuna.tsinghua.edu.cn/simple
```

在安装好 labeling 的虚拟环境里输入 labellmg，等大约三秒左右出现 labellmg 界面。

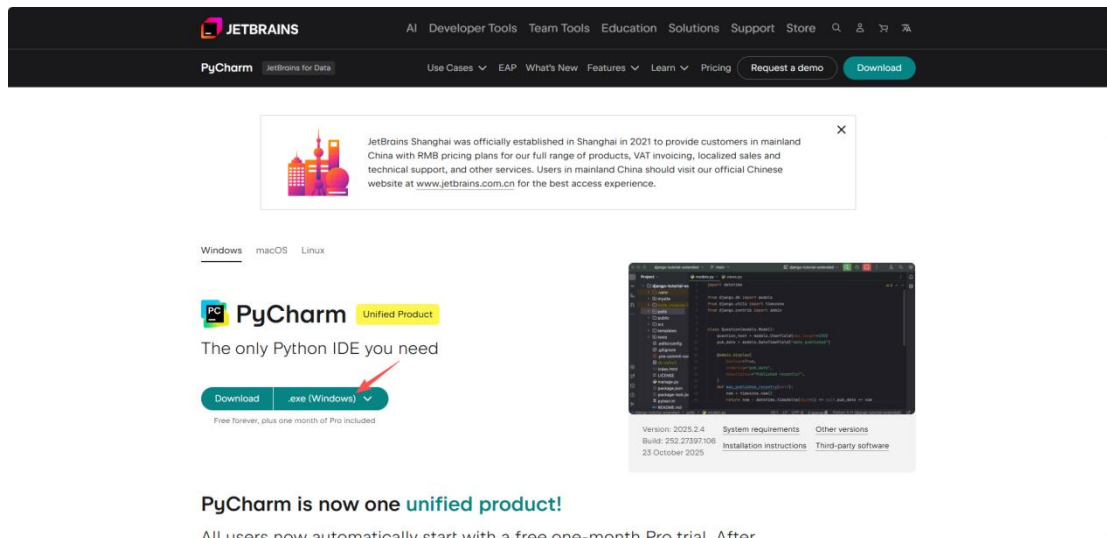
```
(labeling) C:\Users\Administrator>labellmg
```



6.1.1.3. PyCharm 安装

PyCharm 是一种 Python IDE（Integrated Development Environment，集成开发环境），带有一整套可以帮助用户在使用 Python 语言开发时提高其效率的工具，比如调试、语法高亮、项目管理、代码跳转、智能提示、自动完成、单元测试、版本控制。

访问官网 <https://www.jetbrains.com/pycharm/download/#section=windows> 下载 PyCharm 安装包。



下载完成后，双击运行安装包，开始安装，点击下一步。



选择安装位置后，点击下一步。



安装设置界面进行勾选，点下一步。



选择菜单文件夹界面，保持默认，点击安装。



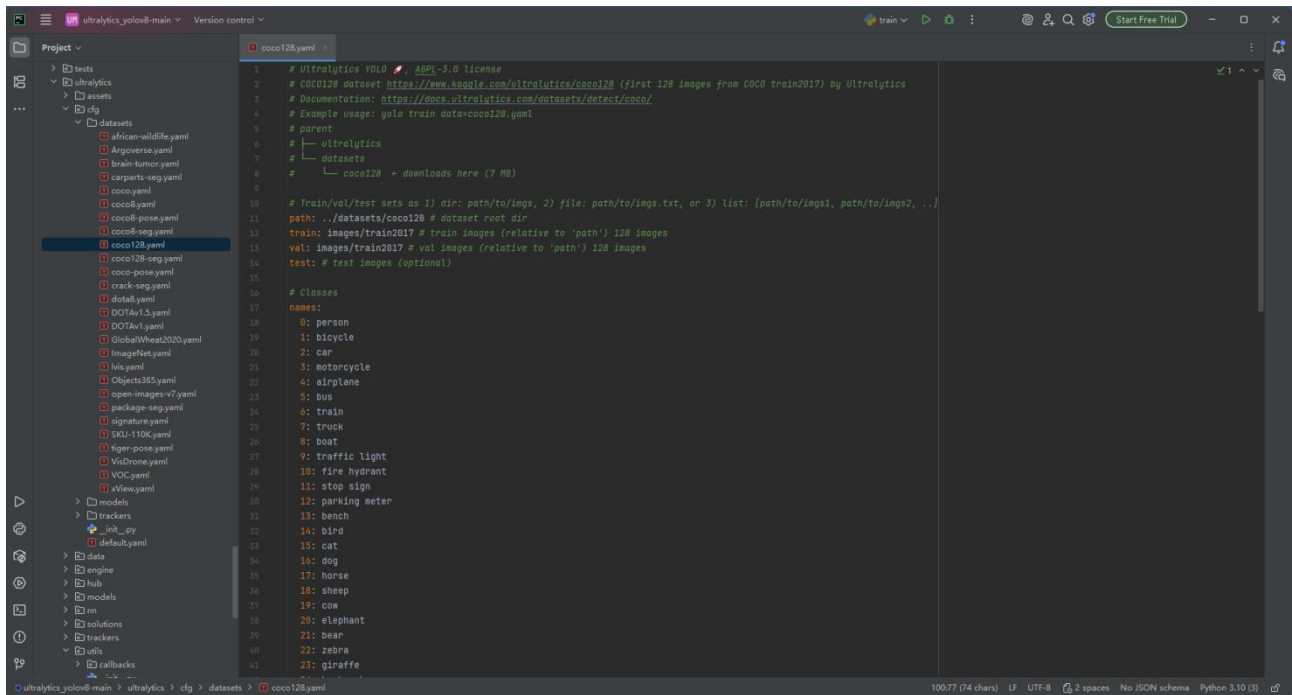
点击完成，完成 PyCharm 的安装。



6.1.2. 数据收集与处理

6.1.2.1. 数据集获取和标注方法

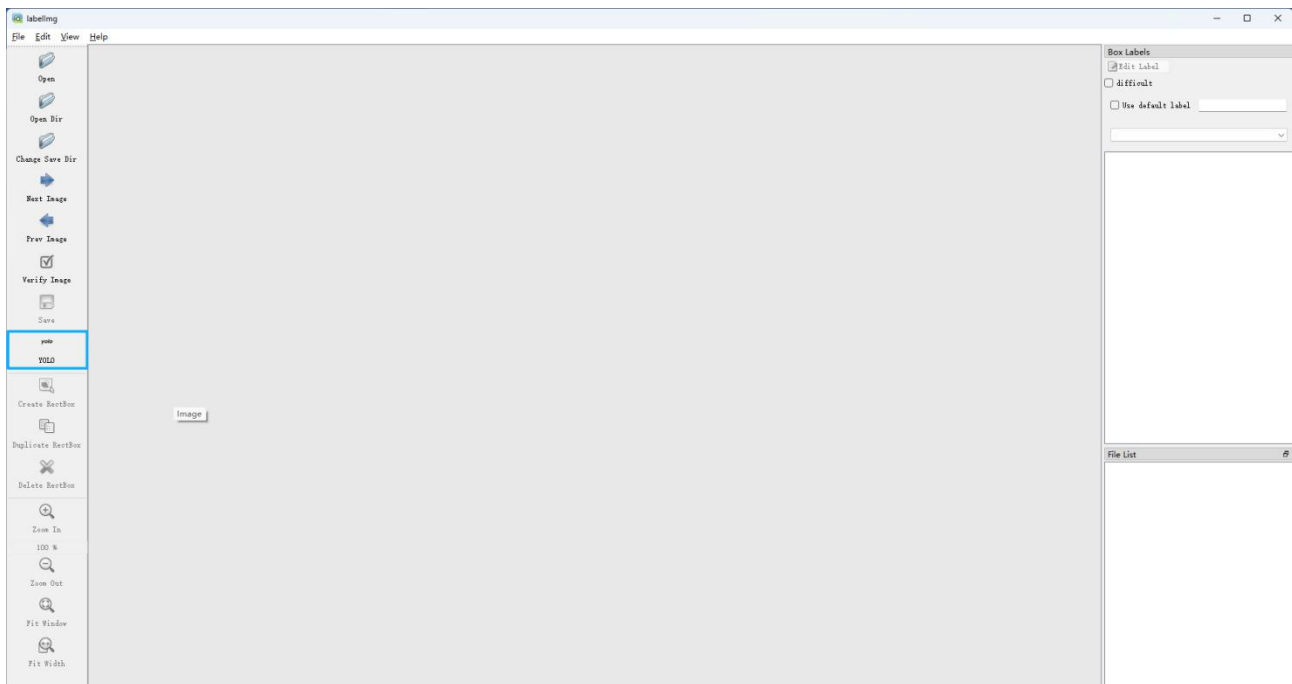
本项目使用开源数据集 coco128, coco128 是一个小型但功能多样的对象检测数据集，由 COCO train 2017 数据集的前 128 张图像组成。在项目中直接指定 `data='coco128.yaml'`，模型训练过程中会直接调用项目文件 `ultralytics/cfg/datasets` 路径下的 `coco128.yaml`。



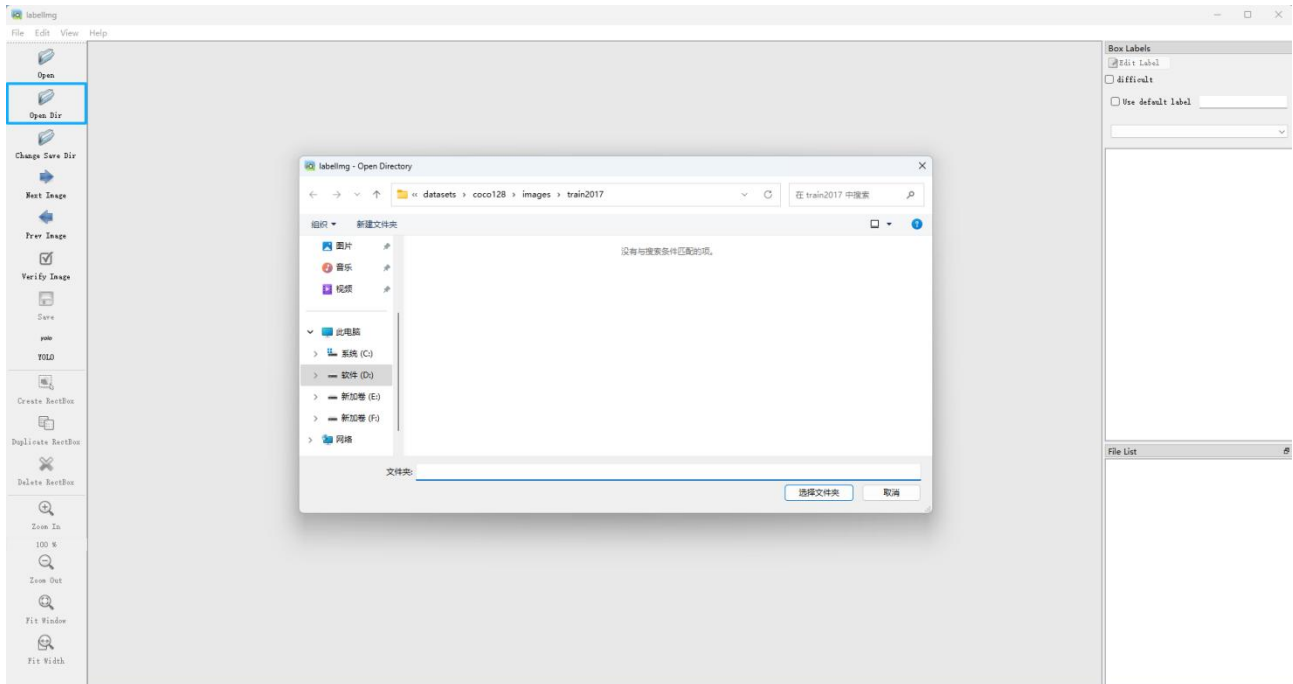
上图是 coco128.yaml 文件内容，主要定义数据集目录的路径、数据集中所有目标类别的名称。模型训练前会从网页 <https://github.com/ultralytics/assets/releases/download/v0.0.0/coco128.zip> 自动下载 coco128 数据集压缩包并解压缩到项目文件 datasets 目录下。

若使用自己的数据集，首先对没有标签的图像数据集进行标注，这里介绍使用 Labellmg 进行数据集标注。Labellmg 支持 PascalVOC（标注保存后会得到与图像同名的 xml 文件）、CreateML（保存标注后会获得图像同名的 json 文件）和 YOLO 三种标注格式。

首先，点击左侧标注的格式将标注格式切换到 YOLO。

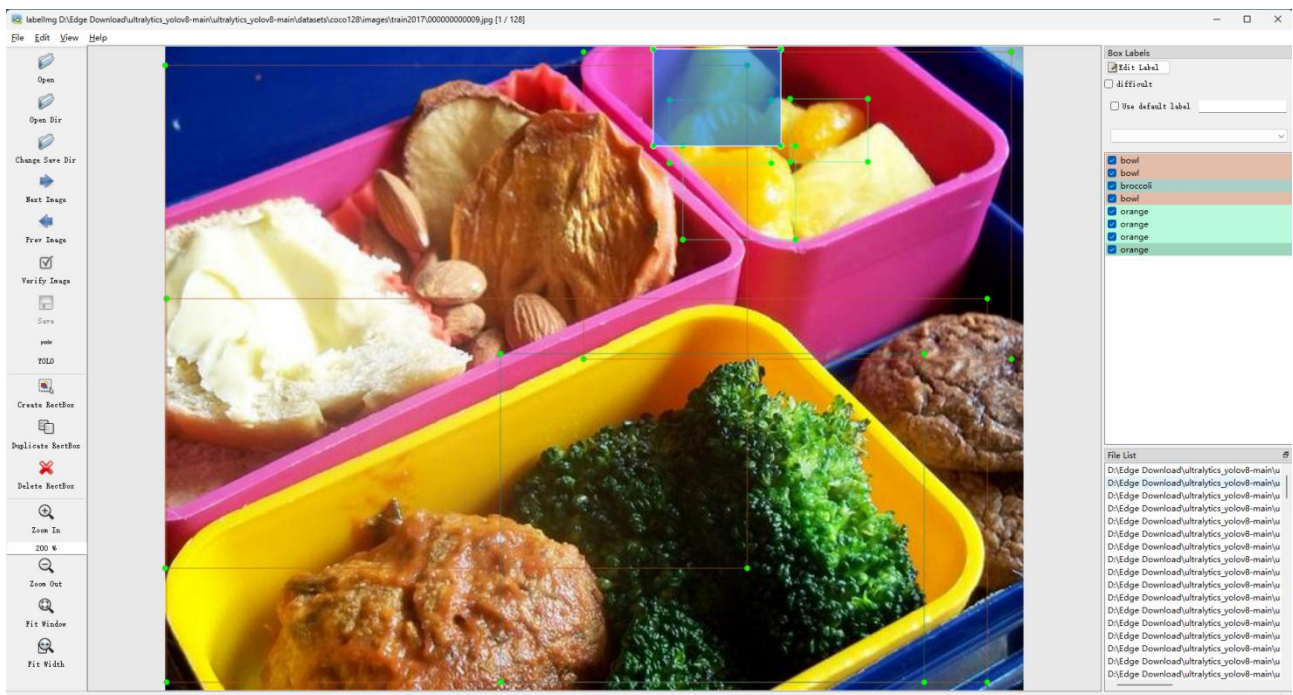


然后，点击 Open Dir 打开需要标注的图片文件夹。

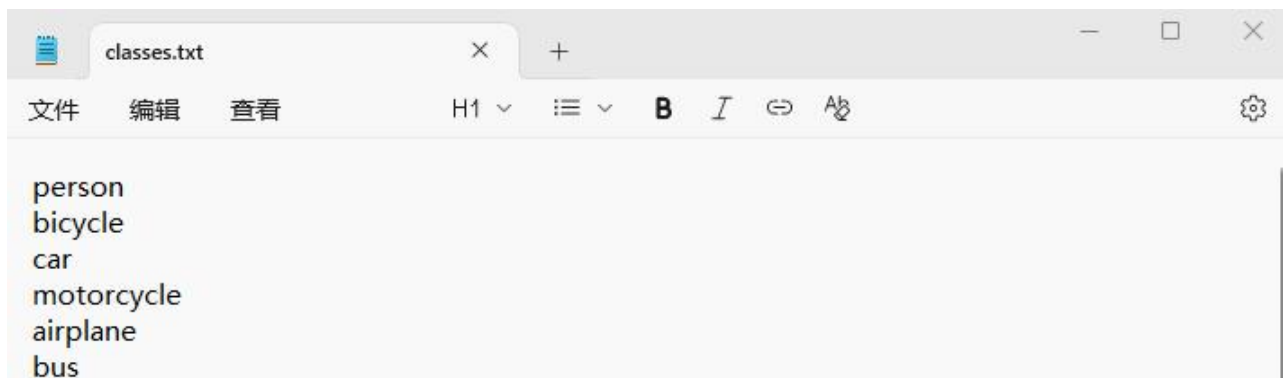


并点击 **Change Save Dir** 设置标注文件保存的目录。

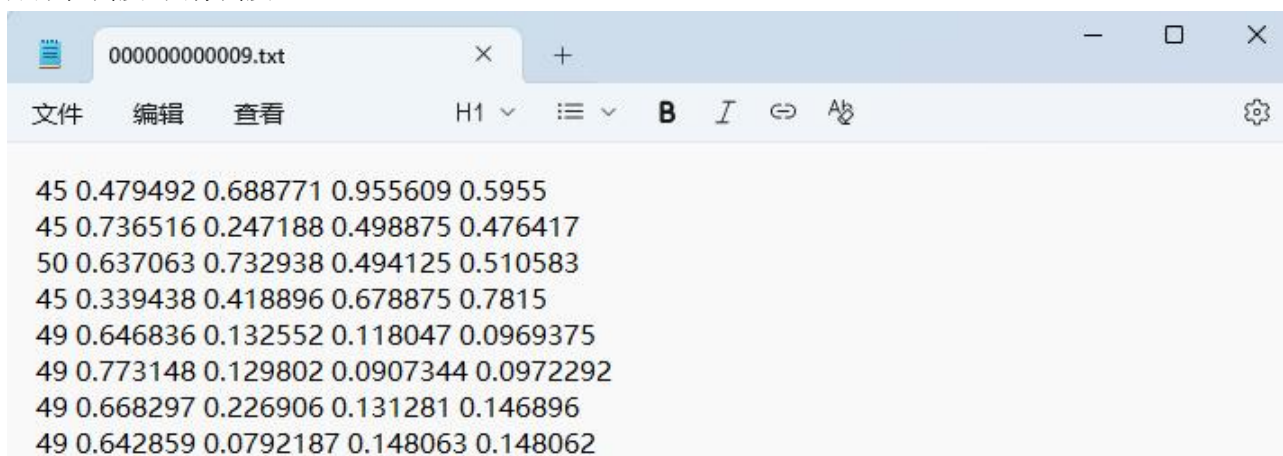
最后，点击左侧的 **Create RectBox** 进行画框，并输入标签名进行标注，这里以项目中使用的 **coco128** 数据集的图片为例。



点击左侧的 **Save** 进行保存，会在刚才设置标注文件保存的目录下生成图像同名的 **txt** 文件和 **classes.txt**，**classes.txt** 无论标注多少张只会出现一次，内容为标签名称，如果下一张有新的标签会在 **classes.txt** 中再添加。

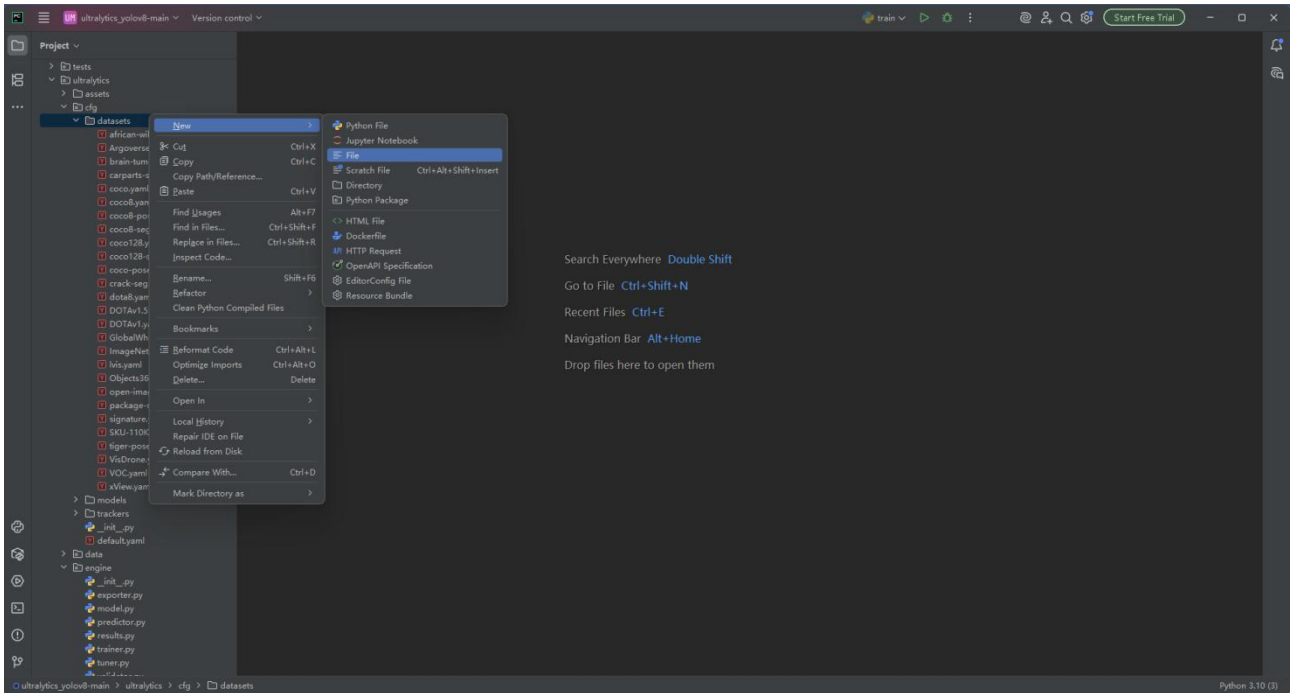


与图像同名的 **txt** 文件（000000000009.txt）内容如下，第一列是标签的索引，后面四列依次是归一化坐标：目标中心点 **x** 坐标÷图像宽度、目标中心点 **y** 坐标÷图像高度、目标矩形框宽度÷图像宽度、目标矩形框高度÷图像高度。

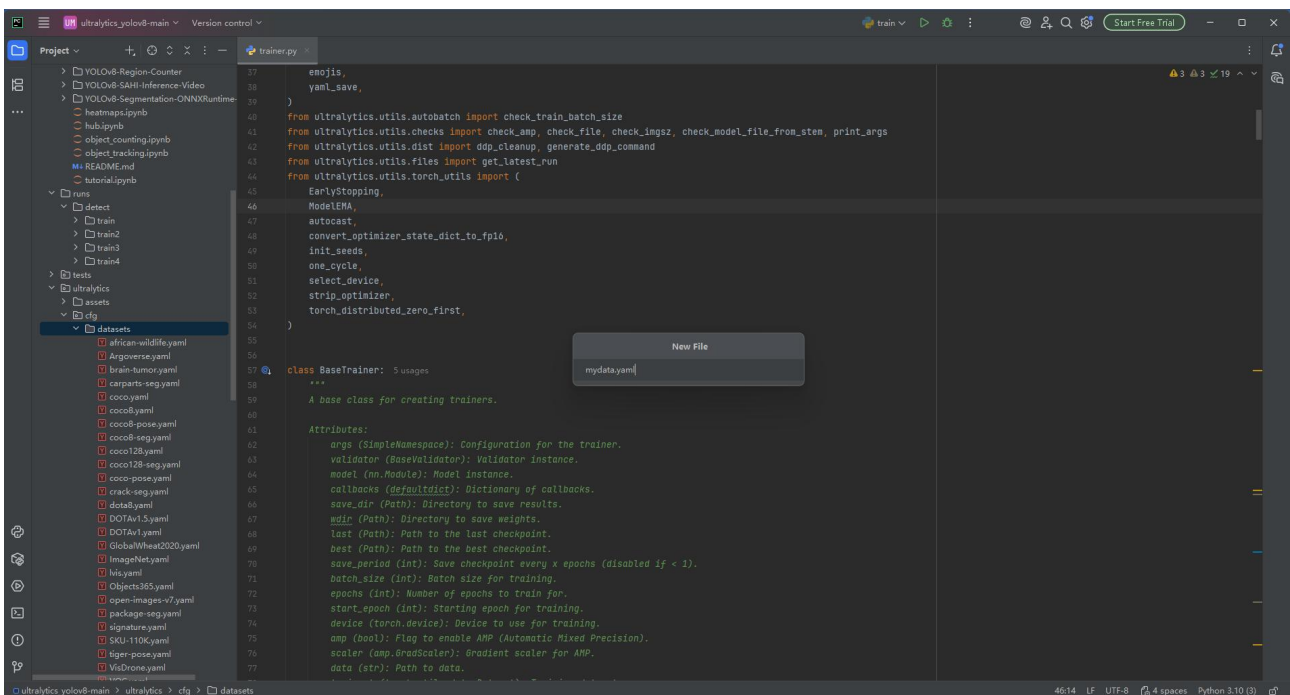


若使用自己的数据集，将自己的数据集收集标注完后需要进行整理。首先，在 **ultralytics_yolov8-main** 项目文件路径下创建一个 **datasets** 文件夹，在 **datasets** 文件夹中建立一个名为 **mydatasets** 的文件夹用于存放数据集，在 **mydatasets** 文件夹里分别创建一个名为 **train** 的文件夹用于存放训练数据集和一个名为 **val** 的文件夹用于存放验证数据集。然后，分别在 **train** 文件夹和 **val** 文件夹下创建一个名为 **images** 的文件夹和一个名为 **labels** 的文件夹，用于存放训练数据集的图片和标签文件以及验证数据集的图片和标签文件。最后，将收集好的图片和标签文件按照一定比例放在创建好的对应的文件夹里。

整理完数据集之后，为了在训练的时候能读取数据集，需要在项目文件 **ultralytics/cfg/datasets** 目录下创建数据集配置文件。这里以创建名为 **mydata.yaml** 的数据集配置文件为例。右击项目文件目录下的 **ultralytics/cfg/datasets**，点击 **New**，选择 **File** 进行文件创建。



输入 mydata.yaml 文件名完成数据集配置文件的创建。



在刚才创建好的数据集配置文件 **mydata.yaml** 定义数据集路径和类别信息。通过 **path** 指定数据集根目录为 **../datasets/mydatasets**，再以相对路径分别定义训练集（**train/images**）、验证集（**val/images**）的图片存放位置；随后通过 **names** 字段列出了 17 个类别的 ID 与对应名称（比如 ID=0 对应“person”、ID=1 对应“bicycle”），让模型能识别数据中的目标类别。其中，**test** 字段用于指定测试集的路径，是可选字段，这里 **test** 字段为空（未设置），实际使用时可根据需求补充。在 **mydata.yaml** 数据集配置文件中，数据集路径是以刚才整理创建的 **mydatasets** 数据集为例，以 17 个数据集类别为例。在实际使用中，数据集路径和类别信息须根据自身数据集的实际情况进行调整。



```
trainer.py  mydata.yaml x
1  path: ../datasets/mydatasets # dataset root dir
2  train: train/images # train images (relative to 'path')
3  val: val/images # val images (relative to 'path')
4  test: # test images (optional)
5
6  # Classes
7  names:
8      0: person
9      1: bicycle
10     2: car
11     3: motorcycle
12     4: airplane
13     5: bus
14     6: train
15     7: truck
16     8: boat
17     9: traffic light
18     10: fire hydrant
19     11: stop sign
20     12: parking meter
21     13: bench
22     14: bird
23     15: cat
24     16: dog
```

6.1.2.2. 数据集划分

在 YOLOv8 项目中，数据集的划分完全由用户控制，框架仅通过配置文件中的路径“识别”已划分好的训练/验证集，不涉及自动拆分逻辑。用户需要提前完成数据划分（比如按 8:2 比例拆分原始数据到 train/val 文件夹），再在配置文件中指定对应路径。但是为了让读者更轻松的理解和使用数据集划分功能，下面以 python 脚本的方式提供数据集划分实例，将原始图片和标签文件按照指定比例自动划分为训练集、验证集和测试集，并分别存储到新的目录结构中。

划分数据集脚本路径：[进阶篇之-基于 RK 平台的 AI 模型训练到部署\AI 例程源码\数据集划分.py](#)

实际使用的时候只需修改数据集路径和划分比例：

```
if __name__ == '__main__':
    # 1. 修改这里的路径，指向你的图片和标签文件夹
    file_path = "coco128/images/train2017" # 需要划分的图片文件夹
    xml_path = 'coco128/labels/train2017'   # 需要划分的标签文件夹

    # 2. 修改这里的路径，指定划分后的数据保存位置
    new_file_path = "coco"                  # 保存划分好的路径

    # 3. 修改这里的比例，根据你的需求调整训练、验证、测试集的占比
    split_data(file_path, xml_path, new_file_path, train_rate=0.7, val_rate=0.1, test_rate=0.2)
```




6.1.3. 模型训练

6.1.3.1. 环境配置

这里以 CPU 环境下 YOLO8 环境配置为例进行 yolov8 环境的配置。

点击 Anaconda Prompt，使用 Anaconda 执行命令创建虚拟环境。

```
(base) C:\Users\Administrator>conda create --name yolov8 python=3.10
```

激活刚才创建的 yolov8 虚拟环境。

```
(base) C:\Users\Administrator>conda activate yolov8
```

使用阿里云镜像安装 ultralytics、tqdm、pandas。

```
(yolov8) C:\Users\Administrator>pip install ultralytics -i https://mirrors.aliyun.com/pypi/simple/  
(yolov8) C:\Users\Administrator>pip install tqdm -i https://mirrors.aliyun.com/pypi/simple/  
(yolov8) C:\Users\Administrator>pip install pandas -i https://mirrors.aliyun.com/pypi/simple/  
(yolov8) C:\Users\Administrator>pip uninstall numpy  
(yolov8) C:\Users\Administrator>pip install "numpy<2" -i https://mirrors.aliyun.com/pypi/simple/
```

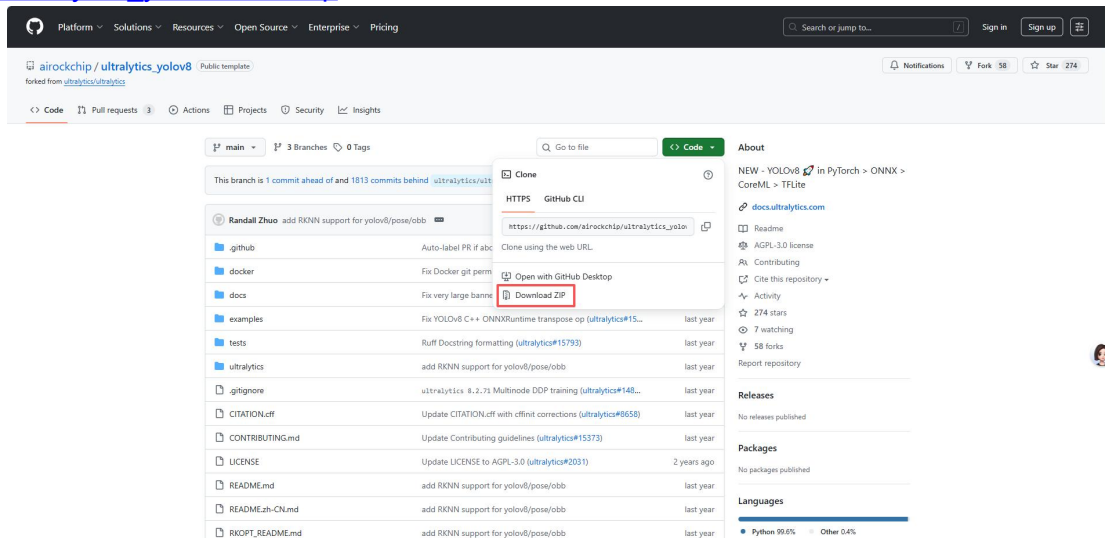
安装 Ultralytics 时无需单独安装 PyTorch、OpenCV 或 NumPy 等依赖包。这是因为执行 `pip install ultralytics` 命令时，安装程序会检测当前环境，如果尚未安装 PyTorch，便会自动安装与其兼容的 CPU 版本 PyTorch，而 PyTorch 自身又会关联安装 OpenCV 等其他必要依赖库。

需要注意的是，安装 Ultralytics 过程中默认会安装 NumPy 2.x 版本。然而在后续模型训练时，由于当前环境使用 NumPy 2.x，而 Ultralytics 依赖的部分模块是基于 NumPy 1.x 编译的，两者之间存在兼容性问题，会导致训练过程失败。因此，需要在安装完成后手动卸载 NumPy 2.x 版本，重新安装 1.x 版本以确保兼容性。

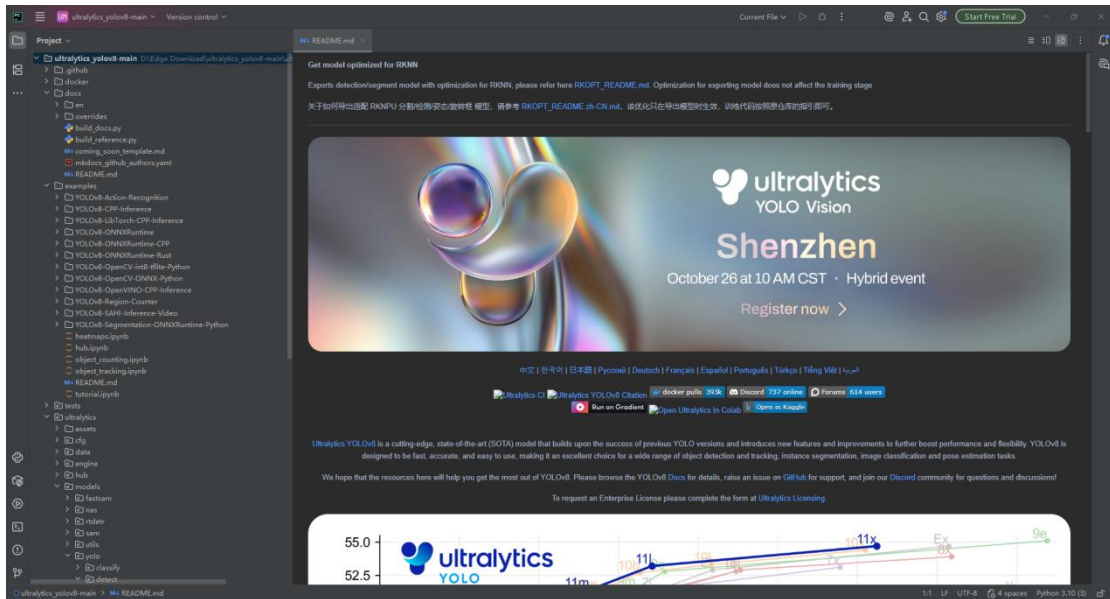
从以下地址下载 `ultralytics_yolov8-main` 源码并解压：

https://github.com/airockchip/ultralytics_yolov8

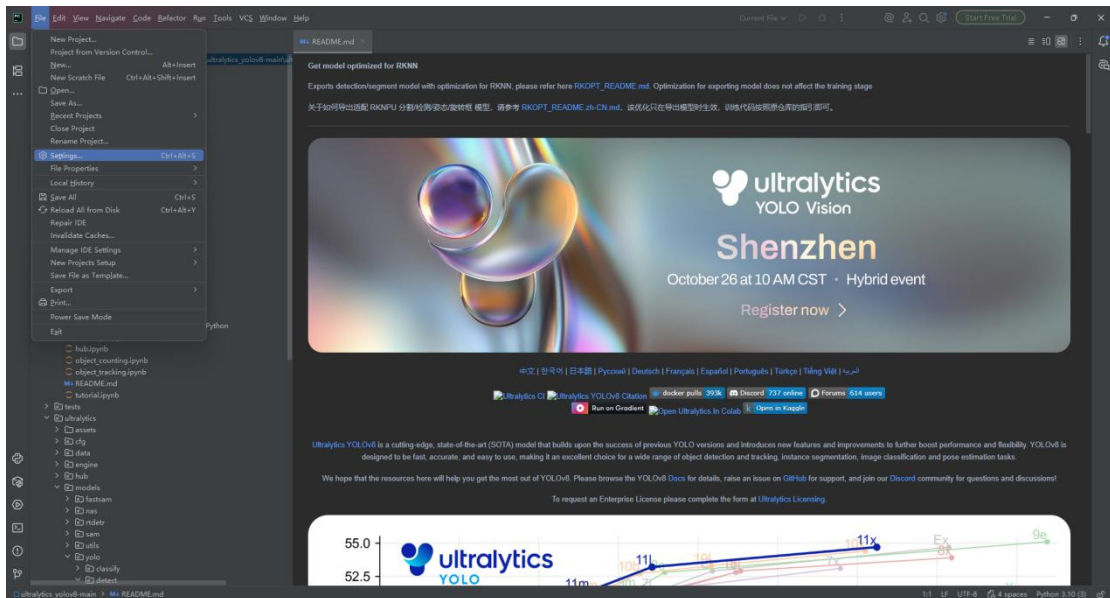
也可以直接使用资料包中提供的源码，源码路径：[进阶篇之-基于 RK 平台的 AI 模型训练到部署AI 例程源码ultralytics_yolov8-main.zip](#)



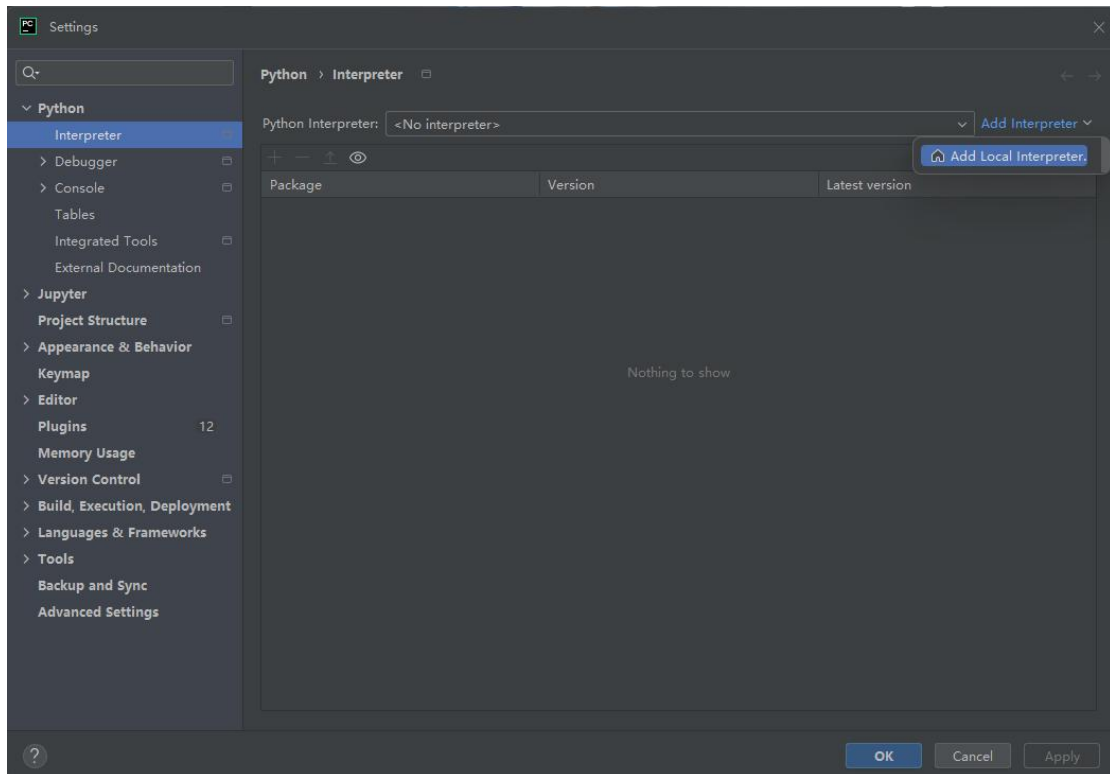
使用 Pycharm 打开。



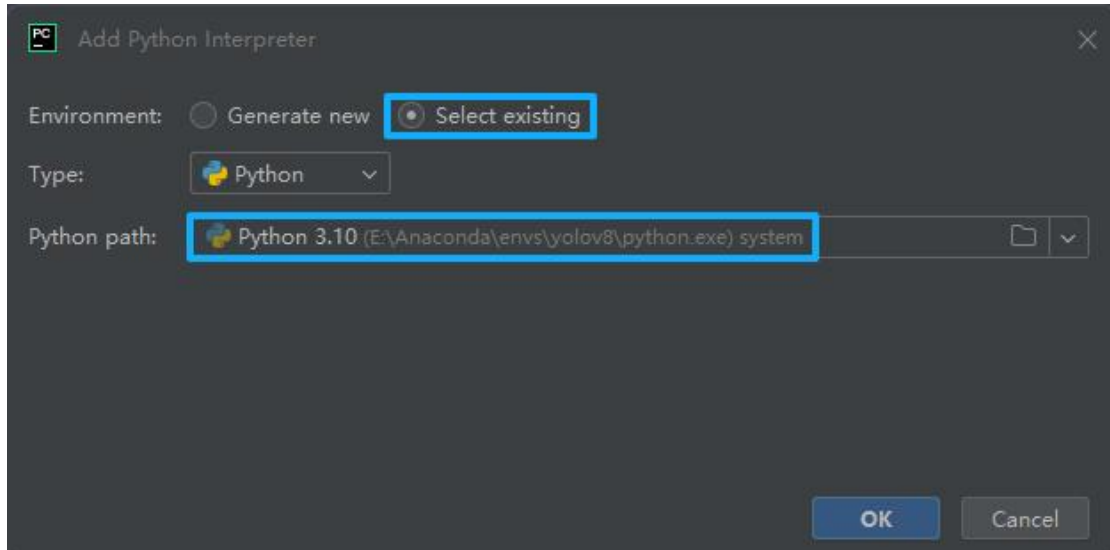
打开主菜单，点击 Settings。



点击 Python 下的 Interpreter，点击 Add Local Interpreter。



点击 **Select existing**，并选择 **Python path**，这里选择刚才创建好的 yolov8 虚拟环境里的 Python 解释器，位于 E 盘 Anaconda 安装目录下的 envs 文件夹里的 yolov8 里的 python.exe，然后点击 **OK**（这里以自己实际安装 anaconda 的位置为准）。



6.1.3.2. 训练模型

添加好 Python 解释器后，进行模型训练，训练方式有两种，一种是通过 Python 程序训练，另一种是通过终端输入 CLI 命令进行训练。这里以 Python 程序训练为例：

```
from ultralytics import YOLO

# 这里有三种训练方式，三种任选其一

#第一种：根据yami文件构建一个新模型进行训练,若对YOLO8网络进行了修改（比如添加了注意力机制）适合选用此种训练方式。但请注意这种训练方式是重头训练（一切参数
```



都要自己训练), 训练时间、资源消耗都是十分巨大的
`model = YOLO('yolov8n.yaml') # build a new model from YAML`

#第二种: 加载一个预训练模型, 在此基础之前对参数进行调整。这种方式是深度学习界最主流的方式。由于大部分参数已经训练好, 我们仅需根据数据集对模型的部分参数进行微调, 因此训练时间最短, 计算资源消耗最小。

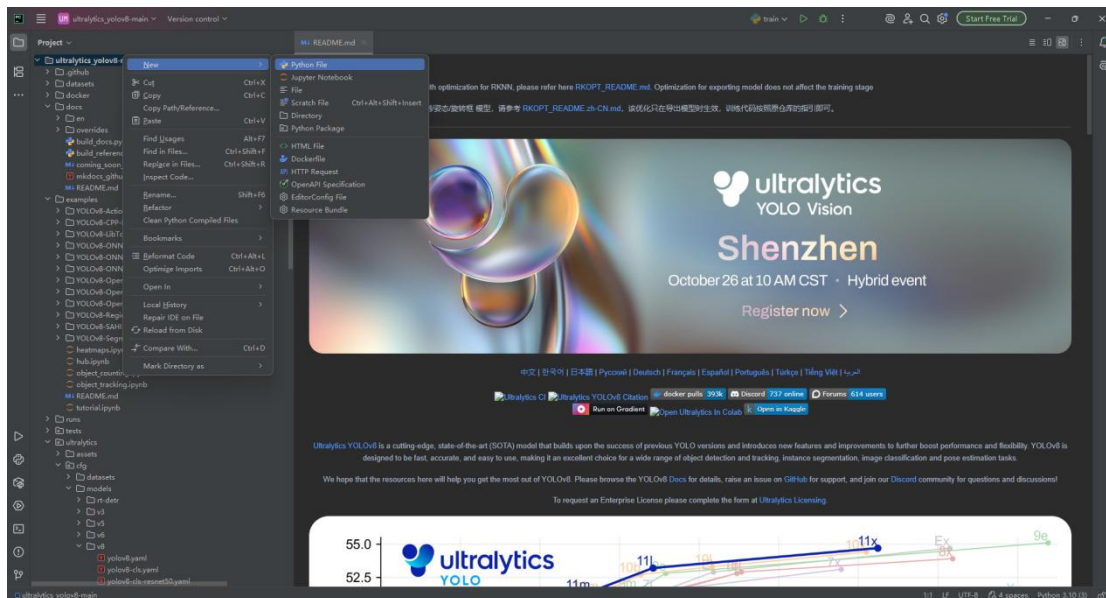
`model = YOLO('yolov8n.pt') # load a pretrained model (recommended for training)`

#第三种: 根据yaml文件构建一个新模型, 然后将预训练模型的参数转移到新模型中, 然后进行训练, 对YOLO8网络进行改进的适合选用此种训练方式, 而且训练时间不至于过长
`model = YOLO('yolov8n.yaml').load('yolov8n.pt') # build from YAML and transfer weights`

#Train the model

`results = model.train(data='data.yaml', epochs=100, imgsz=640)`

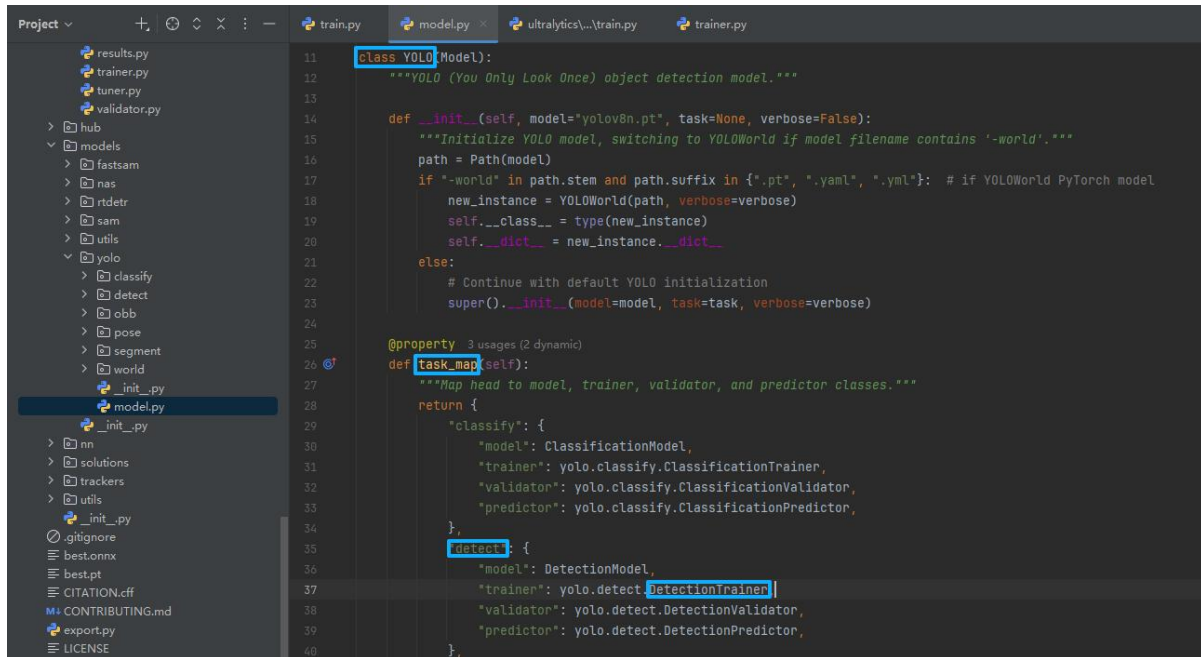
在项目 `ultralytics_yolov8-main` 目录下新建 python 文件, 命名为 `train.py`。



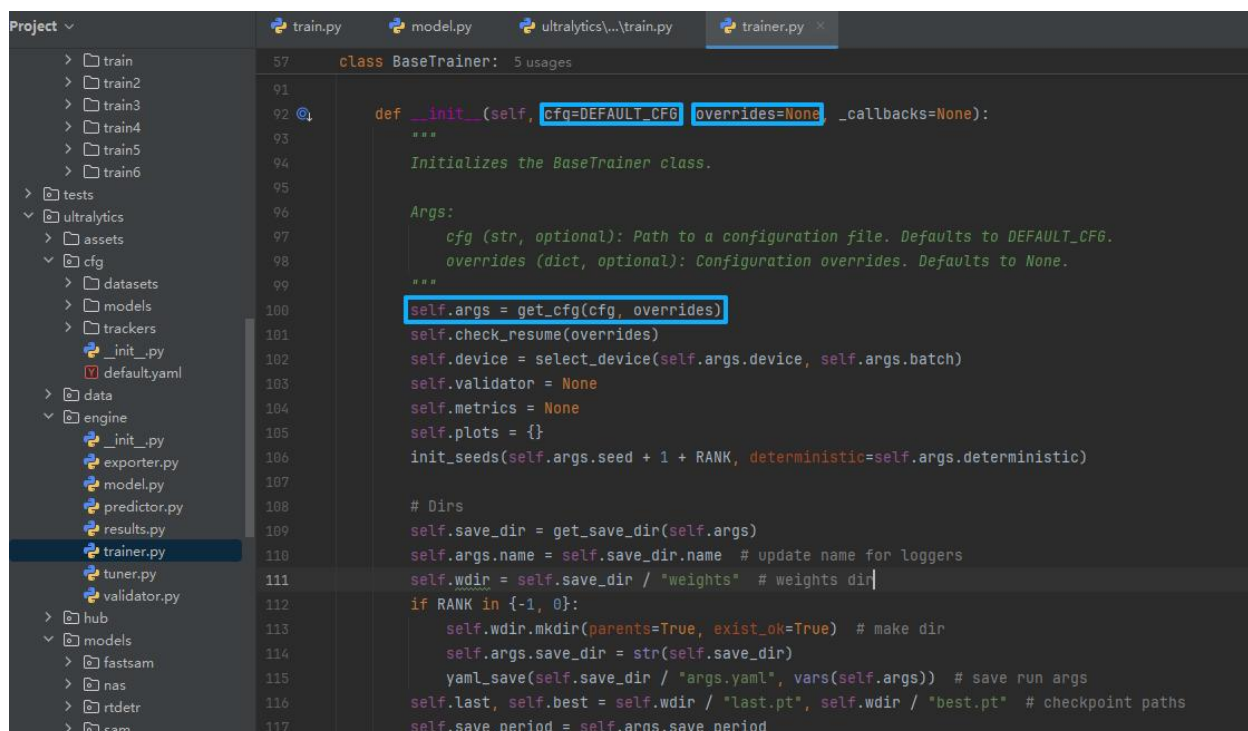
这里以第二种训练方式为例, 训练过程中会自动下载 YOLOv8 的预训练权重文件 `yolov8n.pt` 并加载该模型, 训练使用 `coco128` 数据集, 指定输入图片大小为 `640×640`, 训练 100 个轮次, `train.py` 内容如下:

```
train.py
1 from ultralytics import YOLO
2
3 model = YOLO('yolov8n.pt')
4
5 results = model.train(data='coco128.yaml', epochs=100, imgsz=640)
```

首先, 从 `ultralytics` 库中导入 `YOLO` 类, `YOLO` 类是 `Ultralytics YOLO` 框架的核心入口类, 封装了目标检测、分割、分类等多种任务的模型加载、训练等功能。然后, 进行模型初始化, `model = YOLO('yolov8n.pt')` 加载预训练的 YOLOv8 nano 模型 (`yolov8n.pt`), 内部会自动解析模型结构并关联目标检测任务对应的处理类, 通过 `YOLO` 类的 `task_map` 属性, 绑定 `detect` 任务的 `DetectionTrainer`。



最后，传递训练参数进行训练，`model.train(data='coco128.yaml', epochs=100, imsz=640)`触发训练流程，传入的参数包括：`data='coco128.yaml'`，指定数据集配置文件；`epochs=100`，设置训练总轮次为100；`imsz=640`，指定输入图片尺寸为 640x640。除了这些显式传入的参数，框架还会通过默认配置文件（`ultralytics/cfg/default.yaml`）加载其他未显式指定的参数，形成完整的训练配置。用户显式传入的参数优先级最高，会覆盖默认配置文件中的对应值。



在 `trainer.py` 的初始化过程中（`BaseTrainer` 的 `__init__` 方法），框架会通过 `get_cfg` 函数合并默认配置、用户传入参数，生成最终的训练参数字典（`self.args`）。其中 `cfg` 用于指定默认配置，`cfg` 默认传入 `DEFAULT_CFG`，就是 `default.yaml` 解析后的默认参数，`overrides` 是用户显式传入的参数，这里，`overrides=None` 只是 `__init__` 方法的默认参数值，表示“如果没有传入自定义参数，则使用默认配置”。在实际训练中，用户通过 `model.train(...)` 传入参数，`overrides` 会被实际参数覆盖。在 4.2 节主要文件介绍中



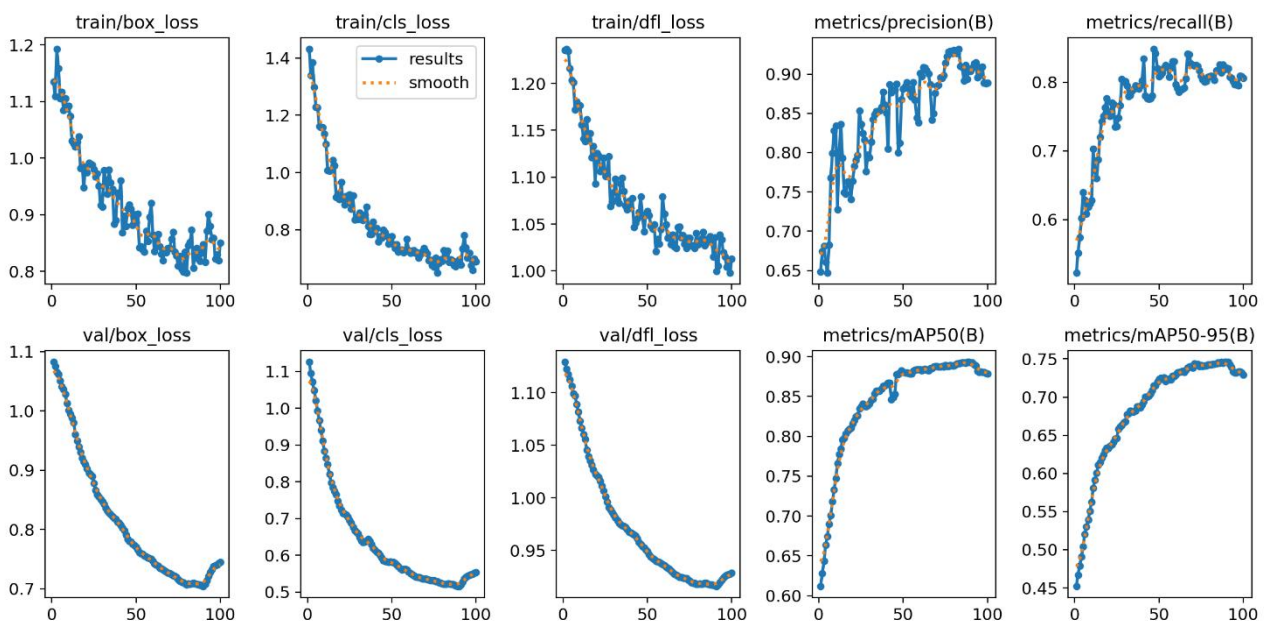
已经对 `trainer.py` 和 `default.yaml` 文件进行了介绍，这里不再赘述。在本项目中，除了显示指定的 `data`、`epochs` 和 `imgsz` 这三个参数外，其他参数会使用 `default.yaml` 解析后的默认参数。本项目中配置的主要参数如下：

参数	含义
<code>task=detect</code>	指定任务类型为目标检测
<code>mode=train</code>	运行模式为训练
<code>model=yolov8n.pt</code>	指定使用的预训练模型， <code>yolov8n.pt</code> 是 YOLOv8 的“nano”版本
<code>data=coco128.yaml</code>	指定数据集配置文件路径为 <code>coco128.yaml</code>
<code>epochs=100</code>	训练总轮数，即整个数据集将被迭代训练 100 次
<code>batch=16</code>	批次大小，每次迭代输入模型的图像数量为 16 张
<code>imgsz=640</code>	输入图像的尺寸，训练时所有图像会被调整到 640×640
<code>device=None</code>	指定训练设备， <code>None</code> 表示自动选择
<code>workers=8</code>	数据加载的进程数设置为 8
<code>optimizer=auto</code>	自动选择优化器
<code>lr0=0.01</code>	初始学习率设置为 0.01
<code>lrf=0.01</code>	最终学习率因子为 0.01，最终学习率= $\text{lr0} \times \text{lrf}$
<code>weight_decay=0.0005</code>	权重衰减参数为 0.0005（L2 正则化系数，防止过拟合）

这些参数需要根据用户的具体场景（如数据集特性、硬件条件、任务需求等）灵活调整，以达到最佳训练效果。若用户需要修改参数，可通过上文提到的显示指定或修改 `default.yaml` 文件的默认参数配置。

运行 `train.py` 进行训练，调用 `model.train()` 方法后，框架通过 `task_map` 找到 `DetectionTrainer`（继承自 `engine/trainer.py` 的 `BaseTrainer`），并实例化训练器。随后自动执行 `BaseTrainer` 中的核心逻辑，包括解析参数、加载数据集、初始化优化器、执行训练循环、验证模型、保存权重等，最终完成 100 轮训练并返回训练结果。训练过程中会先下载 `coco128` 数据集到 `datasets` 文件目录下，训练结果会保存到 `runs/detect/train` 目录下，训练好的模型权重文件会保存在 `runs/detect/train/weights` 文件下，`best.pt` 为最优模型权重文件，`last.pt` 为最后模型权重文件。

训练结果的损失曲线和其他性能指标曲线如下图所示。



图中损失曲线包含 `box_loss`（边界框回归损失）、`cls_loss`（类别分类损失）、`df_l_loss`（分布焦点损失），曲线整体呈下降趋势，说明模型在训练集上的拟合效果逐步提升。验证集的损失曲线同样持续下降



且与训练损失趋势一致，表明模型未出现过拟合，泛化能力良好。模型预测为正例的样本中实际为正例的比例 **precision(B)**（精确率）最终稳定在 0.9 左右，说明预测结果的准确性较高。实际为正例的样本中被模型预测为正例的比例 **recall(B)**（召回率）最终稳定在 0.8 左右，说明模型对目标的覆盖能力较好。**IoU** 阈值为 0.5 时的目标检测精度 **mAP50(B)** 最终接近 0.9，**IoU=0.5~0.95** 时的平均精度最终约 0.75，反映了模型在不同严格程度下的综合检测能力。上图反映了模型的评估指标，可帮助用户判断训练是否充分、是否需要调整超参数（如学习率、训练轮次等）。

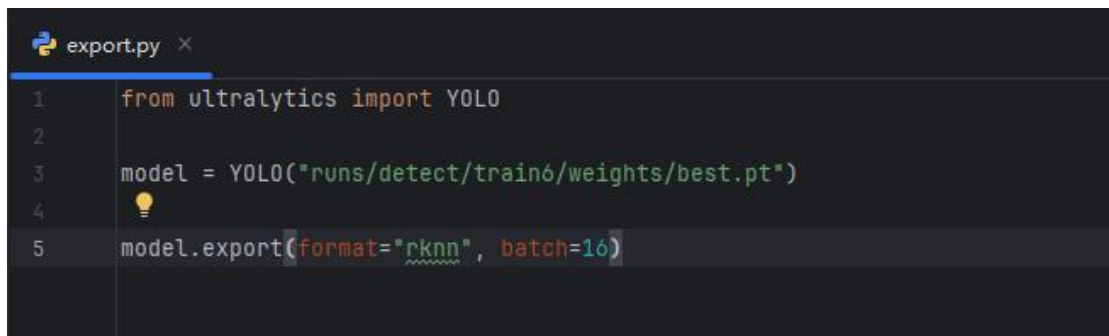
6.1.4. 模型导出

在 PyCharm 中，Alt+F12 调出终端，安装依赖，这里以自己实际安装 **anaconda** 的位置为准：

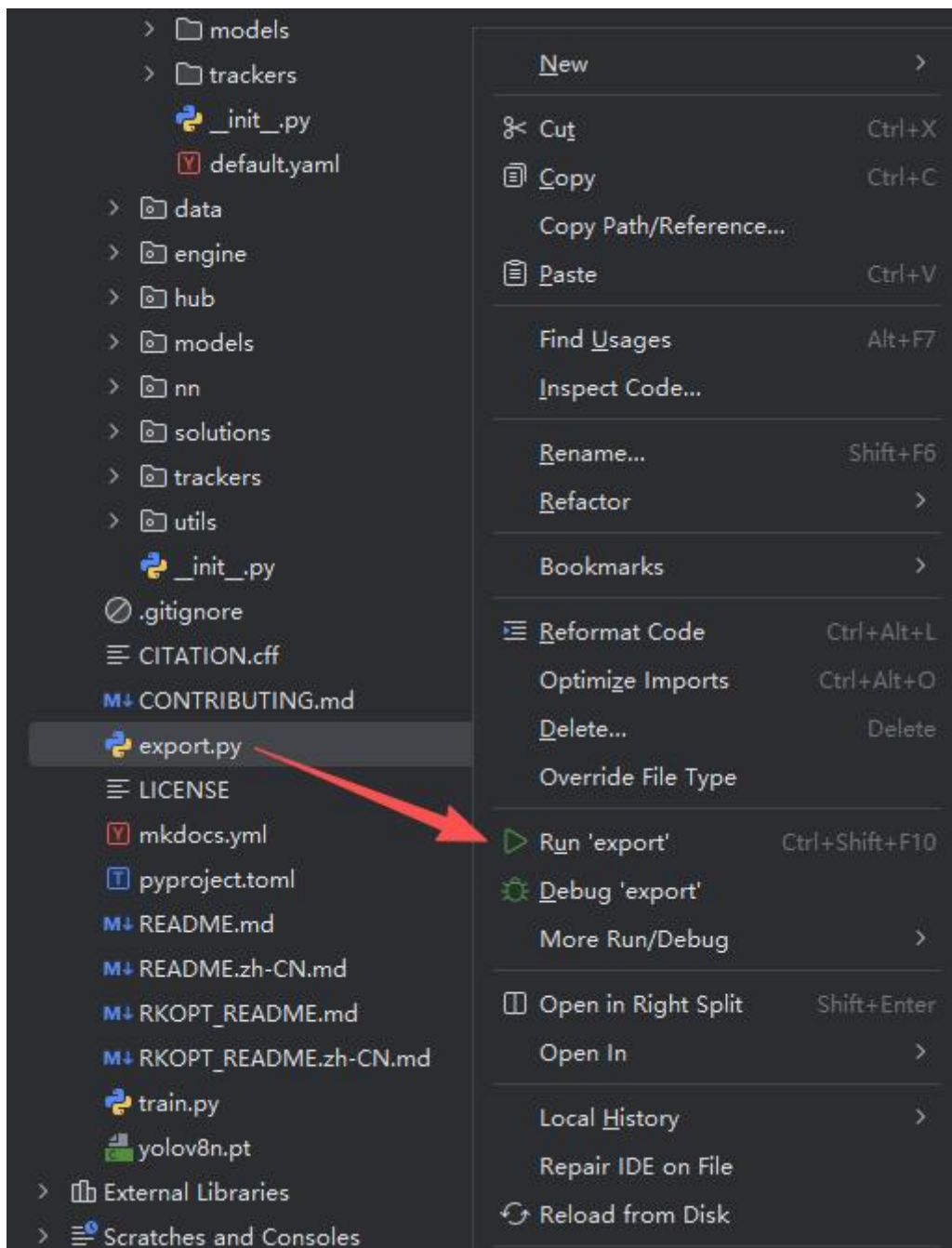
```
E:\Anaconda\envs\yolov8\python.exe -m pip install onnxscript -i  
https://pypi.doubanio.com/simple/
```

在 **ultralytics_yolov8-main** 项目文件路径下创建 **export.py**，指定需要导出的模型路径和需要导出的格式：

```
from ultralytics import YOLO  
  
model = YOLO("runs/detect/train5/weights/best.pt")    //以自己实际路径为准  
  
model.export(format="rknn", batch=16)
```



鼠标右键点击 **export.py** 文件，选择 **Run 'export'**，运行 **export.py** 导出 **.onnx** 模型：



导出的模型 **best.onnx** 将保存到与 **best.pt** 相同文件路径下。

如果出现 **best.onnx.data** 文件，说明 PyTorch 版本过高导致导出的模型权重未内嵌，引发模型转化的时候大量权重未被正确加载，可通过降低 PyTorch 版本解决，参考第七章 FAQ 问题 2。

6.2. 模型转换

RK 模型转换需依赖 RKNN-Toolkit2 工具和 RKNN Model Zoo 资源库，本文档中不同开发板型号对应不同版本组合：其中 ELF 2_Lite 和 ELF-RV1126B 开发板需搭配 RKNN-Toolkit2v2.3.2 与 RKNN Model Zoo v2.3.2 版本，而 ELF 2 开发板则需使用 RKNN-Toolkit2v2.1.0 与 RKNN Model Zoo v2.1.0 版本，通过这些工具可实现模型向 RKNN 格式的转换，以适配相应嵌入式平台的部署需求。以下以 ELF 2_Lite 开发板为例进行模型转换中的环境搭建，安装相应版本的 RKNN-Toolkit2 工具和 RKNN Model Zoo 资源库，ELF-RV1126B 开发板模型转换中的环境搭建与 ELF 2_Lite 开发板一致，ELF 2 开发板模型转换中的环境



搭建可仿照 ELF 2_Lite 开发板进行不同版本的 RKNN-Toolkit2 工具和 RKNN Model Zoo 资源库安装。

6.2.1. 环境搭建

6.2.1.1. 安装 Python 环境

```
elf@ubuntu:~$ sudo apt-get update
elf@ubuntu:~$ sudo apt-get install python3 python3-dev python3-pip
elf@ubuntu:~$ sudo apt-get install libxslt1-dev zlib1g zlib1g-dev libglib2.0-0 libsm6
libgl1-mesa-glx libprotobuf-dev gcc
```

6.2.1.2. 安装 Miniforge 工具

下载 Miniforge 安装脚本，也可以直接使用资料包中提供的安装脚本，路径：[进阶篇之-基于 RK 平台的 AI 模型训练到部署\AI 例程源码\Miniforge3-Linux-x86_64.sh](#)

```
elf@ubuntu:~$ sudo su
root@ubuntu:/home/elf# wget -c
https://mirrors.tuna.tsinghua.edu.cn/github-release/conda-forge/miniforge/LatestRelease/Miniforge3-Linux-x86_64.sh
```

安装 Miniforge:

```
root@ubuntu:/home/elf# chmod 777 Miniforge3-Linux-x86_64.sh
root@ubuntu:/home/elf# bash Miniforge3-Linux-x86_64.sh
```

6.2.1.3. 创建 RKNN-Toolkit2 Conda 环境

进入 Conda base 环境:

```
root@ubuntu:/home/elf# source ~/miniforge3/bin/activate # Miniforge 安装的目录
```

创建一个名为 RKNN-Toolkit2 且 Python 版本为 3.8（建议版本）的 Conda 环境:

```
(base) root@ubuntu:/home/elf# conda create -n RKNN-Toolkit2 python=3.8
```

进入 RKNN-Toolkit2 Conda 环境:

```
(base) root@ubuntu:/home/elf# conda activate RKNN-Toolkit2
```

6.2.1.4. 安装 RKNN-Toolkit2

这里采用在线的方式安装 RKNN-Toolkit2:

```
(RKNN-Toolkit2) root@ubuntu:/home/elf# pip install rknn-toolkit2==2.3.2 -i https://py
pi.tuna.tsinghua.edu.cn/simple
```

使用 pip list 查看 rknn-toolkit2 安装成功:

```
Activities Terminal 10月31日 10:38 root@ubuntu:/home/elf
(RKNN-Toolkit2) root@ubuntu:/home/elf# pip list
Package Version
-----
coloredlogs 15.0.1
fast-histogram 0.13
filelock 3.16.1
flatbuffers 25.9.23
fsspec 2025.3.0
humanfriendly 10.0
Jinja2 3.1.6
MarkupSafe 2.1.5
mpmath 1.3.0
networkx 3.1
numpy 1.24.4
nvidia-cublas-cu12 12.1.3.1
nvidia-cuda-cupti-cu12 12.1.105
nvidia-cuda-nvrtc-cu12 12.1.105
nvidia-cuda-runtime-cu12 12.1.105
nvidia-cudnn-cu12 9.1.0.70
nvidia-cufft-cu12 11.0.2.54
nvidia-curand-cu12 10.3.2.106
nvidia-cusolver-cu12 11.4.5.107
nvidia-cuspars-cu12 12.1.0.106
nvidia-nccl-cu12 2.20.5
nvidia-nvjitlink-cu12 12.9.86
nvidia-nvtx-cu12 12.1.105
onnx 1.17.0
onnxruntime 1.19.2
opencv-python 4.12.0.88
packaging 25.0
pip 24.3.1
protobuf 4.25.4
psutil 7.1.2
rknn-toolkit2 2.3.2
ruamel.yaml 0.18.16
ruamel.yaml.clib 0.2.8
scipy 1.10.1
setuptools 75.3.0
```




此外，还可以通过以下途径获取 RKNN Toolkit2 v2.3.2 工具链的压缩包：

官方仓库：<https://github.com/airockchip/rknn-toolkit2/tree/v2.3.2>

提供的资料包，路径：[进阶篇之-基于 RK 平台的 AI 模型训练到部署\AI 例程源码\rknn-toolkit2-2.3.2.zip](#)

该工具链包含了瑞芯微官方提供的 Python 开发工具及完整文档，主要用于 RKNN 模型的转换、推理以及性能分析。

6.2.1.5. 安装 RKNN Model Zoo

RKNN Model Zoo 是基于 RKNPU SDK 工具链开发的，并为当前主流算法提供了部署示例。包括使用和推理 RKNN 模型的过程。包含大量预训练模型（如 YOLO、ResNet 等）及转换脚本，方便快速将模型转为 RKNN 格式部署到瑞芯微芯片。我们需要 RKNN Model Zoo 提供的模型转换脚本进行模型转化。

从https://github.com/airockchip/rknn_model_zoo下载rknn_model_zoo源码。

也可以直接使用资料包中提供的源码，路径：[进阶篇之-基于RK平台的AI模型训练到部署\AI例程源码\rknn_model_zoo-main.zip](#)

将 rknn_model_zoo-main.zip 拷贝到开发环境/home/elf 目录下并解压：

```
(RKNN-Toolkit2) root@ubuntu:/home/elf# unzip rknn_model_zoo-main.zip
```

6.2.2. 模型转换

下面进行模型转换操作。

1. 为目录/home/elf/rknn_model_zoo-main/examples/yolov8/model 设置读写权限。

```
(RKNN-Toolkit2) root@ubuntu:~# chmod 777 /home/elf/rknn_model_zoo-main/examples/yolov8/model
```

2. 使用FileZilla客户端将导出的模型文件传输到文件路径/home/elf/rknn_model_zoo-main/examples/yolov8/model下。

3. 执行模型转换

```
(RKNN-Toolkit2) root@ubuntu:~# cd /home/elf/rknn_model_zoo-main/examples/yolov8/python/
(RKNN-Toolkit2) root@ubuntu:/home/elf/rknn_model_zoo-main/examples/yolov8/python# python convert.py ../model/best.onnx rk3576 i8 ../model/best.rknn
```

执行转换命令中选择的平台是 rk3576，若平台是 rk3588 或者 rv1126b，可将转换命令中的 rk3576 改为 rk3588 或 rv1126b；命令中的 i8 为模型量化类型，可选模型量化类型为 i8（INT8 量化）或 fp（浮点型），转换后的模型保存路径为 rknn_model_zoo/examples/yolov8/model/best.rknn。

```
root@ubuntu:/home/elf/rknn_model_zoo-main/examples/yolov8/python# python convert.py ../model/best.onnx rk3576 i8 ../model/best.rknn
I OpFusing 1 : 100% | 100/100 [00:00<00:00, 1284.40it/s]
I OpFusing 0 : 100% | 100/100 [00:00<00:00, 1105.47it/s]
I OpFusing 1 : 100% | 100/100 [00:00<00:00, 978.29it/s]
I OpFusing 2 : 100% | 100/100 [00:00<00:00, 277.24it/s]
W build: found outlier value, this may affect quantization accuracy
      const name      abs_mean      abs_std      outlier value
      model.0.conv.weight      2.61      2.68      -18.908
      model.22.cv3.2.1.conv.weight      0.09      0.15      -10.684
      model.22.cv3.1.1.conv.weight      0.12      0.20      14.334, 14.295
      model.22.cv3.0.1.conv.weight      0.18      0.21      -12.122
I GraphPreparing : 100% | 161/161 [00:00<00:00, 3671.77it/s]
I Quantizing : 100% | 161/161 [00:04<00:00, 38.87it/s]
W build: The default output dtype of 'Images' is changed from 'float32' to 'int8' in rknn model for performance!
      Please take care of this change when deploy rknn model with Runtime API!
W build: The default output dtype of '318' is changed from 'float32' to 'int8' in rknn model for performance!
      Please take care of this change when deploy rknn model with Runtime API!
W build: The default output dtype of 'onnx::ReduceSum_326' is changed from 'float32' to 'int8' in rknn model for performance!
      Please take care of this change when deploy rknn model with Runtime API!
W build: The default output dtype of '331' is changed from 'float32' to 'int8' in rknn model for performance!
      Please take care of this change when deploy rknn model with Runtime API!
W build: The default output dtype of '338' is changed from 'float32' to 'int8' in rknn model for performance!
      Please take care of this change when deploy rknn model with Runtime API!
W build: The default output dtype of 'onnx::ReduceSum_346' is changed from 'float32' to 'int8' in rknn model for performance!
      Please take care of this change when deploy rknn model with Runtime API!
W build: The default output dtype of '350' is changed from 'float32' to 'int8' in rknn model for performance!
      Please take care of this change when deploy rknn model with Runtime API!
W build: The default output dtype of '357' is changed from 'float32' to 'int8' in rknn model for performance!
      Please take care of this change when deploy rknn model with Runtime API!
W build: The default output dtype of 'onnx::ReduceSum_365' is changed from 'float32' to 'int8' in rknn model for performance!
      Please take care of this change when deploy rknn model with Runtime API!
W build: The default output dtype of '369' is changed from 'float32' to 'int8' in rknn model for performance!
      Please take care of this change when deploy rknn model with Runtime API!
I rknn building ...
done
--> Export rknn model
done
(RKNN-Toolkit2) root@ubuntu:/home/elf/rknn_model_zoo-main/examples/yolov8/python#
```




6.3. 模型部署

将模型部署到开发板上，通过 NPU 进行模型推理，板端模型推理支持 Python 和 C 两种接口，其中 RKNN-Toolkit Lite2 为用户提供板端模型推理的 Python 接口，方便用户使用 Python 语言进行 AI 应用开发，以下以使用 Python 语言进行部署推理为例进行讲解。

6.3.1. RKNN-Toolkit-Lite2 安装

模型部署需依赖 RKNN-Toolkit-Lite2 工具，本文档中不同开发板型号对应不同版本要求：其中 ELF 2_Lite 和 ELF-RV1126B 开发板需使用 RKNN-Toolkit-Lite2v2.3.2 版本，而 ELF 2 开发板则需搭配 RKNN-Toolkit-Lite2v2.1.0 版本，通过该工具可实现 RKNN 格式模型在对应嵌入式平台上的加载与推理，完成模型部署流程。以下以 ELF 2_Lite 开发板和 ELF 2 开发板为例进行模型部署中的环境搭建，说明如何安装相应版本的 RKNN-Toolkit-Lite2 工具。ELF-RV1126B 开发板的部署环境搭建与 ELF 2_Lite 开发板一致，安装对应版本的 RKNN-Toolkit-Lite2 工具即可。

对于 ELF 2_Lite 开发板使用下列命令进行安装。

这里采用在线的方式安装 RKNN-Toolkit-Lite2：

```
root@elf2l-desktop:~# python3 -m venv elf-env
root@elf2l-desktop:~# source elf-env/bin/activate
(elf-env) root@elf2l-desktop:~# pip install rknn-toolkit-lite2==2.3.2 -i https://pypi.tuna.tsinghua.edu.cn/simple
```

也可以使用资料包中提供的库进行安装，路径：[进阶篇之-基于 RK 平台的 AI 模型训练到部署\AI 例程源码\rknn_toolkit_lite2-2.3.2-cp312-cp312-manylinux_2_17_aarch64.manylinux2014_aarch64.whl](#)

将上述文件拷贝到板端，执行下面命令进行安装。

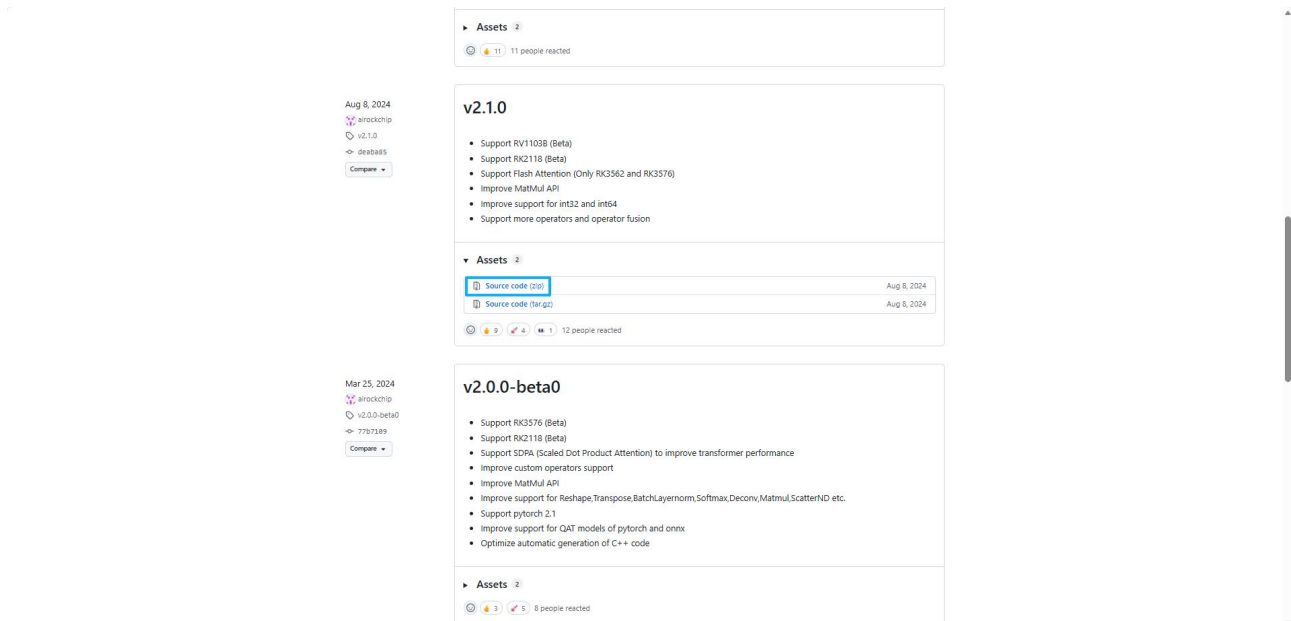
```
(elf-env) root@elf2l-desktop:~# pip install rknn_toolkit_lite2-2.3.2-cp312-cp312-manylinux_2_17_aarch64.manylinux2014_aarch64.whl -i https://mirrors.aliyun.com/pypi/simple/
```

对于 ELF 2 开发板使用下面方法安装。

```
elf@elf2l-desktop:~# sudo apt update
elf@elf2l-desktop:~# sudo apt install python3-pip
```

从以下网址下载 AI 推理工具库：

<https://github.com/airockchip/rknn-toolkit2/releases>



也可以使用 ELF 2 资料包中提供的库，路径：[进阶篇之-基于 RK 平台的 AI 模型训练到部署\AI 例程源码\rknn_toolkit_lite2-2.1.0-cp310-cp310-linux_aarch64.whl](#)

将上述文件拷贝到板端，执行下面命令进行安装。

```
elf@elf2-desktop:~# sudo pip install rknn_toolkit_lite2-2.1.0-cp310-cp310-linux_aarch64.whl -i https://mirrors.aliyun.com/pypi/simple/
```

6.3.2. 编写推理代码

编写推理代码是模型部署流程中连接“模型文件”与“实际应用场景”的关键环节，其核心目的是将训练好的 RKNN 格式模型转化为可在嵌入式设备上实时运行的功能模块，推理代码通过调用 RKNN-Toolkit-Lite2 工具提供的 API，完成模型在 ELF 系列开发板上的加载、初始化与推理计算。推理代码已整理好可直接使用，这里只对代码的主要部分展开介绍。

以下是主程序 main_camera_fps_v8.py。

```
import cv2
import time
# 图像处理函数，实际应用过程中需要自行修改
from rknnpool.rknnpool_Id import rknnPoolExecutor
from func.func_yolov8_optimize import myFunc
out_win = "output_style_full_screen"
cap = cv2.VideoCapture("/dev/video25")
modelPath = "./rknnModel/best.rknn"
# 线程数，增大可提高帧率
TPEs = 8

# 初始化 rknn 池
pool = rknnPoolExecutor(
    rknnModel=modelPath,
    TPEs=TPEs,
    func=myFunc
)
```



```
# 初始化异步所需要的帧
if (cap.isOpened()):
    for i in range(TPEs + 1):
        ret, frame = cap.read()
        if not ret:
            cap.release()
            del pool
            exit(-1)
        pool.put(frame)

frames, loopTime, initTime = 0, time.time(), time.time()
while (cap.isOpened()):
    frames += 1
    ret, frame = cap.read()
    if not ret:
        break
    pool.put(frame)
    frame, flag = pool.get()
    if flag == False:
        break
    cv2.namedWindow(out_win, cv2.WINDOW_NORMAL)
    cv2.setWindowProperty(out_win, cv2.WND_PROP_FULLSCREEN,
cv2.WINDOW_FULLSCREEN)
    frame = cv2.resize(frame, (1280, 800))
    cv2.imshow(out_win, frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
    if frames % 30 == 0:
        print("30 帧平均帧率:\t", 30 / (time.time() - loopTime), "帧")
        loopTime = time.time()

print("总平均帧率\t", frames / (time.time() - initTime))
# 释放 cap 和 rknn 线程池
cap.release()
cv2.destroyAllWindows()
pool.release()
```

代码首先引入基础工具库，cv2用于视频流的捕获与可视化输出，time用于计时与帧率统计；核心功能组件包括rknnPoolExecutor（RKNN多线程池执行器，封装了并行推理逻辑）和myFunc（目标检测流程的核心处理函数，负责将输入图像经过预处理、模型推理、后处理到结果绘制的完整流程）。

然后进行参数配置，包括视频输入源（/dev/video25，对应嵌入式平台的摄像头设备文件）、RKNN模型路径（./rknnModel/best.rknn，转换好的RKNN模型）、线程池规模（TPEs=8，基于硬件算力配置的并行线程数）及输出窗口名称（out_win）。参数设计需根据实际硬件环境（如摄像头接口、开发板NPU性能）进行适配调整。



完成参数配置后进行线程池初始化，通过`rknnPoolExecutor`实例化线程池，传入模型路径、线程数及处理函数`myFunc`，完成多线程推理环境的搭建。为优化异步处理效率，初始化阶段预填充`TPEs+1`帧视频数据至任务队列，确保线程启动时无空闲等待，提升系统响应速度。

异步处理主循环通过`cap.read()`持续读取视频帧，并通过`pool.put()`提交至线程池；同时通过`pool.get()`获取处理完成的帧，经尺寸调整（`cv2.resize`）后全屏显示（`cv2.WINDOW_FULLSCREEN`）。为量化系统性能，每30帧计算一次平均帧率，最终输出全程平均帧率，用于评估实时性。

最后，程序终止时通过`cap.release()`释放视频捕获资源，`cv2.destroyAllWindows()`关闭显示窗口，`pool.release()`销毁线程池，确保硬件资源（摄像头、NPU）与内存的正常回收，避免资源泄露。

以下是主程序中`rknnPoolExecutor`的具体实现`rknnpool_Id.py`。

```
from queue import Queue
from rknnlite.api import RKNNLite
from concurrent.futures import ThreadPoolExecutor, as_completed

def initRKNN(rknnModel="./rknnModel/best.rknn", id=0):
    rknn_lite = RKNNLite()
    ret = rknn_lite.load_rknn(rknnModel)
    if ret != 0:
        print("Load RKNN rknnModel failed")
        exit(ret)
    if id == 0:
        ret = rknn_lite.init_runtime(core_mask=RKNNLite.NPU_CORE_0)
    elif id == 1:
        ret = rknn_lite.init_runtime(core_mask=RKNNLite.NPU_CORE_1)
    elif id == -1:
        ret = rknn_lite.init_runtime(core_mask=RKNNLite.NPU_CORE_0_1)
    else:
        ret = rknn_lite.init_runtime()
    if ret != 0:
        print("Init runtime environment failed")
        exit(ret)
    print(rknnModel, "\t\tdone")
    return rknn_lite

def initRKNNs(rknnModel="./rknnModel/best.rknn", TPEs=1):
    rknn_list = []
    for i in range(TPEs):
        rknn_list.append(initRKNN(rknnModel, i % 2))
    return rknn_list

class rknnPoolExecutor():
    def __init__(self, rknnModel, TPEs, func):
        self.TPEs = TPEs
```



```
self.queue = Queue()
self.rknnPool = initRKNNs(rknnModel, TPEs)
self.pool = ThreadPoolExecutor(max_workers=TPEs)
self.func = func
self.num = 0

def put(self, frame):
    self.queue.put(self.pool.submit(
        self.func, self.rknnPool[self.num % self.TPEs], frame))
    self.num += 1

def get(self):
    if self.queue.empty():
        return None, False
    temp = []
    temp.append(self.queue.get())
    for frame in as_completed(temp):
        return frame.result(), True

def release(self):
    self.pool.shutdown()
    for rknn_lite in self.rknnPool:
        rknn_lite.release()

def get_num(self):
    return self.num
```

代码实现了基于 RKNN-Lite2 的多线程推理池（rknnPoolExecutor），旨在通过并行计算与 NPU 核心分配优化嵌入式平台的神经网络推理效率，首先定义 RKNN 模型初始化函数 initRKNN，通过 RKNNLite 类的 load_rknn 方法加载模型文件；根据 id 参数指定推理使用的 NPU 核心数，支持单核心（NPU_CORE_0 / NPU_CORE_1）、双核心（NPU_CORE_0_1）或默认分配模式，实现对嵌入式平台 NPU 计算资源的精细化调度。由于不同开发板的硬件架构存在差异（ELF 2_Lite、ELF-RV1126B、ELF 2 型号的 NPU 核心数量不同），initRKNN 函数需针对具体硬件进行适配调整，这里只以 ELF 2_Lite 为例，在实际使用过程中需要根据需求对 id 参数进行调整以充分利用 NPU 资源。

然后，定义 RKNN 模型初始化函数 initRKNNs，通过循环调用 initRKNN，生成 TPEs 个模型实例，并通过 i%2 实现 NPU 核心分配的轮询策略（均衡使用不同核心），避免单一核心负载过高。

最后，定义线程池管理类（rknnPoolExecutor），封装了线程池、任务队列与 RKNN 模型池，实现推理任务的异步提交与结果获取。

6.3.3. 板端运行效果

由于 ELF 2_Lite 和 ELF-RV1126B 开发板需使用 RKNN-Toolkit-Lite2v2.3.2 版本，而 ELF 2 开发板则需搭配 RKNN-Toolkit-Lite2v2.1.0 版本，所以，ELF 2_Lite 和 ELF-RV1126B 开发板板端运行方式相似，ELF-RV1126B 开发板板端运行方式可仿照 ELF 2_Lite 开发板板端运行方式运行。ELF 2_Lite 和 ELF 2 开发板板端运行方式不同，下面对这两种开发板的运行方式进行讲解。

对于 ELF 2_Lite 开发板：



激活部署 RKNN-Toolkit-Lite2 工具的环境

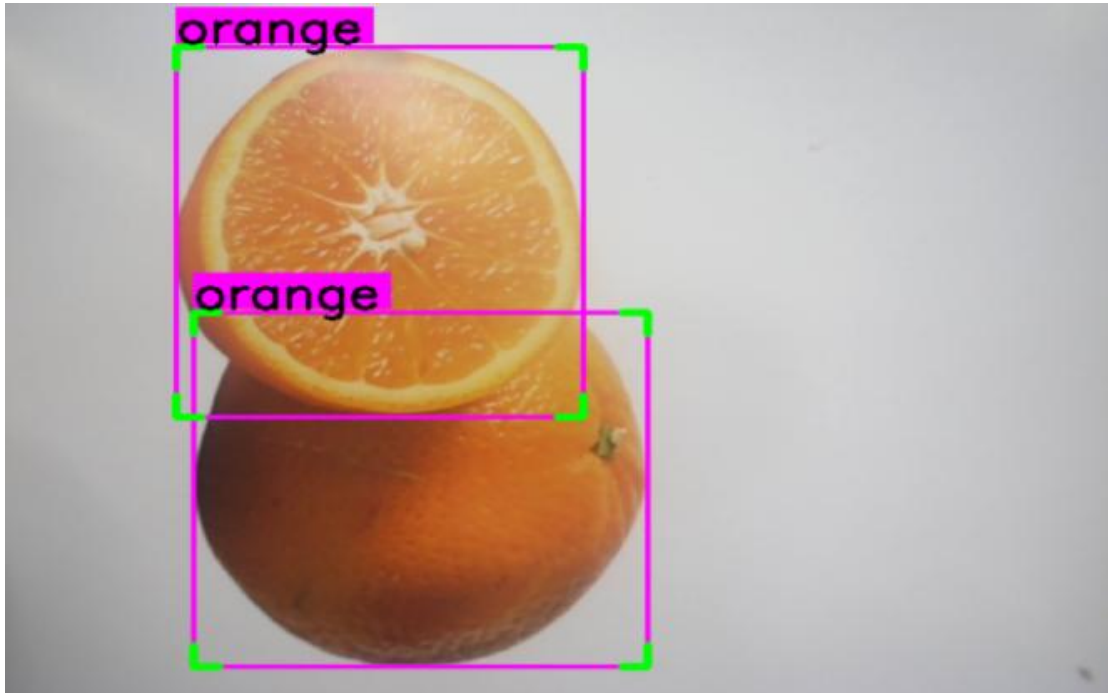
```
root@elf2l-desktop:~# source /elf-env/bin/activate
```

安装 opencv-python

```
(elf-env) root@elf2l-desktop:~# pip install opencv-python -i https://pypi.tuna.tsinghua.edu.cn/simple
```

将[进阶篇之-基于 RK 平台的 AI 模型训练到部署\AI 例程源码\ELF 2_Lite](#)路径下的 demo 文件夹拷贝到板卡的/root 路径下，执行下列命令：

```
(elf-env) root@elf2l-desktop:~# cd demo
(elf-env) root@elf2l-desktop:~# python3 main_camera_fps_v8.py
```



对于 ELF 2 开发板：

安装 opencv-python

```
elf@elf2l-desktop:~# pip install opencv-python -i https://pypi.tuna.tsinghua.edu.cn/simple
```

将[进阶篇之-基于 RK 平台的 AI 模型训练到部署\AI 例程源码\ELF 2](#)路径下的 demo 文件夹拷贝到板卡的/home/elf 路径下，执行下列命令：

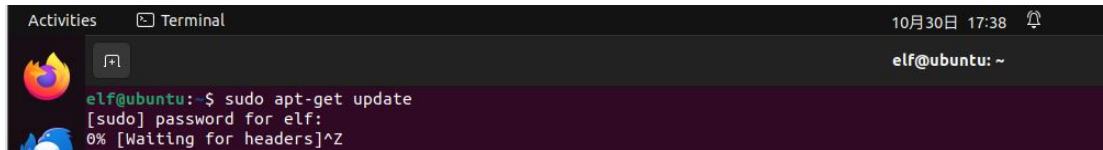
```
elf@elf2l-desktop:~# cd demo
elf@elf2l-desktop:~# export DISPLAY=:0.0           //执行环境变量
elf@elf2l-desktop:~# python3 main_camera_fps_v8.py
```



第七章 FAQ

1. RKNN-Toolkit2 安装，安装 Python 环境。

由于镜像原因造成网络问题



```
elf@ubuntu: ~  
$ sudo apt-get update  
[sudo] password for elf:  
0% [Waiting for headers]^Z
```

解决办法：

替换为国内源，这里使用的是阿里源。

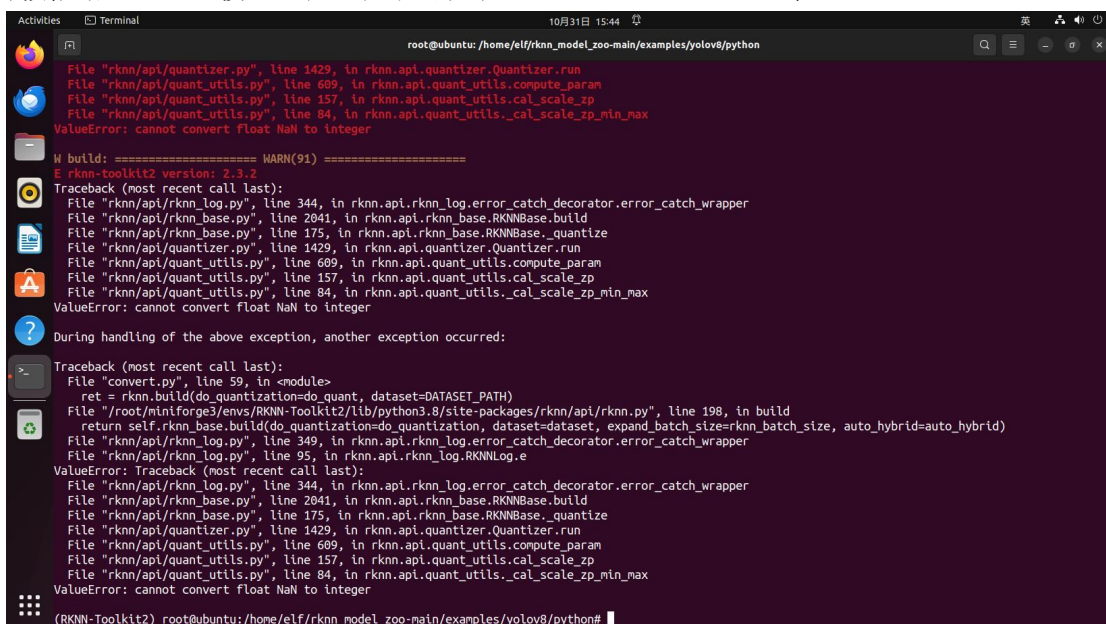
用文本编辑器打开配置文件：

```
sudo gedit /etc/apt/sources.list
```

删除文件中所有内容，粘贴以下国内阿里源：

```
deb http://mirrors.aliyun.com/ubuntu/ jammy main restricted universe multiverse  
deb-src http://mirrors.aliyun.com/ubuntu/ jammy main restricted universe multiverse  
  
deb http://mirrors.aliyun.com/ubuntu/ jammy-security main restricted universe multiverse  
deb-src http://mirrors.aliyun.com/ubuntu/ jammy-security main restricted universe multiverse  
  
deb http://mirrors.aliyun.com/ubuntu/ jammy-updates main restricted universe multiverse  
deb-src http://mirrors.aliyun.com/ubuntu/ jammy-updates main restricted universe multiverse  
  
deb http://mirrors.aliyun.com/ubuntu/ jammy-backports main restricted universe multiverse  
deb-src http://mirrors.aliyun.com/ubuntu/ jammy-backports main restricted universe multiverse
```

2. PyTorch 高版本导致导出的模型权重未内嵌，引发模型转化的时候大量权重未被正确加载，进行 int8 量化的时候报错，RKNN 模型量化过程中出现了 NaN (Not a Number) 值。



```
File "rknn/api/quantizer.py", line 1429, in rknn.api.quantizer.Quantizer.run  
File "rknn/api/quant_utils.py", line 609, in rknn.api.quant_utils.compute_param  
File "rknn/api/quant_utils.py", line 157, in rknn.api.quant_utils.cal_scale_zp  
File "rknn/api/quant_utils.py", line 84, in rknn.api.quant_utils._cal_scale_zp_min_max  
ValueError: cannot convert float NaN to integer  
  
W build: ===== WARN(91) =====  
E rknn-toolkit2 version: 2.3.2  
Traceback (most recent call last):  
File "rknn/api/rknn_log.py", line 344, in rknn.api.rknn_log.error_decorator.error_catch_wrapper  
File "rknn/api/rknn_base.py", line 2041, in rknn.api.rknn_base.RKNNBase.build  
File "rknn/api/rknn_base.py", line 175, in rknn.api.rknn_base.RKNNBase.quantize  
File "rknn/api/quantizer.py", line 1429, in rknn.api.quantizer.Quantizer.run  
File "rknn/api/quant_utils.py", line 609, in rknn.api.quant_utils.compute_param  
File "rknn/api/quant_utils.py", line 157, in rknn.api.quant_utils.cal_scale_zp  
File "rknn/api/quant_utils.py", line 84, in rknn.api.quant_utils._cal_scale_zp_min_max  
ValueError: cannot convert float NaN to integer  
  
During handling of the above exception, another exception occurred:  
  
Traceback (most recent call last):  
File "convert.py", line 59, in <module>  
ret = rknn.build(do_quantization=do_quant, dataset=DATASET_PATH)  
File "/root/miniforge3/envs/RKNN-Toolkit2/lib/python3.8/site-packages/rknn/api/rknn.py", line 198, in build  
return self.rknn_base.build(do_quantization=do_quantization, dataset=dataset, expand_batch_size=rknn_batch_size, auto_hybrid=auto_hybrid)  
File "rknn/api/rknn_log.py", line 349, in rknn.api.rknn_log.error_decorator.error_catch_wrapper  
File "rknn/api/rknn_log.py", line 95, in rknn.api.rknn_log.RKNNLog.e  
ValueError: Traceback (most recent call last):  
File "rknn/api/rknn_log.py", line 344, in rknn.api.rknn_log.error_decorator.error_catch_wrapper  
File "rknn/api/rknn_base.py", line 2041, in rknn.api.rknn_base.RKNNBase.build  
File "rknn/api/rknn_base.py", line 175, in rknn.api.rknn_base.RKNNBase.quantize  
File "rknn/api/quantizer.py", line 1429, in rknn.api.quantizer.Quantizer.run  
File "rknn/api/quant_utils.py", line 609, in rknn.api.quant_utils.compute_param  
File "rknn/api/quant_utils.py", line 157, in rknn.api.quant_utils.cal_scale_zp  
File "rknn/api/quant_utils.py", line 84, in rknn.api.quant_utils._cal_scale_zp_min_max  
ValueError: cannot convert float NaN to integer  
(RKNN-Toolkit2) root@ubuntu:/home/elf/rknn_model_zoo-main/examples/yolov8/python#
```

解决办法：降级 PyTorch 到 2.0.0。



在 PyCharm 中，Alt+F12 调出终端，输入以下命令卸载当前高版本 PyTorch，这里以自己实际安装 anaconda 的位置为准：

```
E:\Anaconda\envs\yolov8\python.exe -m pip uninstall -y torch torchvision torchaudio
```

安装兼容版本：

```
E:\Anaconda\envs\yolov8\python.exe -m pip install torch==2.0.0 torchvision==0.15.1 torchaudio==2.0.1 -i https://pypi.tuna.tsinghua.edu.cn/simple
```

3. 为什么训练出来的 yolov 在虚拟机上可以正常跑，转到到板子上跑出来的图片上都是乱框的。

可从以下两个方面进行问题排查：

（1）模型转换与量化误差：虚拟机中通常使用原始权重（如 PyTorch 格式）或高精度 ONNX 模型推理，而开发板需将模型转换为 RKNN 等嵌入式专用格式，转换过程中若量化策略（如 INT8 量化）不合理，可能导致特征提取精度损失，检测框位置错乱。

（2）预处理与后处理适配问题：虚拟机与开发板的图像预处理逻辑（如尺寸缩放、通道顺序、归一化参数）若存在细微差异，会导致输入模型的特征分布不一致；后处理阶段（如 NMS 阈值、坐标映射公式）若未严格对齐，也会使边界框坐标与原始图像空间错位，形成“乱框”。

4. 只有版本相同的模型才可以使用对应模型版本的转换脚本进行转换嘛？使用 yolo11 训练的模型能直接用 examples/yolov8/python 里面的 convert.py 完成 onnx 到 rknn 模型的转换吗？

并不是只有版本相同的模型才可以使用对应模型版本的转换脚本进行转换，对比 examples/yolov8/python 里面的 convert.py 和 examples/yolov11/python 里面的 convert.py。唯一的区别是目标 RKNN 模型文件路径不同，examples/yolov8/python 里面的 convert.py：

```
import sys
from rknn.api import RKNN

DATASET_PATH = '../..../datasets/COCO/coco_subset_20.txt'
DEFAULT_RKNN_PATH = '../model/yolov8.rknn'
DEFAULT_QUANT = True

def parse_arg():
    if len(sys.argv) < 3:
        print("Usage: python3 {} onnx_model_path [platform] [dtype(optional)] [output_rknn_path(optional)].format(sys.argv[0]));".format(sys.argv[0]));
        print("platform choose from [rk3562, rk3566, rk3568, rk3576, rk3588, rv1126b, rv1109, rv1126, rk1808]");
        print("dtype choose from [i8, fp] for [rk3562, rk3566, rk3568, rk3576, rk3588, rv1126b]");
        print("dtype choose from [u8, fp] for [rv1109, rv1126, rk1808]");
        exit(1)

    model_path = sys.argv[1]
    platform = sys.argv[2]

    do_quant = DEFAULT_QUANT
    if len(sys.argv) > 3:
        model_type = sys.argv[3]
        if model_type not in ['i8', 'u8', 'fp']:
            print("ERROR: Invalid model type: {}".format(model_type))
            exit(1)
        elif model_type in ['i8', 'u8']:
            do_quant = True
        else:
            do_quant = False

    if len(sys.argv) > 4:
        output_path = sys.argv[4]
    else:
        output_path = DEFAULT_RKNN_PATH

    return model_path, platform, do_quant, output_path

if __name__ == '__main__':
    "convert.py" 74 lines, 2179 bytes
```

examples/yolo11/python 里面的 convert.py：



```
import sys
from rknn.api import RKNN

DATASET_PATH = '.././../datasets/COCO/coco_subset_20.txt'
DEFAULT_RKNN_PATH = '../model/yolo11.rknn'
DEFAULT_QUANT = True

def parse_arg():
    if len(sys.argv) < 3:
        print("Usage: python3 {} onnx_model_path [platform] [dtype(optional)] [output_rknn_path(optional)]".format(sys.argv[0]))
        print("        dtype choose from [rk3562, rk3566, rk3568, rk3576, rk3588, rv1126b, rv1109, rv1126, rk1808]")
        print("        dtype choose from [i8, fp] for [rk3562, rk3566, rk3568, rk3576, rk3588, rv1126b]")
        print("        dtype choose from [u8, fp] for [rv1109, rv1126, rk1808]")
        exit(1)

    model_path = sys.argv[1]
    platform = sys.argv[2]

    do_quant = DEFAULT_QUANT
    if len(sys.argv) > 3:
        model_type = sys.argv[3]
        if model_type not in ['i8', 'u8', 'fp']:
            print("ERROR: Invalid model type: {}".format(model_type))
            exit(1)
        elif model_type in ['i8', 'u8']:
            do_quant = True
        else:
            do_quant = False

    if len(sys.argv) > 4:
        output_path = sys.argv[4]
    else:
        output_path = DEFAULT_RKNN_PATH

    return model_path, platform, do_quant, output_path

if __name__ == '__main__':
    "convert.py" 73 lines, 2157 bytes
```

所以，将 examples/yolov8/python 里面的 convert.py 中目标 RKNN 模型文件路径改成使用 yolo11 训练的模型文件路径，才能用 examples/yolov8/python 里面的 convert.py 完成 onnx 到 rknn 模型的转换。但是为了方便起见，使用 yolo11 训练的模型推荐直接使用 examples/yolov11/python 里面的 convert.py 完成 onnx 到 rknn 模型的转换。

5.模型的 pt 文件输出正常，转到 rknn 部署到板子上就会出现很多的“null 框”。

可能的原因及排查方案如下：

（1）模型转换与量化导致的精度丢失

校准数据集（DATASET_PATH 指向的 COCO 子集）与实际部署场景的图像分布差异过大，会导致量化参数校准不准确，引发检测失效。

转换脚本中 rknn.config() 的预处理参数（mean_values/std_values）与模型训练时的预处理逻辑未严格对齐，转换时配置不一致会导致输入模型的特征分布畸变，直接引发检测失效。

（2）预处理/后处理跨平台逻辑不一致

PyTorch 环境与开发板部署环境的处理链路未对齐：预处理环节，二者图像读取格式（PIL 的 RGB 与 OpenCV 的 BGR）、缩放插值方式、letterbox 填充/缩放逻辑可能存在差异，导致输入模型的特征分布畸变或目标框坐标映射偏差；后处理环节，RKNN 模型与 PyTorch 模型的特征图输出维度/顺序可能不同，且量化后未适配 NMS 阈值与置信度阈值，进而引发目标框解码错误或大量低置信度无效框，最终出现“null 框”。

（3）RKNN 工具链版本不一致

确保模型转换使用的 RKNN-Toolkit 版本与开发板上的 RKNN-Toolkit-Lite2 版本一致。



声明

本手册版权归保定飞凌嵌入式技术有限公司所有。未经本公司的书面许可，任何单位和个人无权以任何形式复制、传播、转载本手册的任何部分，违者将被追究法律责任。

保定飞凌嵌入式有限公司所提供的所有服务内容旨在协助用户加速产品的研发进度，在服务过程中所提供的任何程序、文档、测试结果、方案、支持等资料和信息，都仅供参考，用户有权不使用或自行参考修改，本公司不提供任何的完整性、可靠性等保证，若在用户使用过程中因任何原因造成的特别的、偶然的或间接的损失，本公司不承担任何责任。



更多帮助

注意事项与维护

- 请勿带电插拔核心板及外围模块！
- 请遵循所有标注在产品上的警示和指引信息。
- 请保持本产品干燥。如果不慎被任何液体泼溅或浸润，请立刻断电并充分晾干。
- 使用中注意本产品的通风散热，避免温度过高造成元器件损坏。
- 请勿在多尘、脏乱的环境中使用或存放本产品。
- 请勿将本产品应用在冷热交替环境中，避免结露损坏元器件。
- 请勿粗暴对待本产品，跌落、敲打或剧烈晃动都可能损坏线路及元器件。
- 请勿使用有机溶剂或腐蚀性液体清洗本产品。
- 请勿自行修理、拆卸本公司产品，如产品出现故障请及时联系本公司进行维修。
- 擅自修改或使用未经授权的配件可能损坏本产品，由此造成的损坏将不予以保修。



更新通知与学习资源

ElfBoard 产品相关资料会持续完善和更新，本手册内容也会及时跟进。请在使用这些资料时，确保它们为最新版本；最新的资料更新通知可通过微信公众号获取，课程视频和嵌入式知识科普则可以在抖音、哔哩哔哩等平台观看，以帮助大家更好地了解和使用 ElfBoard 产品。



官方公众号



官方抖音



哔哩哔哩

资料获取

1. 请登录 ElfBoard 官网“www.elfboard.com”，进入“资料”专区进行下载；
2. 下载前请阅读《用户资料极速下载必读》。

售后服务政策

1. 如产品使用过程中出现硬件故障可根据售后服务政策进行维修；
2. 售后服务政策：参见官网“www.elfboard.com”首页->技术服务->售后服务政策。

送修地址

1. 地址：河北省保定市高开区向阳北大街 2699 号飞凌嵌入式技术有限公司新楼五层售后维修部
2. 联系人：售后维修部
3. 电话：0312-3102650-952、953
4. 邮编：071000
5. 邮寄须知：建议使用顺丰、圆通或韵达，且不接收任何到付。

技术支持范围

1. 本公司产品的软、硬件资源提供情况咨询；
2. 本公司产品的软、硬件手册使用过程中遇到的问题。

技术讨论范围

1. 源码的修改以及理解；
 2. 操作系统如何移植；
 3. 用户自行修改以及开发中遇到的软硬件问题；
- 注：以上三点虽不属于技术支持范围，但我公司会尽力为用户提供帮助，如仍然没能解决敬请谅解。

技术支持方式

1. 论坛：bbs.elfboard.com
2. QQ 群：474552704

技术支持时间

1. 周一至周五：上午 9:00—11:30；
下午 13:30—17:00；
2. 公司按照国家法定节假日安排休息，在此期间无法提供技术支持，请将问题发送至论坛开发板专区或 QQ 群，我们会在工作日尽快给您回复。